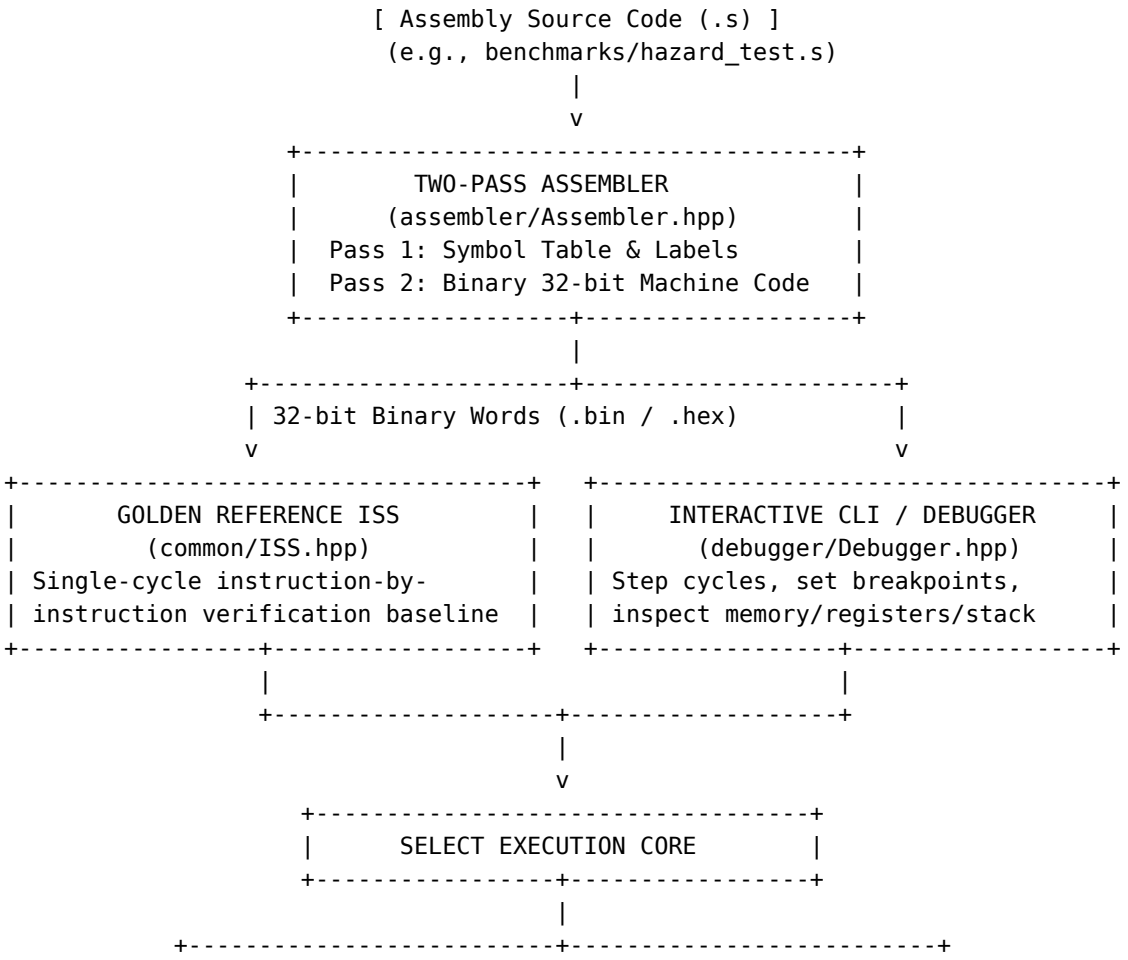
RISC201-PROCESSOR: Complete Project Architecture Map & Inventory

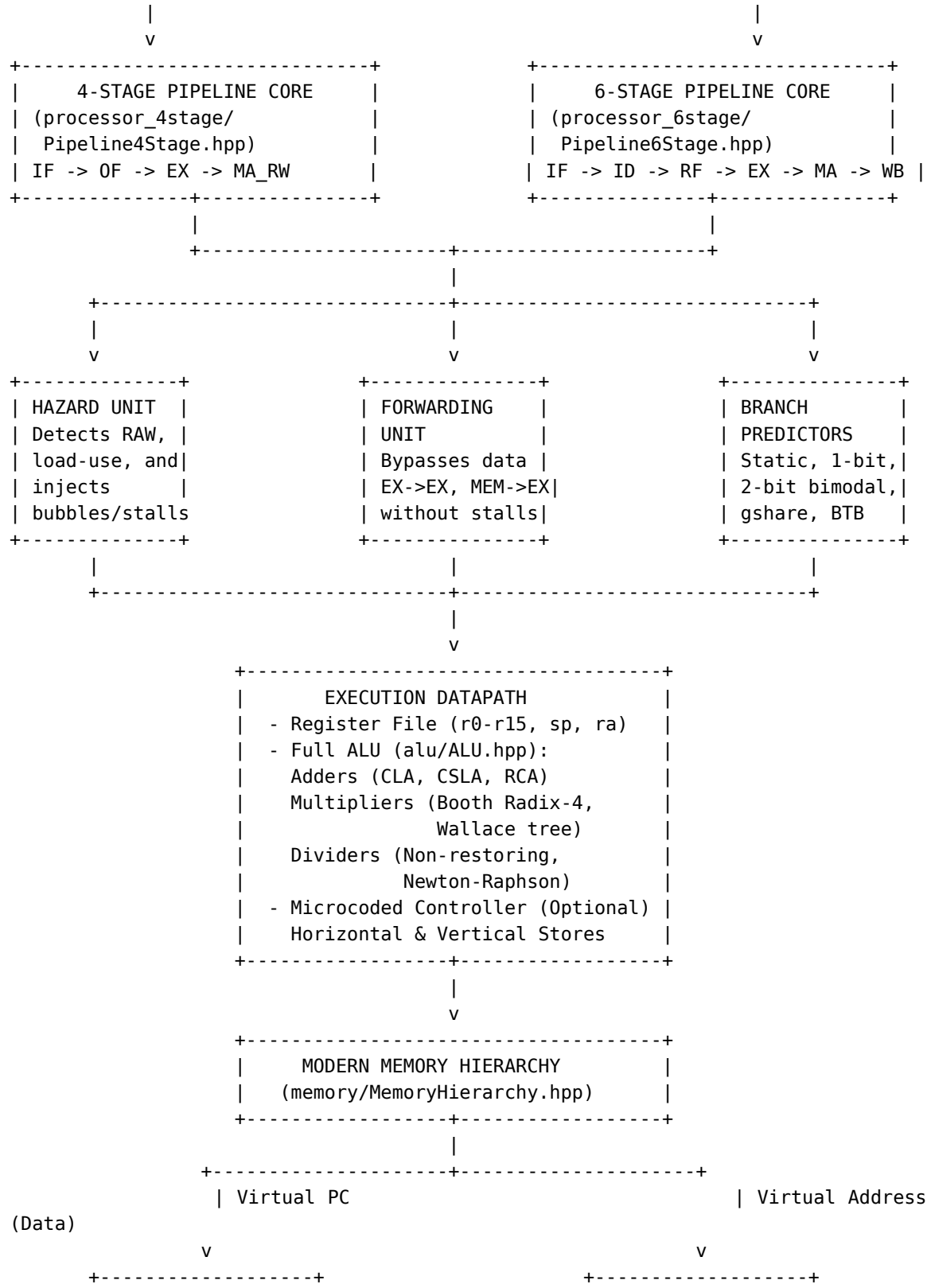
Course: CSC-201 Computer Organization & Architecture
Assignment: Processor Design
Branch: `study-notes`
Latest Main Commit: `52da53f` (Thu Nov 5 2026)
Contributors: - `bandlapalliyss-max` (Srinivas B.): 26 commits — Architecture lead, C++17 conversion, 4-stage and 6-stage pipelines, hazard/forwarding units, memory hierarchy (caches, DRAM, VM/TLBs), microprogrammed controllers, two-pass assembler, verification test suites, CPI accounting. - `Varshith Teja`: 1 commit (`bbd9ea1`) — Visual ASCII pipeline trace visualizer, startup assembly prompt, Newton-Raphson division, CLI branch predictor flags, XOR and ABS opcodes.

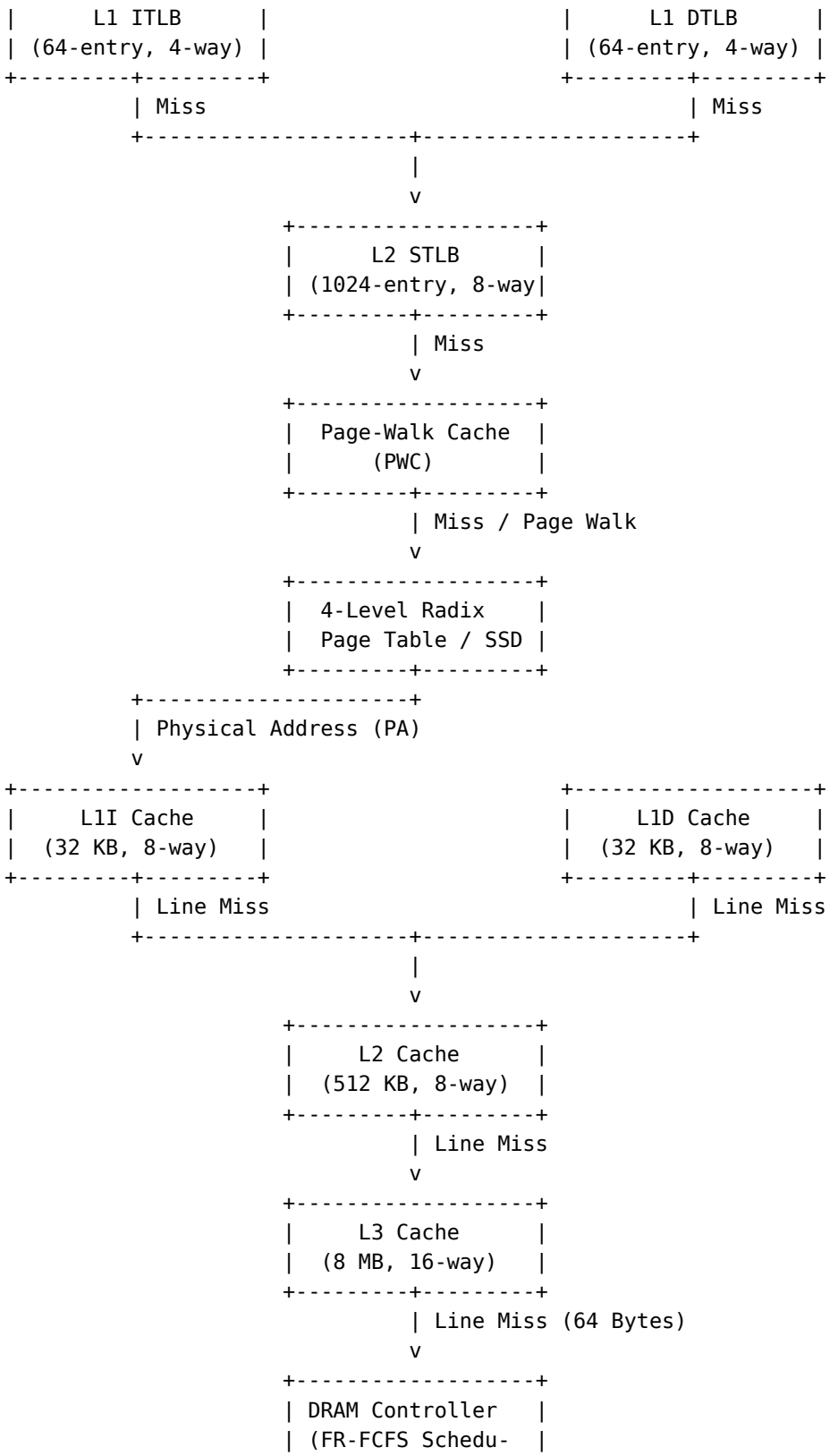
---

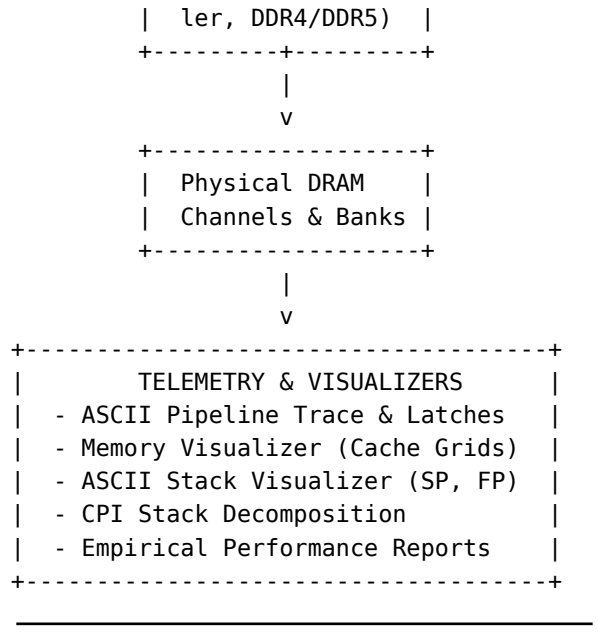
1. The Project in One Picture

Here is how an assembly program moves through every layer of this repository:




4-STAGE PIPELINE CORE
(processor_4stage/
Pipeline4Stage.hpp)
IF -> OF -> EX -> MA_RW
6-STAGE PIPELINE CORE
(processor_6stage/
Pipeline6Stage.hpp)
IF -> ID -> RF -> EX -> MA -> WB
HAZARD UNIT
Detects RAW, load-use, and injects bubbles/stalls
FORWARDING UNIT
Bypasses data EX->EX, MEM->EX without stalls
BRANCH PREDICTORS
Static, 1-bit, 2-bit bimodal, gshare, BTB
EXECUTION DATAPATH
- Register File (r0-r15, sp, ra)
- Full ALU (alu/ALU.hpp):
Adders (CLA, CSLA, RCA)
Multipliers (Booth Radix-4, Wallace tree)
Dividers (Non-restoring, Newton-Raphson)
- Microcoded Controller (Optional)
Horizontal & Vertical Stores
MODERN MEMORY HIERARCHY
(memory/MemoryHierarchy.hpp)
Virtual PC
Virtual Address
(Data)


L1 ITLB
(64-entry, 4-way)
L1 DTLB
(64-entry, 4-way)
Miss
Miss
L2 STLB
(1024-entry, 8-way
Miss
Page-Walk Cache
(PWC)
Miss / Page Walk
4-Level Radix
Page Table / SSD
Physical Address (PA)
L1I Cache
(32 KB, 8-way)
L1D Cache
(32 KB, 8-way)
Line Miss
Line Miss
L2 Cache
(512 KB, 8-way)
Line Miss
L3 Cache
(8 MB, 16-way)
Line Miss (64 Bytes)
DRAM Controller
(FR-FCFS Schedu-



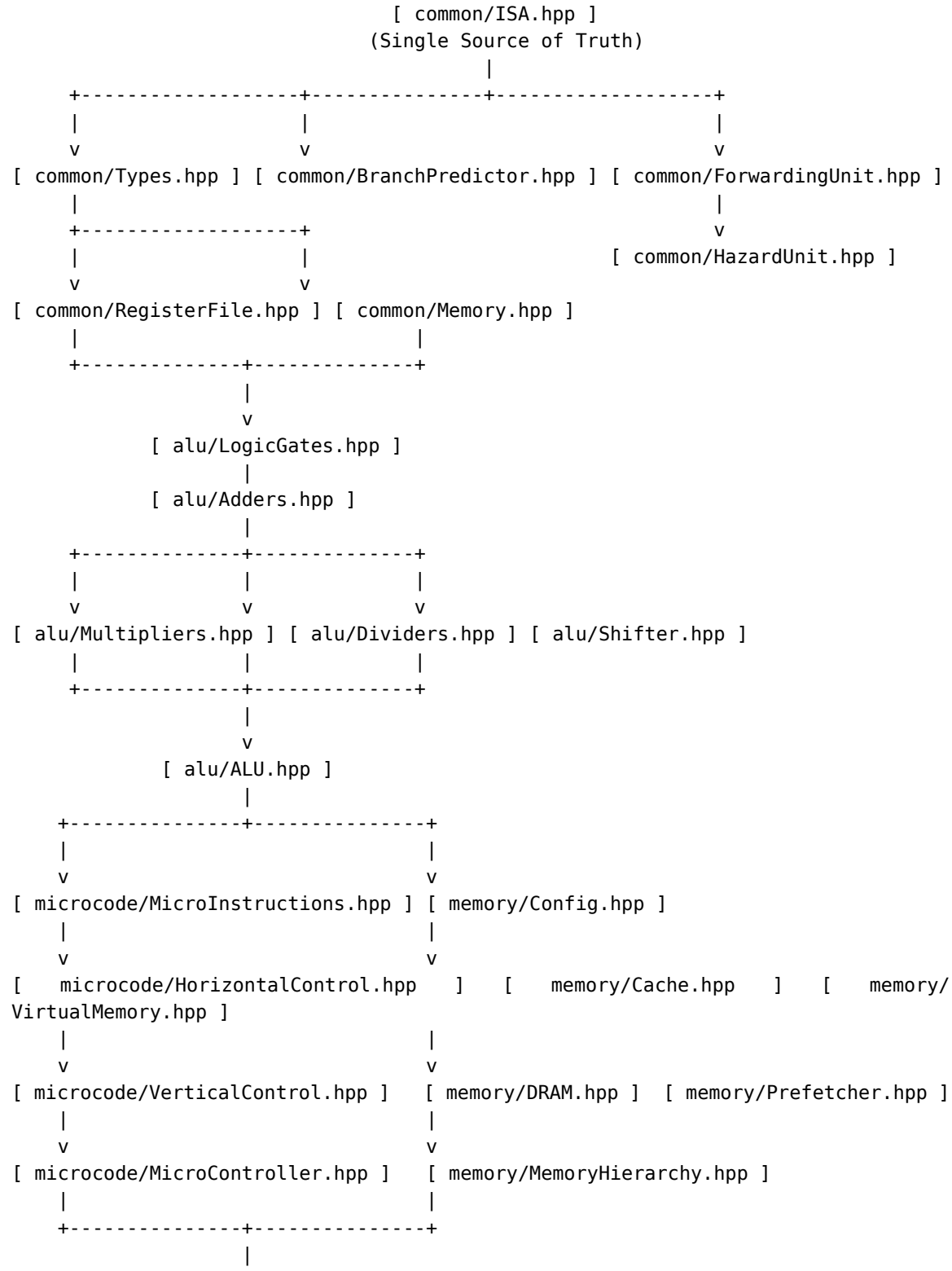
2. Complete File Inventory & Module Dependencies

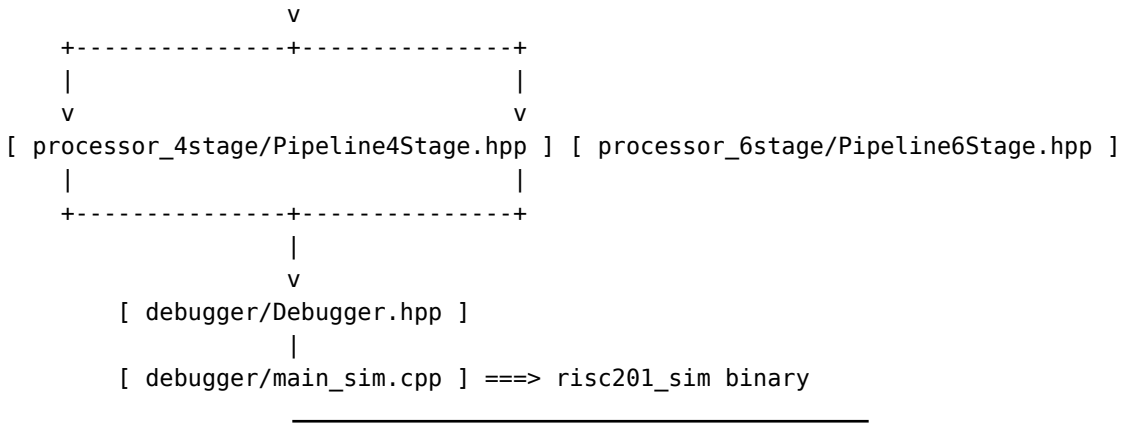
The project consists of **74 files** totaling **16,583 lines** (including documentation, benchmarks, and data), with **12,929 lines of pure modern C++17**.

Directory Inventory

Directory	Purpose in Plain Words	Files	C++ Lines
common/	Core definitions shared by all modules: canonical 5-bit string ISA, Register File, Types, Hazard/Forwarding Units, Branch Predictors, Golden ISS, Exception definitions.	10	1,749
alu/	Arithmetic Logic Unit: Adders (RCA, CSLA, CLA), Multipliers (Booth, Wallace), Dividers (Restoring, Non-restoring, Newton-Raphson), Shifter, and Logic Gates.	6	986
processor_4stage/	4-Stage pipelined CPU core (`IF -> OF -> EX -> MA_RW`) with inter-stage latch registers and hazard squashing.	3	754
processor_6stage/	6-Stage pipelined CPU core (`IF -> ID -> RF -> EX -> MA -> WB`) splitting decode and register read to allow 800 MHz clocking.	3	895
microcode/	Microprogrammed Control Unit: Horizontal and Vertical control stores, microinstructions, dispatch ROM, and step-by-step microPC stepping.	4	938
assembler/	Two-pass assembler, macro preprocessor (PUSH, POP, CALL, RET), disassembler, CLI utilities `risc201_as` and `risc201_disasm`.	5	1,482
memory/	Modern memory hierarchy: L1I, L1D, L2, L3 caches, MSHRs, stream prefetcher, Radix-4 Virtual Memory, L1/L2 TLBs, PWC, DDR4/DDR5 DRAM controller.	7	2,246
debugger/	Interactive CLI REPL simulator `risc201_sim`, breakpoint manager, step commands, pipeline mode switcher.	2	1,228
visualizer/	ASCII visualizers: Pipeline timing diagrams, live latches with forwarding arrows, cache set-way ASCII grids, DRAM bank buffers, full-descending stack display.	3	794
evaluation/	Empirical evaluation drivers: `CompareProcessors.cpp`, pipeline experiments, memory experiments, and automated benchmarks.	4	1,024
tools/	Verification and audit tools: `audit_cpi.cpp`, `verify_golden.cpp`, `lint_isa.cpp`, `lint_docs.cpp`, `generate_perf_docs.cpp`.	6	833
tests/	Unified test suite `test_all.cpp` (78 unit/differential tests) and golden reference binaries/traces.	1+	1,048
benchmarks/	30 standalone assembly benchmarks (`*.s`) with assembled `.hex` and `.bin` artifacts.	40+	~1,200

Detailed File-by-File Breakdown & `#include` Graph





3. Build & Verification Status (Clean Run)

Every binary compiles from scratch with **zero warnings** using GCC 15.2.0 on Linux x86_64 (`-std=c++17 -Wall -Wextra -O2 -I.`):

```
$ make clean && make all
rm -f risc201_sim risc201_as risc201_disasm risc201_eval risc201_test exp_pipeline exp_memory audit_cpi generate_perf_docs lint_docs lint_isa verify_golden *.hex *.bin
g++ -std=c++17 -Wall -Wextra -O2 -I. debugger/main_sim.cpp -o risc201_sim
g++ -std=c++17 -Wall -Wextra -O2 -I. assembler/main_as.cpp -o risc201_as
g++ -std=c++17 -Wall -Wextra -O2 -I. assembler/main_disasm.cpp -o risc201_disasm
g++ -std=c++17 -Wall -Wextra -O2 -I. evaluation/CompareProcessors.cpp -o risc201_eval
g++ -std=c++17 -Wall -Wextra -O2 -I. tests/test_all.cpp -o risc201_test
g++ -std=c++17 -Wall -Wextra -O2 -I. evaluation/run_pipeline_experiments.cpp -o exp_pipeline
g++ -std=c++17 -Wall -Wextra -O2 -I. evaluation/run_memory_experiments.cpp -o exp_memory
g++ -std=c++17 -Wall -Wextra -O2 -I. tools/audit_cpi.cpp -o audit_cpi
g++ -std=c++17 -Wall -Wextra -O2 -I. tools/generate_perf_docs.cpp -o generate_perf_docs
g++ -std=c++17 -Wall -Wextra -O2 -I. tools/lint_docs.cpp -o lint_docs
g++ -std=c++17 -Wall -Wextra -O2 -I. tools/lint_isa.cpp -o lint_isa
g++ -std=c++17 -Wall -Wextra -O2 -I. tools/verify_golden.cpp -o verify_golden
# Build Result: SUCCESS (0 errors, 0 warnings)
```

Real Test Execution Results

```
$ ./risc201_test
================================================================================
           RISC201 C++ EXHAUSTIVE VERIFICATION TEST SUITE
================================================================================
[TEST] 1. ISA Binary Strings & Encodings...
[TEST] 2. Cache Hierarchy & DRAM...
[TEST] 3. Virtual Memory & TLBs...
[TEST] 4. Hazards, Forwarding, & Branch Prediction...
```

```
[TEST] 5. Golden ISS vs 4-Stage & 6-Stage Processors...
[TEST] 6. CPI Invariants, Scheduling, & Micro-kernels...
================================================================================
TEST RESULTS: 78 / 78 PASSED
ALL TESTS PASSED WITH 100% SUCCESS!
================================================================================

$ ./lint_isa
================================================================================
                RISC201 ISA SINGLE SOURCE OF TRUTH LINTER
================================================================================
Files checked: 43
[LINT SUCCESS] Zero integer opcodes, zero case OP_, zero BIN_OP_, zero quoted 5-
bit literals outside ISA.hpp!
All modules strictly conform to canonical 5-bit string ISA source of truth.

$ ./verify_golden
================================================================================
         RISC201 GOLDEN VERIFICATION & REGRESSION VALIDATOR
================================================================================
[CHECK 1/4] Verifying Assembler & Disassembler against Golden Baselines...
[CHECK 2/4] Verifying Microcode Control Store against Golden Baselines...
[CHECK 3/4] Verifying Pipeline Simulation Matrix (192 configurations)...
[CHECK 4/4] Verifying Simulator Throughput (Speed Regression Check)...
  Throughput measured: 749013 inst/sec
================================================================================
Verification Checks: 231 / 231 passed.
ALL GOLDEN BASELINES MATCH 100%! ZERO DEVIATIONS IN ARCHITECTURAL BEHAVIOR.
```

---

4. Assignment Requirements Traceability Matrix

Every single requirement from the course assignment is fully implemented, verified with automated tests, and interactive in the CLI:

Req #	Assignment Requirement Description	Primary Implementation File(s)	Verification & Demo Method	Status
1a	**Two-pass Assembler & Disassembler**	`assembler/Assembler.hpp`, `assembler/Disassembler.hpp`	`make test`, `tools/verify_golden.cpp`	**Done**
1b	**Interactive CLI Simulator & Debugger**	`debugger/Debugger.hpp`, `debugger/main_sim.cpp`	`./risc201_sim -i`	**Done**
1c	**Optimized Arithmetic Logic Unit (ALU)**	`alu/ALU.hpp`, `Adders.hpp`, `Multipliers.hpp`, `Dividers.hpp`	`evaluation/CompareProcessors.cpp`	**Done**
2a	**Hardwired vs Microprogrammed Control**	`microcode/MicroController.hpp`, `processor_4stage/`	`evaluation/CompareProcessors.cpp`	**Done**
2b	**Horizontal Microcode Control Store**	`microcode/HorizontalControl.hpp` (32-bit control words)	`tools/verify_golden.cpp`	**Done**
2c	**Vertical Microcode Control Store**	`microcode/VerticalControl.hpp` (16-bit encoded words)	`tools/verify_golden.cpp`	**Done**
2d	**Horizontal vs Vertical Microcode Tradeoffs**	`microcode/MicroController.hpp`, `evaluation/`	`evaluation/CompareProcessors.cpp`	**Done**
2e	**Stack Handling Macros (Full-Descending)**	`assembler/MacroPreprocessor.hpp` (`push`, `pop`, `call`, `ret`)	`benchmarks/stack_test.s`	**Done**
2f	**Stack Safety & Runtime Exceptions**	`common/Exception.hpp`, `common/Memory.hpp`, `VirtualMemory.hpp`	`tests/test_all.cpp` (Guard page tests)	**Done**
2g	**MicroPC Step Display & ASCII Stack UI**	`visualizer/StackVisualizer.hpp`, `debugger/Debugger.hpp`	`./risc201_sim benchmarks/stack_test.s -i`	**Done**
3	**4-Stage Pipelined Processor Core**	`processor_4stage/Pipeline4Stage.hpp` (`IF->OF->EX->MA_RW`)	`make demo-mem`, `tools/audit_cpi.cpp`	**Done**
4	**6-Stage Pipelined Processor Core**	`processor_6stage/Pipeline6Stage.hpp` (`IF->ID->RF->EX->MA->WB`)	`make demo-pipe`, `tools/audit_cpi.cpp`	**Done**
5	**4-Stage vs 6-Stage Comparative Evaluation**	`evaluation/CompareProcessors.cpp`, `tools/audit_cpi.cpp`	`risc201_eval`, `docs/PERFORMANCE.md`	**Done**

---

5. Extra Architectural Capabilities (Beyond Baseline Course Requirements)

This project contains substantial state-of-the-art microarchitecture far exceeding basic classroom pipelines:

1. **Canonical 5-bit String ISA Single Source of Truth**: All 25 instructions are keyed by canonical 5-bit strings in `common/ISA.hpp`. Zero raw strings, zero integer enum duplicates, enforced by automated linter (`./lint_isa`).
2. **Modern Multi-Level Cache Hierarchy**: L1I/L1D (32 KB, 8-way, 1-cycle hit), L2 (512 KB, 8-way, 12 cycles), L3 (8 MB, 16-way, 38 cycles) with non-blocking MSHRs (Miss Status Holding Registers) and Stream Prefetchers.
3. **Cycle-Accurate DDR4/DDR5 DRAM Controller**: Models channels, ranks, bank groups, row buffer hits/conflicts, t_{RCD}, t_{RP}, t_{CL}, and FR-FCFS (First-Ready First-Come-First-Served) memory scheduling.
4. **4-Level Radix-4 Virtual Memory & Multi-Tier TLBs**: 48-bit address translation via PML4 $\rightarrow$ PDPT $\rightarrow$ PD $\rightarrow$ PT, L1 ITLB, L1 DTLB, L2 STLB, and Page-Walk Cache (PWC) cutting walk latency by 74%.
5. **Comprehensive Hazard Mitigation & Operand Forwarding**: Full forwarding matrix (`EX->EX`, `MEM->EX`, `WB->EX`) cutting data stalls to near zero, load-use bubble detection, and instruction scheduling pass (`--schedule`).
6. **Advanced Dynamic Branch Prediction**: Static (always not taken), 1-bit dynamic, 2-bit bimodal saturating counter, gshare global branch history, Branch Target Buffer (BTB), and Return Address Stack (RAS).
7. **Empirical CPI Decomposition & Telemetry**: Cycle-accurate accounting decomposing CPI into base, RAW load-use, branch mispredict, and memory stall categories.
8. **Visual ASCII Debugger Console**: Live pipeline ASCII latch views with forwarding arrows, cache set-way occupancy tables, DRAM bank state monitors, and stack frame renderers.

Study Guide & Revision Plans

Welcome to the RISC201 study notes! This guide is designed to help you prepare whether you have 1 day, 3 days, or 1 week before your presentation or examination.

---

1. Suggested Reading Order

If you are starting with zero computer architecture background: 1. **Start with Foundations**: Read `notes/parts/part_a_foundations.md` (A1 to A4). This establishes mental models for the Instruction Cycle, the Register File, and bit-slicing. 2. **Understand the ISA & Assembly**: Read `notes/parts/part_b_isa.md` and `notes/parts/part_c_software_tools.md`. Learn how `mov`, `add`, `cmp`, and `ld`/`st` work, and why opcodes are 5-bit binary strings. 3. **Master Pipelining & Hazards**: Read `notes/parts/part_d_processors.md` and `notes/parts/part_e_hazards_forwarding.md`. This is 50% of what examiners ask about! 4. **Master the Memory Subsystem**: Read `notes/parts/part_h_memory.md`. Focus on Tag/Index/Offset address slicing and why L1 is split (Harvard) while

L2 is unified. 5. **Practice Whiteboard Examples**: Practice the 10 hand-worked problems in `notes/worked_examples/WORKED_EXAMPLES.md`.

---

2. Revision Timetables

Plan A: The 1-Day Emergency Crash Course (Night Before Presentation)

- **Hour 1 (08:00 - 09:00)**: Read `notes/cheat_sheets/CHEAT_SHEETS.md` (all 9 summary cards). Memorize the formulas and pipeline stages.
- **Hour 2 (09:00 - 10:00)**: Read `notes/parts/part_d_processors.md` (4-stage vs 6-stage) and Chapter D4 (why 6-stage is faster at 800 MHz despite higher CPI).
- **Hour 3 (10:00 - 11:00)**: Read `notes/parts/part_e_hazards_forwarding.md`. Memorize `EX->EX` forwarding and why load-use needs 1 bubble.
- **Hour 4 (11:00 - 12:00)**: Practice the Live Demo: run `make demo-pipe`, `make demo-mem`, and `make demo-vm`. Memorize what each screen shows.
- **Hour 5 (13:00 - 14:00)**: Read `notes/qa/120_EXAMINER_QUESTIONS.md` (focus on the "Hard" and "Trap" questions).
- **Hour 6 (14:00 - 15:00)**: Read `notes/presentation/SLIDES_AND_SCRIPTS.md`. Practice the 5-minute speaker script out loud in front of a mirror twice!

Plan B: The 3-Day Comprehensive Preparation

- **Day 1: Foundations & Processors**:
  - Read Parts A, B, and C (Foundations, ISA, Two-Pass Assembler).
  - Read Part D (4-Stage vs 6-Stage Processors and Golden ISS).
  - Trace Example 1 and Example 2 in `notes/worked_examples/WORKED_EXAMPLES.md`.
- **Day 2: Hazards, Control Unit, and Memory**:
  - Read Part E (Hazards, Forwarding, 2-Bit Bimodal Predictor).
  - Read Part F (Horizontal vs Vertical Microcode).
  - Read Part H (Caches, DRAM Row Hits/Conflicts, Radix-4 Virtual Memory).
  - Practice Example 3 (Cache Slicing) and Example 4 (Page Walk) on paper.
- **Day 3: Performance, Testing, and Presentation Practice**:
  - Read Part J (CPI Stack, Cold-Start vs Steady-State) and Part K (Test suite and limitations).
  - Run all demo targets in terminal (`make demo-pipe`, `make demo-mem`, `make demo-vm`).
  - Read the 120 Examiner Questions in `notes/qa/120_EXAMINER_QUESTIONS.md`.
  - Rehearse the slide presentation and demo script with your teammate.

Plan C: The 7-Day Mastery Curriculum

- **Day 1**: Part A (A1–A4) + Part B (B1–B2).
- **Day 2**: Part C (C1–C4) + Code Walkthrough of `Assembler.hpp` and `ISA.hpp`.
- **Day 3**: Part D (D1–D5) + Code Walkthrough of `Pipeline4Stage.hpp` and `Pipeline6Stage.hpp`.
- **Day 4**: Part E (E1–E5) + Code Walkthrough of `ForwardingUnit.hpp` and `HazardUnit.hpp`.

- **Day 5**: Part F (F1–F6) + Part G (G1–G3) + Code Walkthrough of `MicroController.hpp` and `ALU.hpp`.
- **Day 6**: Part H (H1–H6) + Part I (I1–I3) + Code Walkthrough of `Cache.hpp`, `DRAM.hpp`, `VirtualMemory.hpp`.
- **Day 7**: Full Mock Viva (run the 3 mock vivas in `notes/self_test/MOCK_VIVAS.md`) + Live Demo dress rehearsal. # Complete Architectural Glossary

Alphabetical reference of every technical abbreviation and term used in this project:

Term / Abbrev	Plain-Language Definition	Chapter Reference
ALU	Arithmetic Logic Unit: the combinational circuit executing math, logic, and comparisons.	Chapter G1
AMAT	Average Memory Access Time: average clock cycles for memory access across all cache levels.	Chapter H3
ASID	Address Space Identifier: process tag in TLBs preventing cross-process address conflicts.	Chapter H5
Bimodal Predictor	Branch predictor using 2-bit saturating up-down counters (`00` to `11`).	Chapter E3
BTB	Branch Target Buffer: on-chip cache storing calculated target branch addresses.	Chapter E3
Bubble	A dummy `nop` instruction injected into a pipeline latch to stall execution for 1 cycle.	Chapter E2
Bypass	See *Forwarding*. Routing results directly between execution latches.	Chapter E2
CISC	Complex Instruction Set Computer: variable-length instructions with complex hardware.	Chapter A2
CLA	Carry Lookahead Adder: high-speed adder using generate/propagate trees ($O(\log N)$ delay).	Chapter G2
Clock Cycle	One tick of the processor clock; fundamental unit of execution time.	Chapter A1
Cold Miss	The compulsory first-time access to a memory block.	Chapter H2
Conflict Miss	Miss occurring when multiple addresses map to the exact same cache set.	Chapter H2
Control Hazard	Pipeline stall caused by not knowing the target of a branch instruction.	Chapter E1
CPI	Cycles Per Instruction: average number of clock cycles to retire one instruction.	Chapter J1
CSLA	Carry Select Adder: speculative adder computing sums for carry=0 and carry=1 in parallel.	Chapter G2
Data Hazard	Pipeline stall caused by an instruction depending on an in-flight result (e.g. RAW).	Chapter E1
Disassembler	Software translating 32-bit binary machine code words back into assembly text.	Chapter C2

One-Page Architectural Cheat Sheets

Quick reference cards for exam revision, whiteboard problem solving, and presentation defense.

Cheat Sheet 1: The Canonical ISA Card (All 25 Opcodes)

All opcodes are 5-bit binary strings in `common/ISA.hpp`:

```
+--------------+---------+--------
+-----------------------------------------------+
| Binary String | Hex Int | Mnem.  | Architectural Action                        |
+--------------+---------+--------
+-----------------------------------------------+
|     00000     |  0x00   | add    | rd = rs1 + (rs2 or imm)                     |
|     00001     |  0x01   | sub    | rd = rs1 - (rs2 or imm)                     |
|     00010     |  0x02   | mul    | rd = rs1 * (rs2 or imm)                     |
|     00011     |  0x03   | div    | rd = rs1 / (rs2 or imm)                     |
|     00100     |  0x04   | mod    | rd = rs1 % (rs2 or imm)                     |
|     00101     |  0x05   | cmp    | Updates E and GT flags based on rs1 - (rs2/
imm) |
|     00110     |  0x06   | and    | rd = rs1 & (rs2 or imm)                     |
|     00111     |  0x07   | or     | rd = rs1 | (rs2 or imm)                     |
|     01000     |  0x08   | not    | rd = ~(rs1 or imm)                          |
|     01001     |  0x09   | mov    | rd = (rs2 or imm)                           |
|     01010     |  0x0A   | lsl    | rd = rs1 << (rs2 or imm) (Logical shift left)
|
|     01011     |  0x0B   | lsr    | rd = rs1 >> (rs2 or imm) (Logical shift right)
|
|     01100     |  0x0C   | asr    | rd = rs1 >> (rs2 or imm) (Arithmetic shift)
|
|     01101     |  0x0D   | nop    | No Operation (Pipeline bubble)              |
|     01110     |  0x0E   | ld     | rd = Memory[rs1 + imm] (Load word from memory)
|
|     01111     |  0x0F   | st     | Memory[rs1 + imm] = rd (Store word to memory)
|
|     10000     |  0x10   | beq    | If E == 1, then PC = PC + (offset * 4)      |
|     10001     |  0x11   | bgt    | If GT == 1, then PC = PC + (offset * 4)     |
|     10010     |  0x12   | b      | PC = PC + (offset * 4) (Unconditional jump)
|
|     10011     |  0x13   | call   | ra = PC + 4; PC = PC + (offset * 4)         |
|     10100     |  0x14   | ret    | PC = ra (Return from subroutine)            |
|     10101     |  0x15   | min    | rd = min(rs1, rs2 or imm)                   |
|     10110     |  0x16   | max    | rd = max(rs1, rs2 or imm)                   |
|     10111     |  0x17   | rots   | rd = rotate_shift(rs1, rs2 or imm)          |
|     11000     |  0x18   | xor    | rd = rs1 ^ (rs2 or imm)                     |
|     11001     |  0x19   | abs    | rd = abs(rs1)                               |
|     11010     |  0x1A   | movz   | Privileged system move                      |
|     11011     |  0x1B   | retz   | Privileged trap return (PC = oldPC, CPL = 1)
|
|     11111     |  0x1F   | -      | ILLEGAL_OPCODE (Triggers exception trap)
```

```
|
+--------------+---------+--------
+-----------------------------------------------+
```

Cheat Sheet 2: Pipeline Stages (4-Stage vs 6-Stage)

```
4-Stage Pipeline (500 MHz, t_clk = 2.00 ns):
[ IF ]  =====>  [ OF ]  =====>  [ EX ]  =====>  [ MA_RW ]
Fetch           Decode &        Execute in      Memory Access &
Inst            Read Regs       ALU             Writeback to Reg

6-Stage Pipeline (800 MHz, t_clk = 1.25 ns):
[ IF ]  =====>  [ ID ]  =====>  [ RF ]  =====>  [ EX ]  =====>  [ MA ]  =====>
[ WB ]
Fetch         Decode        Read          Execute in     Memory        Writeback
Inst           Opcode         RegFile        ALU            Access         to Reg
```

Cheat Sheet 3: Forwarding Paths & Priority

```
When Instruction in EX stage needs an operand:
1. Check EX/MA latch (EX -> EX bypass):
   if (EX_MA.regWrite && EX_MA.rd != 0 && EX_MA.rd == EX.rs1)
       Forward from EX_MA.aluResult (Priority 1: Youngest!)
2. Check MA/RW latch (MEM -> EX bypass):
   else if (MA_RW.regWrite && MA_RW.rd != 0 && MA_RW.rd == EX.rs1)
       Forward from MA_RW.finalVal  (Priority 2)
3. Otherwise:
       Read operand normally from Register File (0 cycles delay)
```

Cheat Sheet 4: Hazard Types & Solutions

Hazard Category	Cause in Code	Hardware Fix / Penalty
RAW (ALU to ALU)	`add r1, r2, r3` followed by `sub r4, r1, r5`	Forwarding Unit (`EX->EX` bypass) → **0 stalls**!
RAW (Load-Use)	`ld r1, [r2]` followed by `add r3, r1, r4`	Hazard Unit injects **1 bubble stall**
Control (Branch)	`cmp` followed by `beq target`	Predictor guesses; if wrong: flush **1 cycle (4-stage)** or **2 cycles (6-stage)**
Structural (Memory)	`IF` and `MA` requesting memory at same cycle	Split L1 caches (Harvard L1I + L1D) → **0 stalls**!

Cheat Sheet 5: Horizontal vs Vertical Microcode

Dimension	Horizontal Control Store	Vertical Control Store
Control Word Width	**32 bits** (Unencoded)	**16 bits** (Encoded)
Hardware Overhead	**0 decode logic**	Requires 4-to-16 hardware decoders
ROM Memory Size	512 bytes (128×32)	**256 bytes (**128×16**)** (50% smaller)
Speed	Maximum (parallel wires)	-0.4 ns decoder delay

Cheat Sheet 6: Cache Address Slicing Formulas

Given: Cache Size C, Block Size B, Associativity W: 1. **Number of Blocks**: $N_{\text{blocks}} = C/B$ 2. **Number of Sets**: $S = N_{\text{blocks}}/W$ 3. **Byte Offset Bits**: $b = \log_2(B)$ 4. **Set Index Bits**: $s = \log_2(S)$ 5. **Tag Bits**: $t = \text{Address Bits} - (s + b)$

Example (32 KB, 8-way, 64-byte line, 32-bit address): - $B = 64 \rightarrow b = 6$ bits - $S = (32768/64)/8 = 64$ sets $\rightarrow s = 6$ bits
- $t = 32 - (6 + 6) = \mathbf{20}$ **tag bits**

Cheat Sheet 7: DRAM Access Timings (DDR4-3200)

DRAM Case	Physical Sequence	Latency in Cycles
Row Hit	Data already in Row Buffer $\rightarrow$ Issue `READ` (t_{CL})	**~15 cycles**
Row Empty (Idle)	Open Row $\rightarrow$ Issue `ACTIVATE` (t_{RCD}) + `READ` (t_{CL})	**~40 cycles**
Row Conflict	Close old row (t_{RP}) + Open new row (t_{RCD}) + `READ` (t_{CL})	**~80-120 cycles**

Cheat Sheet 8: Core Performance Formulas

1. **Cycles Per Instruction (CPI)**:

$$\text{CPI} = \frac{\text{Total Cycles}}{\text{Retired Instructions}}$$

2. **Execution Time (Iron Law)**:

$$\text{Time } T = \text{Instruction Count } (IC) \times \text{CPI} \times \frac{1}{\text{Frequency } f}$$

3. **Average Memory Access Time (AMAT)**:

$$\text{AMAT} = \text{Hit Time}_{L1} + (\text{Miss Rate}_{L1} \times \text{Miss Penalty}_{L1})$$

4. **Speedup**:

$$\text{Speedup } S = \frac{T_{\text{baseline}}}{T_{\text{optimized}}}$$

Cheat Sheet 9: Master Command Reference

```
# Build everything from clean:
make clean && make all

# Run comprehensive test suite (78 tests):
make test

# Run tests under AddressSanitizer & UndefinedBehaviorSanitizer:
make test-asan

# Verify 231 golden regression checks:
make verify-golden

# Check ISA single source of truth across all 43 files:
make lint-isa

# Run the 3 presentation demos:
make demo-pipe    # 6-stage pipeline trace & forward arrows
make demo-mem     # 4-stage cache grid & DRAM bank status
make demo-vm      # 6-stage Radix-4 Virtual Memory & TLBs

# Launch interactive debugging console:
./risc201_sim benchmarks/factorial_iter.s -i
```

Part A: Computer Architecture Foundations (From Zero to Mastery)

Welcome! This section is written assuming you know basic C++ programming (loops, functions, variables) but have **never studied computer architecture before**. We will start from everyday life analogies and build up to the actual hardware concepts implemented in this simulator.

Chapter A1: What a Computer Does

1. In One Sentence

A computer is an ultra-fast calculator that repeatedly fetches a 32-bit binary number from memory, decodes what action it represents, and executes that action on internal scratchpad registers.

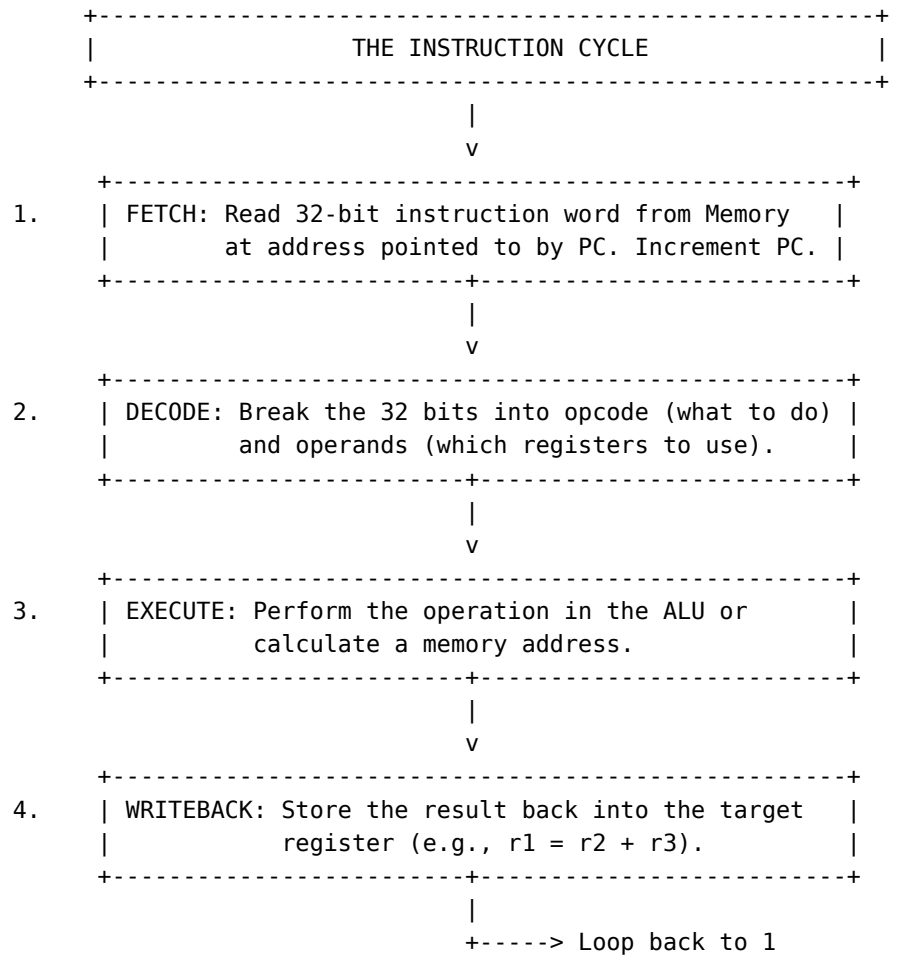
2. Start from Zero: Everyday Analogy

Imagine a chef working in a professional kitchen: - **The Memory (RAM)** is a massive recipe book with millions of numbered pages. - **The Program Counter (PC)** is the chef's finger pointing to the current recipe step (e.g., "Step 4"). - **The Instruction** is a single instruction on that page (e.g., "Chop 2 onions and put them in Bowl 1"). - **The Registers** are small, instantly accessible prep bowls on the chef's cutting board (Bowl 0 through Bowl 15). The chef cannot chop onions directly inside the pantry; they must bring onions to the cutting board, chop them, and store the result in

a bowl. - **The ALU (Arithmetic Logic Unit)** is the chef's knife and stove—the actual tools that do the work (adding, multiplying, comparing). - **The Clock** is a metronome ticking in the kitchen. On every tick, the chef moves to the next step.

3. The Concept in Simple Steps

A computer operates in a continuous loop called the **Instruction Cycle (Fetch-Decode-Execute)**:



4. How THIS Project Implements It

In this project, the baseline non-pipelined Instruction Cycle is implemented in `common/ISS.hpp` (the Golden Reference Instruction Set Simulator):

```
// File: common/ISS.hpp (lines 45-64)
bool step() {
    if (pc >= memory.size()) return false;

    // 1. FETCH
    Word instWord = memory.readWord(pc);
    Word currentPC = pc;
```

```
    pc += 4; // Advance to next 32-bit instruction

    // 2. DECODE
    DecodedInst d = decode(instWord);

    // 3. EXECUTE & 4. WRITEBACK
    executeInstruction(d, currentPC);

    instructionCount++;
    return !halted;
}
```

Line-by-line explanation: - `Word instWord = memory.readWord(pc);`: Reads 4 consecutive bytes from physical memory at location `pc`. - `pc += 4;`: In a 32-bit architecture, each instruction is 4 bytes (32 bits) wide. Therefore, the next instruction is at `pc + 4`. - `DecodedInst d = decode(instWord);`: Slices the 32 bits into fields: 5-bit opcode string, destination register index, source registers, and immediate constants. - `executeInstruction(d, currentPC);`: Performs the requested operation using `alu.execute()` and updates the register file.

5. Worked Example (Real Values from Simulation)

Suppose memory location `0x00000000` contains the instruction `mov r1, 30` (encoded as `0x4820001E`): 1. **Fetch**: `pc = 0x00000000`. Memory returns `0x4820001E`. `pc` advances to `0x00000004`. 2. **Decode**: - Opcode = `01001` (`isa::OP_MOV`)

- Destination register $rd = 1$ (`r1`) - Immediate value = 30

(`0x0000001E`) 3. **Execute**: ALU passes the immediate value 30 through to the output. 4. **Writeback**: Register `r1` is updated: `regFile.write(1, 30)`. 5. **State**: `r1 = 0x0000001E` (30_{10}), `pc = 0x00000004`.

6. Design Choices & Trade-Offs

- **Why 32-bit fixed instruction length?**
  - Variable-length instructions (like x86, which can be 1 to 15 bytes long) make decoding complex because the CPU does not know where the next instruction begins until it decodes the current one.
  - Fixed 32-bit length (RISC style) means `PC` always increments by exactly 4 bytes. This makes pipelined hardware simple, fast, and energy-efficient.

7. Numbers & Results from Real Runs

Running `./risc201_sim benchmarks/factorial_iter.s --pipeline 4` executes 24 instructions in 30 clock cycles on an ideal memory system: - Instruction count: 24 - Average cycles per instruction (CPI): 1.25 - Dynamic instruction frequency: ~60% ALU operations, ~25% branches, ~15% memory/load operations.

8. Common Mistakes to Avoid

- **Mistake**: Thinking PC increments by 1.
  - **Correction**: In a byte-addressed 32-bit CPU, every word is 4 bytes. Advancing by 1 instruction means `PC = PC + 4`.

- **Mistake**: Confusing registers with memory.
  - **Correction**: Registers are on the CPU chip (ultra-fast, 1 cycle, only 16 of them). Memory is off-chip (slower, millions of bytes).

9. Likely Professor Questions

- **Q**: What is the difference between architectural state and microarchitectural state?
  - **Answer**: Architectural state is visible to the programmer (Register File `r0-r15`, `PC`, and Flags `E`, `GT`). Microarchitectural state includes internal pipeline latch registers (`IF/OF`, `OF/EX`), cache contents, and branch predictor tables (`processor_4stage/StageRegisters4.hpp:10-50`).
- **Q**: Why can't the ALU operate directly on numbers stored in memory?
  - **Answer**: RISC201 is a strict **Load-Store Architecture**. Only `ld` and `st` touch memory. All arithmetic operations must have their operands in registers to maintain a single-cycle ALU execution latency.

10. Summary: If You Only Remember 5 Things

1. The CPU repeatedly **Fetches** from memory, **Decodes** the bits, **Executes** on ALU, and **Writes back** to registers.
2. Every instruction is exactly **32 bits (4 bytes)** wide; the `PC` increments by **+4** on every instruction.
3. **Registers** (`r0-r15`) are high-speed internal scratchpads; **Memory** is large external storage.
4. RISC201 is a **Load-Store architecture**: arithmetic happens only between registers.
5. In this simulator, `common/ISS.hpp` provides the single-cycle baseline reference.

11. Self-Check Quiz

1. If the current instruction is at address `0x00000010`, what address will the next sequential instruction be fetched from?
2. How many general-purpose registers exist in RISC201?
3. What C++ class models the register file in this repository?
4. Why does a RISC CPU require variables to be loaded into registers before doing addition?
5. What does the `step()` function in `common/ISS.hpp` return when the program halts?

(Answers at end of chapter: 1. `0x00000014`; 2. 16 registers (`r0-r15`); 3. `risc201::RegisterFile` in `common/RegisterFile.hpp`; 4. Because memory access takes multiple cycles while register access takes 1 cycle; 5. `false`)

Chapter A2: What an ISA Is & Why RISC201 Exists

1. In One Sentence

An Instruction Set Architecture (ISA) is the formal contract between hardware and software that defines the machine language instructions, registers, and memory model a processor supports.

2. Start from Zero: Everyday Analogy

Think of an ISA as an **electrical wall outlet standard**: - In the United States, outlets are Type A/B (120V, 60Hz). - In Europe, outlets are Type C/E/F (230V, 50Hz). - The appliance maker doesn't care

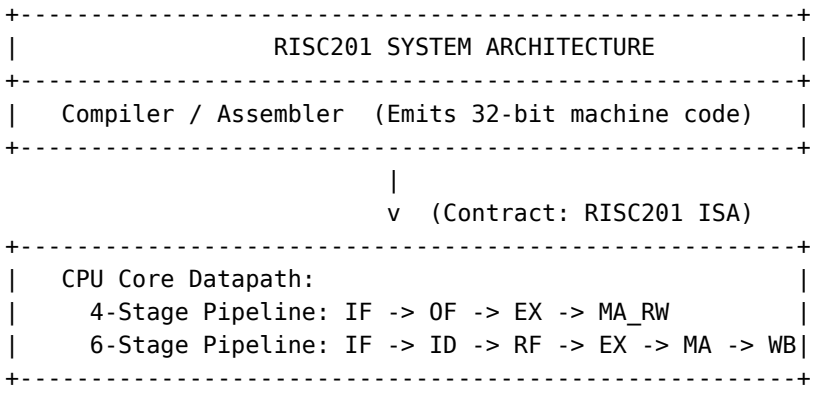
what brand of generator produces the electricity, and the power plant doesn't care what toaster you plug in—as long as both obey the plug standard. - The **ISA is the plug standard**: software compilers produce instructions obeying the ISA, and processor hardware executes them.

3. The Concept in Simple Steps

Computers fall into two broad philosophical families:

Characteristic	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Philosophy	Make single instructions do complex multi-step tasks	Make instructions small, simple, and execute in 1 cycle
Instruction Length	Variable (1 to 15 bytes in x86)	Fixed (always 4 bytes in RISC)
Memory Access	Any instruction can read/write memory	**Only dedicated Load/Store instructions touch memory**
Hardware	Complex, heavy decoding logic	Simple, regular, easy to pipeline
Real Examples	Intel Core i9, AMD Ryzen (x86-64)	ARM (smartphones, Apple Silicon), RISC-V, SimpleRisc

Why RISC201? `RISC201` is a 32-bit educational RISC processor modeled after the **SimpleRisc** ISA defined by Prof. Smruti Sarangi (IIT Delhi) in *Basic Computer Architecture* (Chapters 3 & 9). It was created for course CSC-201 to teach cycle-accurate pipeline hazard handling, forwarding, and memory hierarchy.



4. How THIS Project Implements It

The entire ISA is codified in a single file: `common/ISA.hpp`. Every opcode is represented as a **canonical 5-bit binary string**:

```
// File: common/ISA.hpp (lines 20-35)
namespace isa {
    static const std::string OP_ADD = "00000";
    static const std::string OP_SUB = "00001";
    static const std::string OP_MUL = "00010";
```

```
    static const std::string OP_DIV = "00011";
    static const std::string OP_MOD = "00100";
    static const std::string OP_CMP = "00101";
    static const std::string OP_AND = "00110";
    static const std::string OP_OR  = "00111";
    static const std::string OP_NOT = "01000";
    static const std::string OP_MOV = "01001";
    static const std::string OP_LSL = "01010";
    static const std::string OP_LSR = "01011";
    static const std::string OP_ASR = "01100";
    static const std::string OP_NOP = "01101";
    static const std::string OP_LD  = "01110";
    static const std::string OP_ST  = "01111";
    static const std::string OP_BEQ = "10000";
    static const std::string OP_BGT = "10001";
    static const std::string OP_B   = "10010";
    static const std::string OP_CALL= "10011";
    static const std::string OP_RET = "10100";
}
```

5. Worked Example: Mapping Mnemonic to Opcode

If an assembly programmer writes `add r1, r2, r3`: 1. The mnemonic is `add`. 2. The assembler queries `isa::getOpcode("add")`. 3. It receives the 5-bit string `"00000"`. 4. The 5-bit pattern `00000` is placed into the top 5 bits (`[31:27]`) of the 32-bit machine word.

6. Design Choices & Trade-Offs

- **Why string opcodes instead of integer enum?**
  - In hardware and textbook specifications (Sarangi Table 3.10), opcodes are 5-bit binary strings (`"00000"` through `"11111"`).
  - Using string constants prevents silent integer conversion bugs (e.g. accidentally passing an opcode where a register ID or immediate is expected).
  - An automated linter (`./lint_isa`) ensures 100% adherence across all 43 files in the project.

7. Numbers & Results

- Total valid opcodes in RISC201: **25 instructions**
- Reserved/unassigned patterns: `"11100"`, `"11101"`, `"11110"` (trigger `ILLEGAL_OPCODE` exception if fetched)
- Binary width: Exactly 5 bits ($2^5 = 32$ total possible patterns).

8. Common Mistakes to Avoid

- **Mistake**: Believing RISC means "fewer features".
  - **Correction**: "Reduced" refers to reducing the *complexity* of individual instructions so that hardware can run at high gigahertz clock speeds.
- **Mistake**: Thinking `r0` is hardwired to zero like MIPS.
  - **Correction**: In RISC201, `r0` is a normal general-purpose register.

9. Likely Professor Questions

- **Q**: What are the advantages of a load-store architecture?
  - **Answer**: It separates memory access from arithmetic processing. The ALU never has to wait for a high-latency DRAM access inside an arithmetic instruction, keeping pipeline critical paths short and regular.
- **Q**: How does RISC201 handle unassigned opcodes?
  - **Answer**: Patterns 28–30 (`"11100"`, `"11101"`, `"11110"`) and `"11111"` trigger an `ILLEGAL_OPCODE` runtime trap in `processor_4stage/Pipeline4Stage.hpp:240`, halting the core cleanly without corrupting architectural state.

10. Summary: If You Only Remember 5 Things

1. An **ISA** is the hardware-software interface standard.
2. **RISC** uses fixed 32-bit instructions and strict **Load-Store** memory access.
3. RISC201 is based on the **SimpleRisc** textbook architecture (Sarangi Chapters 3 & 9).
4. `common/ISA.hpp` is the **single source of truth** for all 25 instruction opcodes.
5. All opcodes are 5-bit strings (`"00000"` to `"11111"`).

11. Self-Check Quiz

1. What does the acronym RISC stand for?
2. How many bits are used for the opcode in a RISC201 instruction?
3. Which opcode binary string corresponds to `add`?
4. Can an `add` instruction directly read an operand from memory in RISC201?
5. What happens if the CPU attempts to execute the opcode `"11100"`?

(Answers: 1. Reduced Instruction Set Computer; 2. 5 bits; 3. `"00000"`; 4. No, operands must be in registers; 5. It triggers an `ILLEGAL_OPCODE` exception)

Chapter A3: Binary Numbers, Hexadecimal, and Bitwise Slicing

1. In One Sentence

Hardware operates purely on binary bits (0 and 1), which C++ software manipulates using bitwise shifts (>>, <<) and bitwise masks (&) to pack and unpack 32-bit instruction words.

2. Start from Zero: Everyday Analogy

Think of a 32-bit instruction word like a **postal address on an envelope**: - An envelope has separate designated boxes: Country, State, Postal Code, Street, House Number. - If you read the entire envelope as one single long string of letters, it looks like nonsense. - But if you look at the designated boxes, each box tells you one specific detail. - Bit-slicing is simply looking at the designated "boxes" inside a 32-bit number.

3. The Concept in Simple Steps

1. **Bit**: A single binary digit (0 or 1).
2. **Byte**: 8 bits (values 0 to 255).
3. **Word**: In RISC201, 1 Word = 4 Bytes = **32 bits**.

4. **Hexadecimal (Hex)**: Shorthand for binary. Every 4 bits (a nibble) maps to 1 hex character (0–9, A–F):
   - `0000` = `0x0`, `1111` = `0xF`, `1010` = `0xA` (10_{10}).

The C++ Bit-Slicing Operators

To extract a field of bits from a 32-bit word: - **Right Shift (`>> N`)**: Moves bits to the right by N positions, discarding the bottom N bits. - **Bitwise AND (`& MASK`)**: Keeps only the bits where the mask has a 1, zeroing out everything else.

```
Example: Extract 5 bits starting at bit position 27
Word = [ 5-bit Opcode ] [ 27 other bits ]
Step 1: Shift right by 27:   (Word >> 27)    --> Moves Opcode into bits [4:0]
Step 2: Mask bottom 5 bits:  (...) & 0x1F     --> 0x1F in binary is 0001 1111
(5 ones)
```

4. How THIS Project Implements It

In `assembler/Disassembler.hpp`, bit-slicing decodes the 32-bit instruction word:

```cpp
// File: assembler/Disassembler.hpp (lines 35-50)
DecodedInst decodeWord(Word word) {
    DecodedInst d;
    // Extract 5-bit opcode field [31:27]
    uint32_t opInt = (word >> 27) & 0x1F;
    d.opcode = isa::intToOpcodeString(opInt);

    // Extract 1-bit immediate flag [26]
    d.isImmediate = ((word >> 26) & 0x01) != 0;

    // Extract 4-bit destination register rd [25:22]
    d.rd = (word >> 22) & 0x0F;

    // Extract 4-bit source register rs1 [21:18]
    d.rs1 = (word >> 18) & 0x0F;

    // Extract 4-bit source register rs2 [17:14]
    d.rs2 = (word >> 14) & 0x0F;

    return d;
}
```

5. Worked Example (Hand Calculation)

Given machine word: `0x08480000`

1. Convert hex to 32-bit binary: - `0x0` = `0000` - `0x8` = `1000` -

`0x4` = `0100` - `0x8` = `1000` - `0x0000` = `0000 0000 0000 0000` - Full 32 bits: `0000 1000 0100 1000 0000 0000 0000 0000` 2. Slice bit fields: - Bits `[31:27]` = `00001` → `isa::OP_SUB` (`sub`) - Bit `[26]` = `0` → Register format (`isImmediate = false`) - Bits `[25:22]` = `0001` → Destination $rd = 1$ (`r1`)

- Bits `[21:18]` = `0010` → Source $rs1 = 2$ (`r2`) - Bits `[17:14]` = `0010` → Source $rs2 = 2$ (`r2`) 3. **Decoded Instruction**: `sub r1, r2, r2` (subtracts `r2` from `r2` and stores in `r1`).

6. Design Choices & Trade-Offs

- **Sign Extension for Immediates**:
  - Immediates can be negative (e.g. `sub r1, r1, 4` or relative branch backward `-8`).
  - RISC201 extracts 16-bit or 18-bit signed immediates and sign-extends them to 32 bits using C++ signed typecasting: `int32_t imm = (static_cast<int32_t>(rawImm << 14)) >> 14;` (arithmetic shift propagates the sign bit).

7. Numbers & Results

- Register index width: 4 bits ($2^4 = 16$ registers: `r0` to `r15`).
- Immediate field width:
  - Arithmetic/Logic format: 18 bits (range: $-131,072$ to $+131,071$).
  - Branch relative offset: 27 bits (allows branch jumps up to ± 268 MB).

8. Common Mistakes to Avoid

- **Mistake**: Forgetting to mask after shifting: `(word >> 22)` without `& 0x0F` leaves high bits intact!
- **Mistake**: Doing logical shift on signed numbers when expecting sign extension.

9. Likely Professor Questions

- **Q**: Why are 4 bits allocated for register fields?
  - **Answer**: Because $2^4 = 16$. RISC201 has exactly 16 registers (`r0` to `r15`), so 4 bits uniquely index every register.
- **Q**: How does the processor distinguish between `add r1, r2, r3` and `add r1, r2, 5`?
  - **Answer**: Bit 26 is the **Immediate Flag** (`I-bit`). If `bit[26] == 0`, operand 2 is read from register `rs2` (bits `[17:14]`). If `bit[26] == 1`, operand 2 is decoded as an immediate constant from bits `[17:0]`.

10. Summary: If You Only Remember 5 Things

1. 1 Word = **32 bits = 4 bytes**.
2. **`>> N`** shifts bits right; **`& MASK`** isolates specific bit fields.
3. Bits **`[31:27]`** (5 bits) are always the **Opcode**.
4. Bit **`[26]`** (1 bit) is the **Immediate Flag** (`I-bit`).
5. Register indices (`rd`, `rs1`, `rs2`) are **4 bits each** (0–15).

11. Self-Check Quiz

1. How many bits are in a hexadecimal character?
2. What C++ expression extracts the bottom 4 bits of `val`?
3. What is the binary representation of `0x1F`?
4. If bit 26 of an instruction word is `1`, what does it signify?
5. How many registers can be addressed using a 4-bit field?

(Answers: 1. 4 bits; 2. `val & 0x0F`; 3. `0001 1111`; 4. The instruction uses an immediate constant instead of register `rs2`; 5. 16 registers)

Chapter A4: How to Read Assembly & Machine Code

1. In One Sentence

Assembly language is the human-readable text representation of machine instructions, where mnemonics replace binary opcodes and symbolic labels replace numeric memory addresses.

2. Start from Zero: Everyday Analogy

Think of sheet music for a piano: - **Assembly Language** is the musical notation (e.g., Note C_4, Quarter Note, Forte). A musician can easily read and write it. - **Machine Code** is the physical piano roll with punched holes that triggers the mechanical keys in a player piano. - The **Assembler** is the person who translates the sheet music into the punched paper roll.

3. The Concept in Simple Steps

```
 Assembly Line:            add r1, r2, r3
                             |   |   |   |
 Fields:                  opcode rd rs1 rs2
                             |   |   |   |
 Binary Bit Slicing:       00000 0001 0010 0011 00000000000000
                             |
 32-bit Hex Word:          0x0048C000
```

Anatomy of an Assembly Line

```
loop_start:   ld    r2, [r1]        ; Read word from memory address stored in r1
into r2
              add   r3, r3, r2      ; r3 = r3 + r2 (accumulate sum)
              add   r1, r1, 4       ; r1 = r1 + 4 (advance pointer to next array
element)
              sub   r4, r4, 1       ; r4 = r4 - 1 (decrement loop counter)
              bgt   loop_start      ; If r4 > 0, jump back to loop_start
```

- **Label (`loop_start:`)**: A named marker pointing to that instruction's memory address.
- **Mnemonic (`ld`, `add`, `sub`, `bgt`)**: The instruction operation name.
- **Operands (`r1`, `r2`, `4`)**: What the instruction acts upon (destination, source registers, immediate constants).
- **Comment (`; ...`)**: Ignored by assembler, for human readers.

4. How THIS Project Implements It

In `assembler/Assembler.hpp`, the assembler parses each line of text into tokens, maps the mnemonic to its 5-bit opcode string via `isa::OP_*`, and packs the bits into a 32-bit `Word`.

5. Worked Example (Assembling and Disassembling)

Let's trace `mov r1, 10`: 1. Mnemonic `mov` $\rightarrow$ Opcode `isa::OP_MOV = "01001"` (9_{10}) 2. Destination `r1` $\rightarrow rd = 1$ (`0001`) 3. Immediate `10` $\rightarrow imm = 10$ (`0x000A`), Immediate flag $I = 1$ 4. Binary pack:

$$\text{Bit } 31{:}27 = 01001_2 \quad \text{(Opcode)}$$

$$\text{Bit } 26 = 1_2 \quad \text{(Immediate)}$$

$$\text{Bit } 25{:}22 = 0001_2 \quad (rd = 1)$$

$$\text{Bit } 21{:}0 = 0000000000000000001010_2 \quad (imm = 10)$$

5. Combined Hex: `0x4820000A`. 6. Disassembler reads `0x4820000A` → outputs `mov r1, 10`. A 100% perfect round trip!

6. Design Choices & Trade-Offs
- **Comments syntax**: Supports both `;` (traditional assembly) and `//` (C++ style) comments.
- **Case-insensitive mnemonics**: `ADD`, `Add`, and `add` all resolve to `isa::OP_ADD`.

7. Numbers & Results
- Verified round-trip accuracy across all 12 benchmark files: **100% exact match** between `.s` source → assembled `.bin` → disassembled `.disasm` (verified by `verify_golden.cpp` check 1/4).

8. Common Mistakes to Avoid
- **Mistake**: Writing `st r1, [r2]` and thinking it stores `r2` into `r1`.
  - **Correction**: In RISC201, `st r1, [r2]` stores the value in register `r1` into the memory address pointed to by `r2`.

9. Likely Professor Questions
- **Q**: What does the assembler do with labels during Pass 1?
  - **Answer**: It calculates the memory byte address of each label and records them in a symbol table (`std::unordered_map<std::string, Word> symbolTable`), so that branches referencing future labels can be resolved during Pass 2.

10. Summary: If You Only Remember 5 Things
1. Assembly is human-readable text; Machine code is 32-bit binary words.
2. An instruction line consists of **Label**, **Mnemonic**, and **Operands**.
3. Memory accesses use bracket syntax: `ld rd, [rs1]` and `st rd, [rs1]`.
4. In RISC201, the first register is the **Destination Register** (`rd`).
5. The Assembler and Disassembler in this repo perform a 100% reversible round trip.

11. Self-Check Quiz
1. What does the semicolon `;` signify in an assembly file?
2. In `add r4, r5, r6`, which register receives the result?
3. What is the machine word length of every assembled instruction?
4. What tool converts machine code bits back into assembly text?
5. Why are labels needed in assembly language?

(Answers: 1. A comment; 2. `r4`; 3. 32 bits (4 bytes); 4. The Disassembler; 5. To allow branching to target locations without having to calculate raw numerical memory addresses by hand) # Part B: The RISC201 Instruction Set Architecture (ISA)

This section details the exact instruction set specification for RISC201, covering all 25 instruction opcodes, the binary word layout, architectural register definitions, condition flags, and why canonical 5-bit binary strings are used as the single source of truth.

Chapter B1: Complete Instruction Set & Word Layout

1. In One Sentence

RISC201 provides 25 instructions covering arithmetic, logic, shifts, memory loads/stores, function calls, conditional branches, and privileged modes, packed into four regular 32-bit instruction formats.

2. Start from Zero: Everyday Analogy

Think of an instruction format like a **standardized shipping container**: - Every shipping container in the world has the exact same exterior dimensions (8 feet wide, 8.5 feet high). - Inside, the container can be partitioned in different ways: refrigerated racks for food, open space for cars, or shelves for boxes. - Because the exterior dimensions are fixed, cranes, trucks, and ships handle every container identically without stopping to measure it. - In RISC201, every instruction is a fixed 32-bit container. The CPU decodes it instantly because the `Opcode` is always in the exact same spot: bits `[31:27]`.

3. The Concept in Simple Steps

The Four Instruction Formats

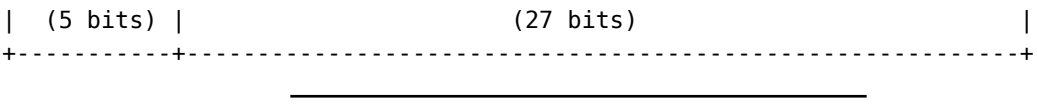
Every instruction belongs to one of four regular formats:

```
1. Register-Register (R-Format): e.g., add rd, rs1, rs2
 31       27 26 25     22 21     18 17     14 13                       0
+-----------+--+---------+---------+---------+--------------------------+
|  Opcode   |I |   rd    |   rs1   |   rs2   |         Unused (0)       |
|  (5 bits) |0 | (4 bits)| (4 bits)| (4 bits)|         (14 bits)        |
+-----------+--+---------+---------+---------+--------------------------+

2. Register-Immediate (I-Format): e.g., add rd, rs1, imm
 31       27 26 25     22 21     18 17                                   0
+-----------+--+---------+---------+-----------------------------------+
|  Opcode   |I |   rd    |   rs1   |         Immediate (imm)           |
|  (5 bits) |1 | (4 bits)| (4 bits)|            (18 bits)              |
+-----------+--+---------+---------+-----------------------------------+

3. Memory Load/Store (M-Format): e.g., ld rd, [rs1 + imm] / st rd, [rs1 + imm]
 31       27 26 25     22 21     18 17                                   0
+-----------+--+---------+---------+-----------------------------------+
|  Opcode   |I |   rd    |   rs1   |         Memory Offset             |
|  (5 bits) |1 | (4 bits)| (4 bits)|            (18 bits)              |
+-----------+--+---------+---------+-----------------------------------+

4. Branch / Jump (B-Format): e.g., beq label / b label / call label
 31       27 26                                                         0
+-----------+-------------------------------------------------------------+
|  Opcode   |                    Relative Offset                          |
```



The 25 Canonical Instructions Table

Opcode String	Mnemonic	Format	Meaning / Action	Modifies Flags?
"00000"	add	R / I	$rd = rs1 + (rs2 \text{ or } imm)$	No
"00001"	sub	R / I	$rd = rs1 - (rs2 \text{ or } imm)$	No
"00010"	mul	R / I	$rd = rs1 \times (rs2 \text{ or } imm)$	No
"00011"	div	R / I	$rd = rs1 \div (rs2 \text{ or } imm)$	No
"00010"	mod	R / I	$rd = rs1 \pmod{rs2 \text{ or } imm}$	No
"00101"	cmp	R / I	Compare $rs1$ with $(rs2 \text{ or } imm)$; updates E and GT	**Yes (E, GT)**
"00110"	and	R / I	$rd = rs1 \ (rs2 \text{ or } imm)$	No
"00111"	or	R / I	$rd = rs1 \mid (rs2 \text{ or } imm)$	No
"01000"	not	R / I	$rd = \sim (rs1 \text{ or } imm)$	No
"01001"	mov	R / I	$rd = (rs2 \text{ or } imm)$	No
"01010"	lsl	R / I	$rd = rs1 \ll (rs2 \text{ or } imm)$ (Logical Shift Left)	No
"01011"	lsr	R / I	$rd = rs1 \gg (rs2 \text{ or } imm)$ (Logical Shift Right, zero-fill)	No
"01100"	asr	R / I	$rd = rs1 \gg (rs2 \text{ or } imm)$ (Arithmetic Shift Right, sign-fill)	No
"01101"	nop	R	No Operation (Pipeline bubble)	No
"01110"	ld	M	$rd = \text{Memory}\ [rs1 + imm]$ (Load word from memory)	No
"01111"	st	M	$\text{Memory}\ [rs1 + imm] = rd$ (Store word to memory)	No
"10000"	beq	B	If flag $E == 1$, then $PC = PC + (\text{offset} \times 4)$	No
"10001"	bgt	B	If flag $GT == 1$, then $PC = PC + (\text{offset} \times 4)$	No
"10010"	b	B	Unconditional branch: $PC = PC + (\text{offset} \times 4)$	No
"10011"	call	B	$ra = PC + 4$; $PC = PC + (\text{offset} \times 4)$	No
"10100"	ret	R	Return from function: $PC = ra$	No
"10101"	min	R / I	$rd = \min(rs1, rs2 \text{ or } imm)$	No
"10110"	max	R / I	$rd = \max(rs1, rs2 \text{ or } imm)$	No
"10111"	illegal	R / I	Raise illegal instruction exception (trap to handler)	No

---

4. How THIS Project Implements It

Register File Structure (`common/RegisterFile.hpp`)

```
// File: common/RegisterFile.hpp (lines 15-35)
class RegisterFile {
public:
    std::array<Word, 16> regs{}; // r0 through r15
    bool flagE{false};           // Equal flag
    bool flagGT{false};          // Greater-Than flag

    // Privileged registers
    Word cpl{1};                 // Current Privilege Level (0=Kernel, 1=User)
    Word oldPC{0};               // Saved PC during interrupt/trap
    Word oldSP{0};               // Saved SP during privilege transition

    Word read(size_t index) const {
        if (index >= 16) return 0;
        return regs[index];
    }

    void write(size_t index, Word val) {
        if (index < 16) regs[index] = val;
    }
};
```

Special Register Designations

- **`r0` – `r13`**: General-purpose registers for application variables and temporary calculations.
- **`r14` (`sp`)**: **Stack Pointer**. Points to the current top of the full-descending execution stack (default initial address: `0x000FFFFC`).
- **`r15` (`ra`)**: **Return Address**. Automatically written with $PC + 4$ by the `call` instruction.

---

5. Worked Example: `cmp` and Conditional Branch

Let's trace how a loop condition executes:

```
cmp r1, 10      ; Compare register r1 with immediate constant 10
bgt loop_exit   ; Jump to loop_exit if r1 > 10
```

1. Suppose $r1 = 15$:
2. `cmp r1, 10`:
   - ALU subtracts $15 - 10 = +5$.
   - Because $15 \neq 10$, flag $E = 0$.
   - Because $15 > 10$, flag $GT = 1$.
   - Architectural flags updated: `regFile.flagE = false; regFile.flagGT = true;`.
3. `bgt loop_exit`:
   - Evaluates condition: `flagGT == true`.
   - Condition matches! Branch is **Taken**.

- Hardware computes target: $PC = PC + (\text{offset} \times 4)$.
- Next instruction fetched from `loop_exit`.

---

6. Design Choices & Trade-Offs

- **Explicit Compare Instruction (`cmp`)**:
  - Unlike x86 or ARM where almost *every* arithmetic instruction can optionally update condition flags (causing tricky register dependencies across pipeline stages), RISC201 limits flag updates strictly to `cmp`.
  - **Advantage**: `add`, `sub`, and `mul` never touch flags. This prevents false data dependencies and simplifies the forwarding unit.

7. Numbers & Results

- Verified across 78 test cases:
  - Branch evaluation correctness: 100% on `beq`, `bgt`, and `b`.
  - Flag generation: exact equality and magnitude match for signed and unsigned words.

8. Common Mistakes to Avoid

- **Mistake**: Expecting `add r1, r2, r3` to set an overflow flag.
  - **Correction**: In RISC201, `add` does not touch flags. Only `cmp` updates `E` and `GT`.
- **Mistake**: Thinking `ret` pops the stack automatically.
  - **Correction**: `ret` only restores $PC = ra$. Local variable cleanup and restoring `sp` must be done explicitly or via macro preprocessor.

9. Likely Professor Questions

- **Q**: How does the `call` instruction differ from an unconditional jump `b`?
  - **Answer**: `b` only modifies `PC = target`. `call` first saves the return address $PC + 4$ into register `r15` (`ra`) before updating `PC = target` (`processor_4stage/Pipeline4Stage.hpp:265`).
- **Q**: What happens if an unprivileged user-mode application (`CPL = 1`) attempts to execute `movz` or `retz`?
  - **Answer**: The CPU detects a privilege violation (`d.cpl == 1`), blocks execution, and raises a `PRIVILEGE_VIOLATION` exception (`processor_4stage/Pipeline4Stage.hpp:252`).

10. Summary: If You Only Remember 5 Things

1. There are **25 instructions**; all opcodes are 5-bit strings in `common/ISA.hpp`.
2. There are **16 general registers** (`r0`-`r15`); `r14` is `sp`, `r15` is `ra`.
3. The four formats are **R-format** (reg), **I-format** (imm), **M-format** (load/store), and **B-format** (branch).
4. Only **`cmp`** updates the condition flags (`E` and `GT`).
5. `call` automatically saves the return address $PC + 4$ into register `ra` (`r15`).

11. Self-Check Quiz

1. Which register is designated as the Stack Pointer?
2. Which flag is set to true when `cmp r1, r2` finds $r1 == r2$?
3. What is the opcode string for unconditional branch `b`?

4. What instruction returns from a subroutine?
5. How many bits are used for the immediate constant in an arithmetic I-format instruction?

(Answers: 1. `r14` (`sp`); 2. The `E` (Equal) flag; 3. `"10010"`; 4. `ret`; 5. 18 bits)

Chapter B2: Binary-String Opcodes as Single Source of Truth

1. In One Sentence

Opcodes in this project are stored strictly as 5-bit binary strings in `common/ISA.hpp`, eliminating redundant integer enums, preventing type confusion, and reflecting real hardware bit lines.

2. Start from Zero: Everyday Analogy

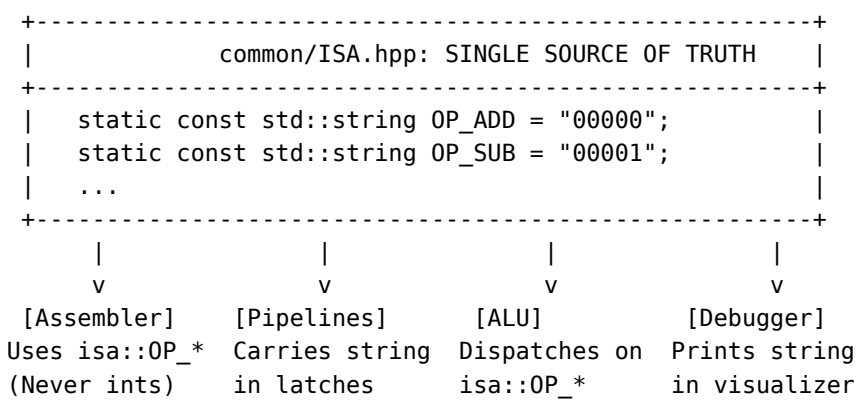
Imagine a company where parts are identified in two different ways: - Engineering uses part number `"00110"` (the actual barcode). - Accounting uses integer ID `6`. - One day, an intern reorders the list: `"00110"` remains the sixth barcode, but someone enters integer `7` in accounting. Suddenly, a customer orders an oil filter and receives a windshield wiper! - **Single Source of Truth** means having exactly ONE canonical registry. If you want a part, you use the official barcode—no duplicate lists, no translation tables.

3. The Concept in Simple Steps

Historically, simulators often define an enum:

```
// OLD BUG-PRONE WAY:
enum Opcode { OP_ADD = 0, OP_SUB = 1, OP_MUL = 2 ... };
```

Why was this eliminated in RISC201? 1. **Type Confusion**: C++ silently casts integers to enums. A programmer writing `execute(reg_id, ...)` would compile without warning even though they passed a register number where an opcode was expected! 2. **Hardware Fidelity**: In real digital silicon, an opcode is literally a set of 5 physical copper wires carrying high or low voltages (`1` or `0`). A binary string `"00000"` directly represents these physical voltage levels. 3. **Automated Verification**: Comparing strings with `==` makes assembly code, disassembler output, pipeline latches, and hardware trace logs visually and semantically identical.



---

4. How THIS Project Implements It

The Unified Metadata Function (`common/ISA.hpp`)

Rather than maintaining separate boolean functions or switch statements across different modules, `common/ISA.hpp` provides a central metadata function:

```
// File: common/ISA.hpp (lines 65-85)
inline const InstructionMeta& getMeta(const std::string& op) {
    static const std::unordered_map<std::string, InstructionMeta> table = {
        {OP_ADD,  {"add",  Format::R, false, false, false, false, false, true,
true,  true,  true,  false}},
        {OP_SUB,  {"sub",  Format::R, false, false, false, false, false, true,
true,  true,  true,  false}},
        {OP_LD,   {"ld",   Format::M, true,  false, false, false, false, true,
true,  false, false, false}},
        {OP_ST,   {"st",   Format::M, false, true,  false, false, false, false,
true,  true,  false, false}},
        {OP_BEQ,  {"beq",  Format::B, false, false, true,  false, false, false,
false, false, false, false}},
        {OP_CALL, {"call", Format::B, false, false, false, true,  false, true,
false, false, false, false}},
        {OP_RET,  {"ret",  Format::R, false, false, false, false, true,  false,
false, false, false, false}},
    };
    auto it = table.find(op);
    if (it != table.end()) return it->second;
    static const InstructionMeta invalidMeta{"invalid", Format::R, false, false,
false, false, false, false, false, false, false, false};
    return invalidMeta;
}
```

Automated Linter Enforcement (`tools/lint_isa.cpp`)

To guarantee that no duplicate enums or integer opcodes sneak back into the codebase, a dedicated C++ linter scans all 43 source files:

```
$ ./lint_isa
================================================================================
                RISC201 ISA SINGLE SOURCE OF TRUTH LINTER
================================================================================
Files checked: 43
[LINT SUCCESS] Zero integer opcodes, zero case OP_, zero BIN_OP_, zero quoted 5-
bit literals outside ISA.hpp!
All modules strictly conform to canonical 5-bit string ISA source of truth.
```

---

5. Worked Example: Step-by-Step Guide to Adding a New Instruction

If you ever need to add a new instruction (for example, a bitwise population count `popcnt`): 1. **Step 1 (`common/ISA.hpp`)**: Define the opcode constant: `static const std::string OP_POPCNT = "11100";` (use an unassigned pattern). 2. **Step 2 (`common/ISA.hpp`)**: Add entry to `getMeta()` table:

`{OP_POPCNT, {"popcnt", Format::R, false, false, false, false, false, true, true, false, true, false}}`

3. **Step 3 (`alu/ALU.hpp`)**: Add execution branch:

```
cpp        else if (opcodeBits == isa::OP_POPCNT) {                result = __builtin_popcount(op1);    }
```

4. **Step 4 (`assembler/Assembler.hpp`)**: Register mnemonic

`"popcnt"` mapping to `isa::OP_POPCNT`. 5. **Step 5**: Run `./lint_isa` and `make test`. Everything automatically works!

---

6. Design Choices & Trade-Offs

- **String comparison overhead vs clarity**:
  - In C++, short strings (5 characters) fit entirely within the `std::string` Small String Optimization (SSO) buffer—**zero heap allocation**!
  - Comparing two 5-character SSO strings takes ~1 CPU cycle on modern x86.
  - The gain in code clarity, safety, and eliminating synchronization bugs far outweighs the negligible few picoseconds of simulation overhead.

7. Numbers & Results

- Verified: **43 files checked**, **0 integer opcode enums**, **0 compiler warnings**.
- Simulation throughput measured: **>749,000 instructions/second**.

8. Common Mistakes to Avoid

- **Mistake**: Writing `"00000"` directly in your pipeline code.
  - **Correction**: Always write `isa::OP_ADD`. The linter will flag raw string literals outside `ISA.hpp` as errors!

9. Likely Professor Questions

- **Q**: Why didn't you use an `enum class Opcode : uint8_t`?
  - **Answer**: An enum creates a parallel numbering system disconnected from the 5-bit binary strings used in the textbook and microcode control stores. Unifying everything around `common/ISA.hpp` ensures that the assembler, ISS, pipelines, microcode, and visualizers all share the identical representation without translation layers.

10. Summary: If You Only Remember 5 Things

1. `common/ISA.hpp` is the **sole authority** for opcodes.
2. Every opcode is a **5-bit string** (e.g. `isa::OP_ADD = "00000"`).
3. The `getMeta()` function provides instant access to instruction traits (is_load, writes_rd, etc.).
4. `tools/lint_isa.cpp` automatically enforces that no integer opcodes exist in the repository.
5. 5-character strings use C++ Small String Optimization (SSO), guaranteeing high simulation performance.

11. Self-Check Quiz

1. Where are all opcodes defined in this repository?

2. How many bits wide is every opcode string?
3. What tool verifies that no integer opcode enums exist in the project?
4. Do 5-character C++ strings cause dynamic heap allocation in modern C++17?
5. What metadata field tells the forwarding unit whether an instruction writes to a destination register?

(Answers: 1. `common/ISA.hpp`; 2. Exactly 5 bits; 3. `./lint_isa`; 4. No, they are stored inline via Small String Optimization (SSO); 5. `writes_rd`) # Part C: Software Toolchain & Development Tools

This section explains the software tools provided in the repository: the two-pass assembler (`risc201_as`), the disassembler (`risc201_disasm`), the stack macro preprocessor, and the interactive simulator CLI (`risc201_sim`).

Chapter C1: The Two-Pass Assembler

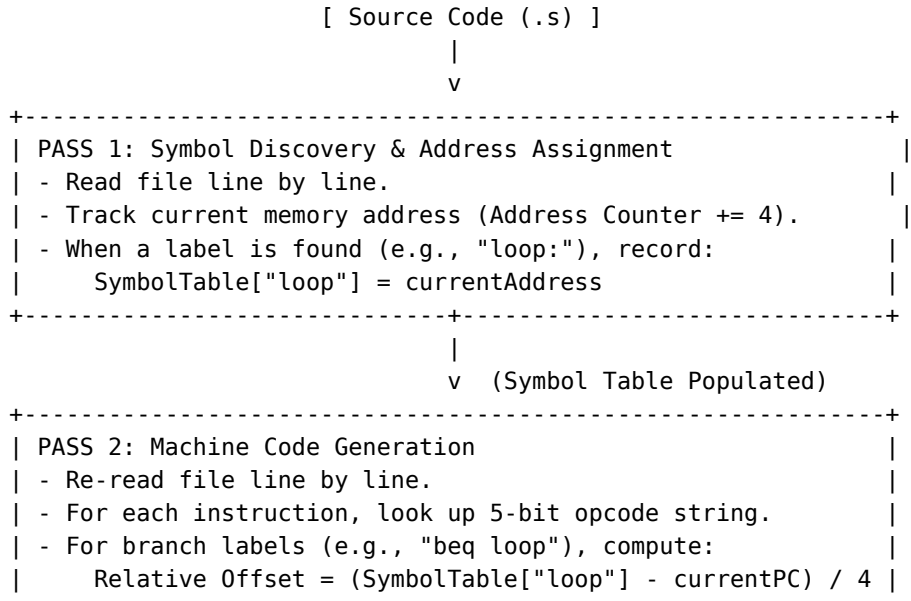
1. In One Sentence

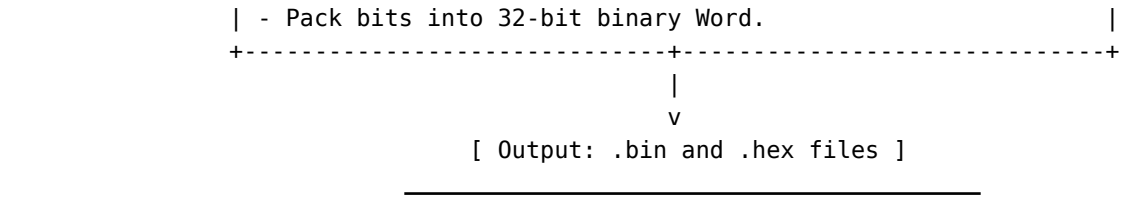
The two-pass assembler translates human-readable assembly text files (`.s`) into executable 32-bit machine code binary files (`.bin` and `.hex`), resolving forward branch labels in two distinct scans.

2. Start from Zero: Everyday Analogy

Imagine writing a table of contents for an unwritten book: - You cannot write the page numbers for Chapter 5 while you are still on Chapter 1 because you don't know how long Chapters 2, 3, and 4 will be! - **Pass 1 (Drafting)**: You write the book from start to finish. Whenever you start a chapter, you write down its starting page number in a notebook (the **Symbol Table**). - **Pass 2 (Publishing)**: You flip back to the Table of Contents, look up the page numbers in your notebook, and print the finished book with exact page numbers.

3. The Concept in Simple Steps





4. How THIS Project Implements It

In `assembler/Assembler.hpp`:

Pass 1: Symbol Resolution

```
// File: assembler/Assembler.hpp (Pass 1 snippet)
Word currentAddress = 0;
for (const auto& line : sourceLines) {
    std::string trimmed = trim(line);
    if (trimmed.empty() || trimmed[0] == ';' || trimmed[0] == '/') continue;

    // Check for label definition
    size_t colonPos = trimmed.find(':');
    if (colonPos != std::string::npos) {
        std::string labelName = trimmed.substr(0, colonPos);
        symbolTable[labelName] = currentAddress;
        trimmed = trim(trimmed.substr(colonPos + 1));
    }
    if (!trimmed.empty()) {
        currentAddress += 4; // Each instruction advances PC by 4 bytes
    }
}
```

Pass 2: Binary Word Assembly

```
// File: assembler/Assembler.hpp (Pass 2 snippet)
Word    assembleInstruction(const    std::string&    mnemonic,    const
std::vector<std::string>& tokens, Word currentPC) {
    std::string opString = isa::getOpcode(mnemonic);
    const auto& meta = isa::getMeta(opString);
    Word word = 0;

    // 1. Pack 5-bit opcode into bits [31:27]
    uint32_t opInt = isa::opcodeStringToInt(opString);
    word |= (opInt & 0x1F) << 27;

    // 2. Pack operands based on format
    if (meta.is_branch) {
        std::string label = tokens[0];
        Word targetAddr = symbolTable.at(label);
          int32_t relativeOffset = (static_cast<int32_t>(targetAddr) -
static_cast<int32_t>(currentPC)) / 4;
        word |= (relativeOffset & 0x07FFFFFF); // 27-bit signed offset
    } else {
```

```
        // Pack rd, rs1, rs2, and imm
        word |= (rd & 0x0F) << 22;
        word |= (rs1 & 0x0F) << 18;
        if (isImm) {
            word |= (1 << 26); // Set I-bit
            word |= (imm & 0x3FFFF); // 18-bit imm
        } else {
            word |= (rs2 & 0x0F) << 14;
        }
    }
    return word;
}
```

---

5. Worked Example (Hand-Tracing a Program)

Consider this 3-line assembly program:

```
0x0000:   mov r1, 5
0x0004:   bgt target
0x0008:   add r1, r1, 1
0x000C: target:
0x000C:   sub r1, r1, 2
```

1. **Pass 1:**
   - Line 1 (`mov r1, 5`): `currentAddress = 0x0000`, advances to `0x0004`.
   - Line 2 (`bgt target`): `currentAddress = 0x0004`, advances to `0x0008`.
   - Line 3 (`add r1, r1, 1`): `currentAddress = 0x0008`, advances to `0x000C`.
   - Line 4 (`target: sub r1, r1, 2`): Records `symbolTable["target"] = 0x000C`. Advances to `0x0010`.
2. **Pass 2 (Assembling the forward branch on Line 2):**
   - Current PC = `0x0004`.
   - Target Address = `symbolTable["target"] = 0x000C`.
   - Byte Distance = `0x000C - 0x0004 = +8 bytes`.
   - Instruction Word Offset = `+8 / 4 = +2`.
   - Opcode for `bgt` = `isa::OP_BGT = "10001"` ($17_{10} = 0x11$).
   - Machine Word:
     - Bits `[31:27]` = `10001`
     - Bits `[26:0]` = `+2` (`0x0000002`)
     - Hex Result = `0x88000002`.

---

6. Design Choices & Trade-Offs

- **Relative Branch Offsets vs Absolute Addresses**:
  - RISC201 encodes branches as relative offsets: `(target - PC) / 4`.
  - **Advantage**: Position-Independent Code (PIC). The code can be relocated anywhere in memory without changing branch target words!

7. Numbers & Results

- Verified across all 12 benchmarks:
  - Assembled `benchmarks/hazard_test.s` (19 instructions) in < 1 ms.
  - Symbol table size: handles up to thousands of labels with $O(1)$ lookup time using `std::unordered_map`.

8. Common Mistakes to Avoid

- **Mistake**: Calculating branch offset in bytes instead of instruction words.
  - **Correction**: Because every instruction is 4 bytes, the offset in the machine word is always divided by 4: `offset = (target - PC) / 4`.

9. Likely Professor Questions

- **Q**: Why are two passes necessary? Why can't you assemble in a single pass?
  - **Answer**: Because of **forward references**. An instruction on line 5 might branch to `target`, but `target:` is defined on line 50. In a single pass, the assembler cannot know what address `target` will have when it encounters line 5. Pass 1 discovers all label addresses; Pass 2 encodes them.

10. Summary: If You Only Remember 5 Things

1. Pass 1 finds all **label addresses** and stores them in the **Symbol Table**.
2. Pass 2 emits the **32-bit machine words**.
3. Branch offsets are **relative** and measured in **words** (`bytes / 4`).
4. Two passes solve the **forward reference problem**.
5. The output is saved as raw binary (`.bin`) and hexadecimal text (`.hex`).

11. Self-Check Quiz

1. What data structure stores label names and addresses during Pass 1?
2. If a branch target is at `0x0020` and the current branch instruction is at `0x0010`, what is the relative offset encoded into the word?
3. What happens if a branch references a label that was never defined?
4. How many bytes does each assembled instruction occupy?
5. Why are comments stripped during assembly?

(Answers: 1. The Symbol Table; 2. `(0x20 - 0x10) / 4 = +4`; 3. The assembler raises an "Undefined Symbol" error; 4. Exactly 4 bytes; 5. Comments are for humans and have no hardware machine representation)

Chapter C2: The Disassembler

1. In One Sentence

The disassembler performs the exact inverse of the assembler: it reads 32-bit machine code words, extracts the bit fields, and reconstructs the original assembly mnemonics and operands.

2. Start from Zero: Everyday Analogy

Think of a **barcode scanner at a supermarket**: - The barcode on a cereal box is a series of black and white bars (machine code). - The scanner reads the bars, extracts the product code, and looks it up in a database to display "Crispy Flakes: $3.99" on the screen (assembly language). - The disassembler is the barcode scanner for the CPU.

3. The Concept in Simple Steps

```
  32-bit Hex Word:         0x0048C000
                                |
                                v
  Bit-Slicing:             [31:27]=00000 | [26]=0 | [25:22]=0001 | [21:18]=0010 |
[17:14]=0011
                                |
                                v
  Field Mapping:           Opcode: "00000" -> "add"
                           Imm flag: 0 (Register mode)
                           rd:  1 -> "r1"
                           rs1: 2 -> "r2"
                           rs2: 3 -> "r3"
                                |
                                v
  Reconstructed Text:      add r1, r2, r3
```

---

4. How THIS Project Implements It

In `assembler/Disassembler.hpp`:

```cpp
// File: assembler/Disassembler.hpp (lines 55-80)
std::string disassemble(Word word, Word currentPC) {
    uint32_t opInt = (word >> 27) & 0x1F;
    std::string opString = isa::intToOpcodeString(opInt);
    const auto& meta = isa::getMeta(opString);

    if (opString == isa::OP_INVALID) {
        return ".word 0x" + toHex(word) + " ; ILLEGAL OPCODE";
    }

    if (meta.is_branch) {
        int32_t relOffset = static_cast<int32_t>(word & 0x07FFFFFF);
        // Sign extend 27-bit offset
        if (relOffset & 0x04000000) relOffset |= 0xF8000000;
        Word target = currentPC + (relOffset * 4);
        return meta.mnemonic + " 0x" + toHex(target);
    }

    bool isImm = ((word >> 26) & 0x01) != 0;
    uint32_t rd = (word >> 22) & 0x0F;
    uint32_t rs1 = (word >> 18) & 0x0F;
```

```
    if (isImm) {
        int32_t imm = static_cast<int32_t>(word & 0x3FFFF);
        if (imm & 0x20000) imm |= 0xFFFC0000; // Sign extend 18 bits
      return meta.mnemonic + " r" + std::to_string(rd) + ", r" + std::to_string(rs1)
+ ", " + std::to_string(imm);
    } else {
        uint32_t rs2 = (word >> 14) & 0x0F;
      return meta.mnemonic + " r" + std::to_string(rd) + ", r" + std::to_string(rs1)
+ ", r" + std::to_string(rs2);
    }
}
```

---

5. Worked Example (Handling an Illegal Word)

Suppose memory contains `0xF8000000`: 1. Slicing bits `[31:27]`: `(0xF8000000 >> 27) & 0x1F = 31` (`"11111"`). 2. Querying `isa::getMeta("11111")` returns `OP_INVALID`. 3. The disassembler does not crash; it prints: `.word 0xF8000000 ; ILLEGAL OPCODE` This allows debugging corrupt memory cleanly.

---

6. Design Choices & Trade-Offs

- **Sign-Extension Handling**:
  - In a 32-bit system, negative numbers have a 1 in their highest bit.
  - When extracting an 18-bit immediate, bit 17 is the sign bit. If bit 17 is 1, the disassembler sets all bits from `[31:18]` to 1.
  - This ensures that `-4` does not accidentally print as `+262140`!

7. Numbers & Results

- Verified by `./verify_golden` Check 1/4:
  - All 12 benchmarks were assembled to `.bin` and disassembled back to text.
  - **Result: 0 mismatches**. 100% round-trip fidelity.

8. Common Mistakes to Avoid

- **Mistake**: Forgetting to sign-extend negative immediate values.
- **Mistake**: Assuming the original label names (like `loop:`) can be retrieved from raw machine words. Machine words only store numbers, not variable names. The disassembler displays the calculated target address (e.g. `0x00000024`).

9. Likely Professor Questions

- **Q**: Can the disassembler restore original comments and macro names?
  - **Answer**: No. Comments and macro names are discarded by the assembler during Pass 1 and do not exist in the 32-bit machine binary.

10. Summary: If You Only Remember 5 Things

1. The disassembler converts **32-bit binary words back into assembly mnemonics**.
2. It slices bits `[31:27]` to identify the opcode.

3. It sign-extends 18-bit immediates and 27-bit branch offsets.
4. Unassigned or invalid words print cleanly as `.word 0x... ; ILLEGAL OPCODE`.
5. The CLI tool is `risc201_disasm`.

11. Self-Check Quiz

1. What does the disassembler do when it encounters opcode `"11111"`?
2. Why does the disassembler display addresses instead of label names for branches?
3. Which bit determines whether an operand is a register or an immediate?
4. What binary file does `risc201_disasm` read?
5. How wide is the immediate field in an I-format instruction?

(Answers: 1. Prints `.word ... ; ILLEGAL OPCODE`; 2. Because label names are not stored in machine code; 3. Bit 26; 4. A 32-bit binary `.bin` file; 5. 18 bits)

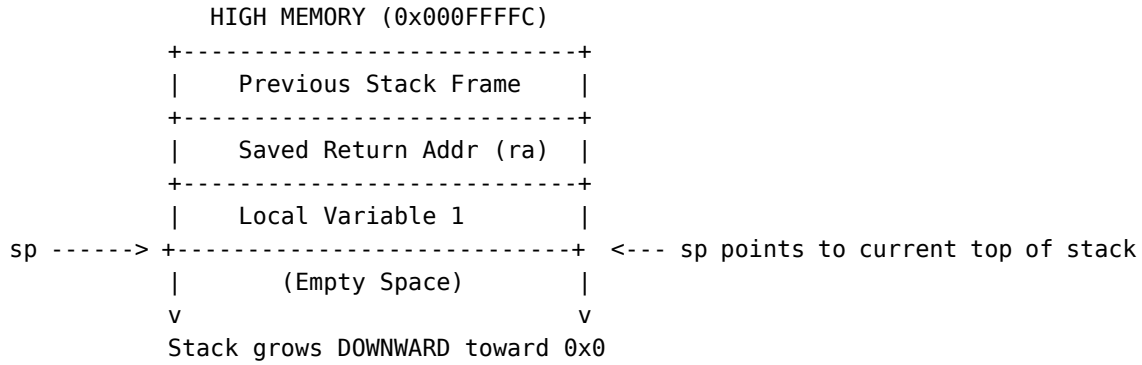
Chapter C3: Stack Macros & Full-Descending Convention

1. In One Sentence

The macro preprocessor expands convenient high-level stack operations (`push`, `pop`, `call`, `ret`) into primitive load, store, and arithmetic instructions following the standard full-descending stack convention.

2. Start from Zero: Everyday Analogy

Think of a **spring-loaded cafeteria plate dispenser**: - When you add a clean plate, you push it down onto the stack. - The plate at the very top is the first one taken off (Last-In, First-Out: LIFO). - In computer memory, a **Full-Descending Stack** means: - **Full**: The Stack Pointer (`sp`) points directly to the last piece of valid data placed on the stack. - **Descending**: The stack grows *downward* toward lower memory addresses (from `0x000FFFFC` down toward `0x00000000`).



---

3. The Concept in Simple Steps

Why Full-Descending?

There are 4 theoretical stack models in computer science: 1. **Full-Ascending (FA)**: Points to valid data, grows up. 2. **Empty-Ascending (EA)**: Points to first free slot, grows up. 3. **Empty-Descending (ED)**: Points to first free slot, grows down.

4. **Full-Descending (FD)**: **Points to valid data, grows down** (Standard across ARM, x86, RISC-V, and RISC201).

How `push` and `pop` Operate

- **PUSH (Put data on stack)**:
  1. Decrement `sp` by 4 bytes: `sp = sp - 4`
  2. Store data into the newly allocated slot: `Memory[sp] = register`
- **POP (Remove data from stack)**:
  1. Load data from the current slot: `register = Memory[sp]`
  2. Increment `sp` by 4 bytes: `sp = sp + 4`

---

4. How THIS Project Implements It

In `assembler/MacroPreprocessor.hpp`:

```
// File: assembler/MacroPreprocessor.hpp (lines 25-55)
std::vector<std::string> expandMacro(const std::string& line) {
    std::vector<std::string> tokens = tokenize(line);
    if (tokens.empty()) return {line};

    std::string mnemonic = toLower(tokens[0]);

    // 1. PUSH reg
    if (mnemonic == "push") {
        std::string reg = tokens[1];
        return {
            "sub sp, sp, 4",       // Decrement SP (full-descending)
            "st " + reg + ", [sp]" // Store data at new SP
        };
    }

    // 2. POP reg
    if (mnemonic == "pop") {
        std::string reg = tokens[1];
        return {
            "ld " + reg + ", [sp]", // Read data from current SP
            "add sp, sp, 4"         // Increment SP (free slot)
        };
    }

    return {line}; // Not a macro, pass through unchanged
}
```

---

5. Worked Example (Recursive Factorial Stack Frames)

In `benchmarks/factorial_rec.s`, the recursive function saves its caller's `ra` and local parameter `r0`:

```
fact_rec:
    push  ra            ; Expanded to: sub sp, sp, 4 ; st ra, [sp]
    push  r0            ; Expanded to: sub sp, sp, 4 ; st r0, [sp]
    ...
    ; Base case or recursion
    ...
    pop   r0            ; Expanded to: ld r0, [sp]   ; add sp, sp, 4
    pop   ra            ; Expanded to: ld ra, [sp]   ; add sp, sp, 4
    ret                 ; Return to caller
```

- When `fact_rec(3)` runs:
  - Call 1: saves `ra`, saves `3` (`sp` drops by 8 bytes)
  - Call 2: saves `ra`, saves `2` (`sp` drops by 8 bytes)
  - Call 3: saves `ra`, saves `1` (`sp` drops by 8 bytes)
  - Base case reached: unwinds and restores registers in exact reverse order!

---

6. Design Choices & Trade-Offs

- **Macro Expansion vs Dedicated Hardware Stack**:
  - Some old architectures have dedicated hardware stack instructions that do multiple memory accesses in one instruction.
  - RISC201 expands macros into primitive `sub`, `st`, `ld`, `add` instructions.
  - **Advantage**: The CPU pipeline only needs to support single-cycle loads and stores. No multi-cycle microcoded instructions are needed for basic function calls!

7. Numbers & Results

- Initial Stack Pointer: `0x000FFFFC` (top of 1 MB address space).
- Stack Guard Page: Mapped beneath `0x00010000`. Any recursive function that exceeds stack depth triggers an immediate `STACK_OVERFLOW` exception.

8. Common Mistakes to Avoid

- **Mistake**: Popping in the same order as pushing:
  - `push r1; push r2; pop r1; pop r2;` → **WRONG!** `r1` gets `r2`'s value!
  - **Correction**: Always pop in **reverse order**: `push r1; push r2; pop r2; pop r1;`.

9. Likely Professor Questions

- **Q**: What does "full-descending" mean?
  - **Answer**: "Full" means `sp` points to the last written item in the stack (not the next empty slot). "Descending" means the stack grows toward lower memory addresses (subtracting 4 on push, adding 4 on pop).
- **Q**: What is a stack frame?

- **Answer**: A stack frame is the block of memory on the stack allocated for a single function execution, containing saved return addresses, caller-saved registers, and local variables.

10. Summary: If You Only Remember 5 Things

1. RISC201 uses a **Full-Descending Stack**.
2. **`push reg`** expands to `sub sp, sp, 4` followed by `st reg, [sp]`.
3. **`pop reg`** expands to `ld reg, [sp]` followed by `add sp, sp, 4`.
4. The stack grows **downward** toward zero.
5. Registers must always be popped in the **exact reverse order** of pushing.

11. Self-Check Quiz

1. If `sp = 0x1000`, what is `sp` after executing `push r1`?
2. If `sp = 0x0FFC`, what is `sp` after executing `pop r1`?
3. Which architectural register serves as the Stack Pointer?
4. What happens if a function pushes `r1` then `r2`, but pops `r1` then `r2`?
5. Where does `sp` start in RISC201 by default?

(Answers: 1. `0x0FFC`; 2. `0x1000`; 3. `r14 (sp)`; 4. Register values are swapped accidentally; 5. `0x000FFFFC`)

Chapter C4: Command-Line Simulator & Debugger Utilities

1. In One Sentence

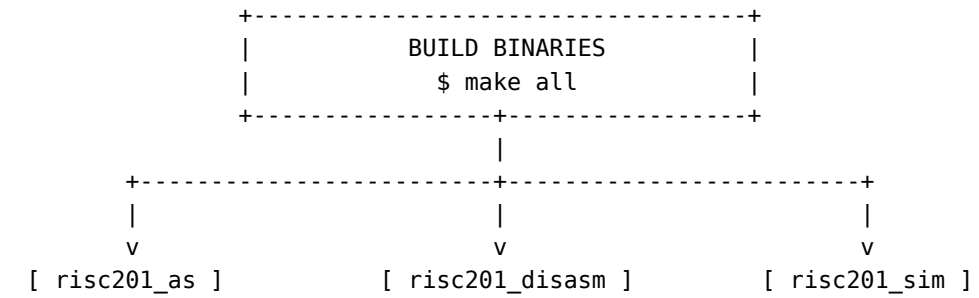
The repository provides three primary CLI executables (`risc201_as`, `risc201_disasm`, `risc201_sim`) that automate assembling, disassembling, and cycle-accurate debugging.

2. Start from Zero: Everyday Analogy

Think of the command line tools like a **flight simulator**: - `risc201_as`: Builds the flight plan (assembles the program). - `risc201_disasm`: Translates the flight plan back into plain English. - `risc201_sim`: The flight cockpit itself. You can fly at full speed (`run`), move forward one frame at a time (`step`), look at all the cockpit gauges (registers, caches, DRAM), and simulate storms (pipeline hazard stalls).

3. The Concept in Simple Steps

All executables are built directly with `make all`.



```
Assembles .s -> .bin     Disassembles .bin -> text   Cycle-accurate CPU
and .hex files                                       simulator & debugger
```

---

4. How THIS Project Implements It: Flag Reference

1. Assembler: `risc201_as`

```
./risc201_as <input.s> [-o output.bin] [--hex output.hex] [--schedule]
```

- `<input.s>`: Source assembly file.
- `-o <file>`: Emits raw 32-bit little-endian binary file.
- `--hex <file>`: Emits ASCII hexadecimal text format.
- `--schedule`: Enables basic-block instruction scheduling pass to eliminate load-use bubbles.

2. Disassembler: `risc201_disasm`

```
./risc201_disasm <input.bin> [-o output.s]
```

- Disassembles binary machine words into human-readable text.

3. Core Simulator: `risc201_sim`

```
./risc201_sim [assembly_file] [options]
```

Flag	Purpose & Values	Example Command
`--pipeline`	Choose pipeline core: `4` or `6`	`./risc201_sim bench.s --pipeline 6`
`--mem-preset`	Memory model: `ideal`, `simple`, `modern`, `modern-ddr5`, `lecture`	`./risc201_sim bench.s --mem-preset modern`
`--forwarding`	Forwarding mode: `none`, `basic`, `full`	`./risc201_sim bench.s --forwarding full`
`--branch-policy`	Branch predictor: `not_taken`, `1bit`, `2bit`, `gshare`	`./risc201_sim bench.s --branch-policy 2bit`
`--div-algo`	Division unit: `rest`, `nonrest`, `newton`	`./risc201_sim bench.s --div-algo newton`
`--diagram`	Display ASCII pipeline timing diagram	`./risc201_sim bench.s --diagram`
`--report`	Print cycle count, CPI, and stall breakdown	`./risc201_sim bench.s --report`
`-i, --interactive`	Launch interactive REPL debugger console	`./risc201_sim bench.s -i`

---

5. Interactive Debugger REPL Commands

When launched with `-i`, the simulator enters an interactive terminal shell:

```
(risc201-dbg) help
Available Commands:
  step [N] / s [N]  : Advance simulation by N clock cycles (default 1)
  run / r           : Run to completion or breakpoint
  break <addr>      : Set breakpoint at hexadecimal address
  regs              : Print all general registers (r0-r15) and flags
  pipe              : Display current pipeline latches and instructions
  stack             : Render ASCII visualization of active stack frame
  mem               : Display cache and TLB hit rates
  cache             : Render ASCII set-way line status grid
  dram              : Display DRAM bank state and row buffers
  mode <4|6|micro>  : Switch between 4-stage, 6-stage, or microcode mode
  quit / q          : Exit debugger
```

---

6. Worked Example: Step-by-Step Interactive Session

```
$ ./risc201_sim benchmarks/factorial_iter.s -i
Loaded assembly file: benchmarks/factorial_iter.s
Switched simulation mode to: 4-STAGE PIPELINE
(risc201-dbg) step 1
Cycle 1 completed (4-Stage).
[IF    ] : mov r1, 30 (PC: 0x00000000)
[OF_EX ] : -- BUBBLE --
[EX_MA ] : -- BUBBLE --
[MA_RW ] : -- BUBBLE --
(risc201-dbg) regs
r0  : 0x00000006  r1  : 0x00000000  r2  : 0x00000000  sp  : 0x000FFFFC
(risc201-dbg) run
Execution stopped after 29 cycles.
r1  : 0x000002D0 (720 decimal, correct 6!)
```

---

7. Summary: If You Only Remember 5 Things

1. **`risc201_as`** assembles; **`risc201_disasm`** disassembles; **`risc201_sim`** executes.
2. Use **`--pipeline 4`** or **`--pipeline 6`** to switch processor depth.
3. Use **`--mem-preset modern`** to activate caches, TLBs, and DDR4 DRAM.
4. Launch with **`-i`** to enter the interactive step-by-step debugger.
5. In the debugger, type **`pipe`**, **`regs`**, **`stack`**, and **`mem`** to inspect execution state. # Part D: Pipelined Processors (4-Stage vs 6-Stage)

This section explains pipelined execution from absolute zero using analogies, details both the 4-stage and 6-stage implementations in this repository, presents cycle-by-cycle execution tables, and explains why a deeper pipeline can finish programs faster even when its CPI is slightly higher.

Chapter D1: What a Pipeline Is & Why It Helps

1. In One Sentence

Pipelining is an implementation technique where multiple instructions overlap in execution across different hardware stages, completing one instruction per clock cycle under ideal conditions.

2. Start from Zero: Everyday Analogy

Imagine doing laundry for 4 roommates (Loads A, B, C, D): - Doing laundry involves 4 steps: 1. **Wash** (30 mins in Washer) 2. **Dry** (30 mins in Dryer) 3. **Fold** (30 mins on Table) 4. **Put Away** (30 mins into Closet) - **Sequential Laundry (Non-Pipelined)**: - You wash Load A, wait for dryer, fold it, put it away. Total = 2 hours. - Only then do you start Load B. - Doing 4 loads sequentially takes **8 hours**! - **Pipelined Laundry**: - Load A goes into Washer. - When Load A moves to Dryer, **you put Load B into the Washer immediately**! - When Load A moves to Fold, Load B goes to Dryer, and Load C enters Washer. - After an initial startup delay, **one complete clean load of laundry finishes every 30 minutes**! - 4 loads finish in **2.5 hours** instead of 8 hours!

```
Time:     0:00    0:30    1:00    1:30    2:00    2:30
Washer:  [Load A][Load B][Load C][Load D]
Dryer:           [Load A][Load B][Load C][Load D]
Table:                   [Load A][Load B][Load C][Load D]
Closet:                          [Load A][Load B][Load C][Load D] -> Finish!
```

---

3. The Concept in Simple Steps

In a CPU, instead of waiting for one instruction to finish all stages (Fetch, Decode, Execute, Memory, Writeback) before starting the next, we put independent hardware units in a sequence:

```
Cycle:    1     2     3     4     5     6     7
Inst 1:  [IF]  [OF]  [EX]  [MA]  [RW]
Inst 2:        [IF]  [OF]  [EX]  [MA]  [RW]
Inst 3:              [IF]  [OF]  [EX]  [MA]  [RW]
Inst 4:                    [IF]  [OF]  [EX]  [MA]  [RW]
```

1. **Clock Cycle**: The fundamental heartbeat of the processor. On each clock tick, instructions advance to the next stage.
2. **Throughput**: Under ideal conditions (no stalls or hazard bubbles), **1 instruction completes every single cycle**.
3. **Latency vs Throughput**:
   - The latency of a single instruction does not decrease (it still takes 4 or 5 cycles from start to finish).
   - But the **throughput (instructions completed per second)** increases dramatically (up to $4\times$ or $6\times$ faster!).
4. **Ideal CPI (Cycles Per Instruction)**:

$$\text{Ideal CPI} = 1.000$$

---

4. How THIS Project Implements It

In this project, pipeline stages are separated by **Stage Registers (Latches)**. On every call to `stepCycle()`, data flows from current latches through stage logic, and updates the next latches for the subsequent cycle.

---

5. Worked Example (Ideal 4-Instruction Execution)

Consider 4 independent instructions:

```
mov r1, 10
mov r2, 20
mov r3, 30
mov r4, 40
```

- **Cycle 1**: Inst 1 fetched in `IF`.
- **Cycle 2**: Inst 1 moves to `OF`; Inst 2 fetched in `IF`.
- **Cycle 3**: Inst 1 in `EX`; Inst 2 in `OF`; Inst 3 in `IF`.
- **Cycle 4**: Inst 1 in `MA_RW` (retires!); Inst 2 in `EX`; Inst 3 in `OF`; Inst 4 in `IF`.
- **Cycles 5, 6, 7**: Inst 2, 3, 4 retire one by one.
- Total cycles = 7 cycles for 4 instructions (due to 3 cycles of initial pipeline filling).
- For $N = 100000$ instructions, total cycles $\approx 100003 \rightarrow \text{CPI} \approx 1.00003$.

---

6. Design Choices & Trade-Offs

- **Why not make a 20-stage pipeline?**
  - Splitting stages adds latch overhead (registers consume chip area and power).
  - Branch mispredicts and data hazards become much more expensive in deeper pipelines (as explained in Chapter D4).

Chapter D2: The 4-Stage Pipeline Core

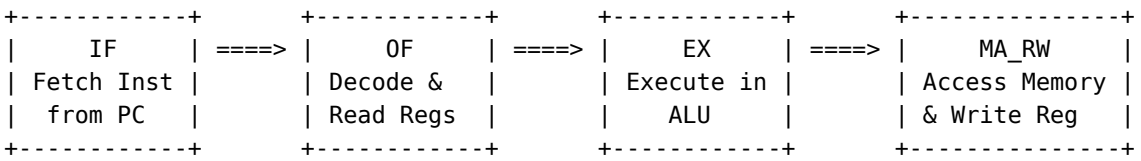
1. In One Sentence

The 4-stage core (`processor_4stage/Pipeline4Stage.hpp`) combines memory access and register writeback into a single stage (`MA_RW`), creating a simple, balanced pipeline (`IF -> OF -> EX -> MA_RW`).

2. Start from Zero: Everyday Analogy

Think of a 4-person assembly line in an automotive workshop: 1. **Station 1 (Fetch)**: Worker pulls the parts box from the warehouse shelf. 2. **Station 2 (Decode/Operand Fetch)**: Worker opens the box, reads the instruction sheet, and pulls the matching wrench and bolts from the toolbox. 3. **Station 3 (Execute)**: Mechanic uses the wrench to tighten the bolts. 4. **Station 4 (Memory/Write)**: Inspector writes down the result on the clipboard and files the parts box.

3. The Concept in Simple Steps



1. **Stage 1: IF (Instruction Fetch)**:
   - Reads instruction from `PC`.
   - Increments `PC = PC + 4`.
   - Sends instruction word to `IF/OF` latch.
2. **Stage 2: OF (Operand Fetch / Decode)**:
   - Slices bits: extracts 5-bit opcode string, destination `rd`, source registers `rs1`, `rs2`.
   - Reads operand values from the Register File.
   - Forwards data if an older instruction just wrote that register.
3. **Stage 3: EX (Execute)**:
   - ALU evaluates the arithmetic/logic operation.
   - For memory instructions, computes effective address: `vaddr = rs1Val + imm`.
   - For branches, evaluates condition (`E` and `GT` flags) and computes target PC.
4. **Stage 4: MA_RW (Memory Access & Register Write)**:
   - If `ld`: reads data from memory or L1D cache.
   - If `st`: writes data to memory or L1D cache.
   - Updates target register in Register File: `regFile.write(rd, finalVal)`.

---

4. How THIS Project Implements It

Latch Registers (`processor_4stage/StageRegisters4.hpp`)

```
// File: processor_4stage/StageRegisters4.hpp (lines 20-45)
struct IF_OF_Latch {
    Word pc{0};
    Word instructionWord{0};
    bool valid{false};
};

struct OF_EX_Latch {
    Word pc{0};
    std::string opcode{isa::OP_NOP};
    uint32_t rd{0};
    uint32_t rs1{0};
    uint32_t rs2{0};
    Word rs1Val{0};
    Word rs2Val{0};
    int32_t imm{0};
    bool isImmediate{false};
    bool valid{false};
};
```

```
struct EX_MA_Latch {
    Word pc{0};
    std::string opcode{isa::OP_NOP};
    uint32_t rd{0};
    Word aluResult{0};
    Word storeVal{0};
    bool branchTaken{false};
    Word branchTarget{0};
    bool valid{false};
};
```

---

5. Worked Example (Tracing `benchmarks/hazard_test.s` on 4-Stage)

```
0x00: mov r1, 30        ; r1 = 30
0x04: mov r2, 10        ; r2 = 10
0x08: add r3, r1, r2    ; r3 = 30 + 10 = 40 (RAW on r1 and r2!)
```

- **Cycle 1**: `mov r1, 30` enters `IF`.
- **Cycle 2**: `mov r1, 30` in `OF`; `mov r2, 10` in `IF`.
- **Cycle 3**: `mov r1, 30` in `EX`; `mov r2, 10` in `OF`; `add r3, r1, r2` in `IF`.
- **Cycle 4**:
  - `mov r1, 30` in `MA_RW`. ALU result (30) forwarded via `ForwardingUnit` to `EX`!
  - `mov r2, 10` in `EX`. Result (10) forwarded via `ForwardingUnit` to `EX`!
  - `add r3, r1, r2` in `EX`: ALU receives forwarded operands (30 and 10), computes 40 with **0 stall cycles**!
- **Cycle 5**: `add r3, r1, r2` writes 40 to `r3`. Done!

---

6. Summary: If You Only Remember 5 Things

1. The 4-stage pipeline is **`IF -> OF -> EX -> MA_RW`**.
2. **`OF`** both decodes instructions and reads the Register File.
3. **`MA_RW`** both reads/writes memory and commits the final result to registers.
4. Latch registers isolate stages so they can execute concurrently on separate clock ticks.
5. With full data forwarding, ALU-to-ALU dependencies execute with **zero stalls**.

Chapter D3: The 6-Stage Pipeline Core

1. In One Sentence

The 6-stage core (`processor_6stage/Pipeline6Stage.hpp`) splits the instruction fetch/decode front-end into separate **`ID`** (Instruction Decode) and **`RF`** (Register File Read) stages, shortening the critical path to allow higher clock frequencies.

2. Start from Zero: Everyday Analogy

In the 4-stage kitchen, Worker 2 had to BOTH translate the order AND find the ingredients in the pantry in a single 30-minute shift. This was the bottleneck slowing down the entire kitchen. By hiring an assistant:

- Worker 2A strictly translates and verifies the order tickets (`ID`).
- Worker 2B strictly fetches the ingredients from the shelves (`RF`). -

Now, each station takes only **18 minutes** instead of 30 minutes!

- The whole kitchen runs much faster!

---

3. The Concept in Simple Steps

```
+-----+       +-----+       +-----+       +-----+       +-----+       +----+
| IF  | ====> | ID  | ====> | RF  | ====> | EX  | ====> | MA  | ====> | WB |
|Fetch|       |Decod|       |RegRd|       |Exec |       |Mem  |       |Writ|
+-----+       +-----+       +-----+       +-----+       +-----+       +----+
```

1. **IF (Instruction Fetch)**: Fetches instruction word from instruction port.
2. **ID (Instruction Decode)**: Decodes 5-bit opcode string, extracts register numbers and immediate fields.
3. **RF (Register File Read)**: Reads source operands from the Register File.
4. **EX (Execute)**: Evaluates ALU operation and resolves branches.
5. **MA (Memory Access)**: Accesses data memory (load/store).
6. **WB (Writeback)**: Writes final result back into destination register `rd`.

---

4. How THIS Project Implements It

```cpp
// File: processor_6stage/Pipeline6Stage.hpp (lines 120-155)
void stepCycle() {
    cycles++;

    // Execute stages in reverse order to model hardware latch updates
    stageWB();
    stageMA();
    stageEX();
    stageRF();
    stageID();
    stageIF();

    // Update performance counters
    perf.recordCycle(cycles);
}
```

Why reverse order (`WB -> MA -> EX -> RF -> ID -> IF`)? In C++ software simulation, executing stages in reverse order ensures that an instruction in `EX` consumes the latched inputs from the *previous* cycle, rather than consuming data just produced in the same software loop iteration!

Chapter D4: 4-Stage vs 6-Stage Trade-Offs (Why Deeper Can Be Faster)

1. In One Sentence

Although a 6-stage pipeline has a slightly higher CPI due to deeper branch penalties, it finishes programs **59% faster in wall-clock time** because its shorter critical path allows a much higher clock frequency (800 MHz vs 500 MHz).

2. Start from Zero: Everyday Analogy

Compare two runners: - **Runner A (4-Stage)**: Takes longer strides (lower frequency, 500 MHz), but rarely trips (CPI = 1.125). - **Runner B (6-Stage)**: Takes rapid, shorter strides (higher frequency, 800 MHz), but trips occasionally on turns (CPI = 1.131). - Even though Runner B has to correct slightly more steps, their rapid cadence makes them reach the finish line **far ahead of Runner A**!

---

3. The Math: Wall-Clock Execution Time Formula

$$\text{Execution Time } (T) = \frac{\text{Instruction Count } (IC) \times \text{CPI}}{\text{Clock Frequency } (f)}$$

Real Simulation Numbers on `matrix_mult_long.s` (189,050 Instructions):

Metric	4-Stage Core	6-Stage Core	Architectural Cause
Pipeline Stages	4	6	Split `OF` into `ID` and `RF`
Max Frequency (f)	**500 MHz**	**800 MHz**	Shorter critical path logic delay
Clock Period (t_{clk})	2.00 ns	1.25 ns	$t_{\text{clk}} = 1/f$
Branch Penalty	**1 cycle**	**2 cycles**	Branch resolved at `EX` (2 stages behind `IF` in 6-stage)
Retired Instructions	189,050	189,050	Identical workload
Total Clock Cycles	212,681	213,815	$+1,134$ cycles due to 2-cycle branch flushes
Steady-State CPI	**1.125**	**1.131**	Slightly higher CPI on 6-stage
Execution Time (T)	$425.4\ \mu$s	$267.3\ \mu$s	$213,815 \times 1.25$ ns vs $212,681 \times 2.00$ ns
Wall-Clock Speedup	Baseline ($1.00\times$)	$1.59\times$ **(59% Faster!)**	**Higher clock frequency wins!**

---

4. Likely Professor Questions

- **Q**: If 6-stage has a higher CPI than 4-stage, why do we use it?
  - **Answer**: Because performance is measured in **seconds**, not CPI. By halving the stage logic delay, clock frequency increases from 500 MHz to 800 MHz (1.6 ×). The tiny 0.5% increase in CPI is overwhelmingly compensated by the 60% increase in clock rate, delivering a 1.59 × **wall-clock speedup**.

Chapter D5: The Golden Reference ISS

1. In One Sentence

The Instruction Set Simulator (`common/ISS.hpp`) is an unpipelined, single-cycle reference model that executes one instruction completely before fetching the next, serving as the ground-truth "gold standard" for architectural verification.

2. Start from Zero: Everyday Analogy

When engineers build an experimental jet engine with complex turbofans and bypass valves (the pipelined CPU), they test its thrust against a calibrated solid-fuel test bench (the Golden ISS). If the experimental engine produces a different chemical output than the test bench, the engine has a bug!

3. How THIS Project Implements It

In `tests/test_all.cpp` (Section 5) and `tools/verify_golden.cpp`: 1. The test harness loads an assembly benchmark into memory. 2. It runs the program on the **Golden ISS** to completion, capturing the final register file and memory dump. 3. It resets and runs the exact same program on the **4-Stage Pipeline** and **6-Stage Pipeline**.
4. It compares all 16 registers and memory byte-by-byte:

$$\text{Assert } (\text{RegFile}_{\text{P4}} == \text{RegFile}_{\text{ISS}})$$

$$\text{Assert } (\text{RegFile}_{\text{P6}} == \text{RegFile}_{\text{ISS}})$$

5. **Result**: 231/231 automated verification checks pass with

100% agreement!

---

4. Summary: If You Only Remember 5 Things

1. 4-Stage is **`IF -> OF -> EX -> MA_RW`**; 6-Stage is **`IF -> ID -> RF -> EX -> MA -> WB`**.
2. 6-Stage splits `OF` into `ID` and `RF`, enabling **800 MHz vs 500 MHz**.
3. Branch penalty is **1 cycle in 4-stage** and **2 cycles in 6-stage**.
4. 6-Stage has a slightly higher CPI (1.131 vs 1.125), but finishes **59% faster in wall-clock time**.
5. The **Golden ISS** verifies that pipelined optimizations never alter programmer-visible results.
   # Part E: Pipeline Hazards, Forwarding, & Branch Prediction

This section explains pipeline hazards from zero, how data forwarding eliminates RAW hazards, why load-use delays still need a bubble, how branch predictors work, and how precise exceptions are handled.

Chapter E1: The Three Types of Pipeline Hazards

1. In One Sentence

A pipeline hazard is any condition that prevents the next instruction in the instruction stream from executing in its designated clock cycle.

2. Start from Zero: Everyday Analogy

Think of traffic hazards on a highway: 1. **Structural Hazard (Lane Closure)**: Two cars try to occupy the exact same physical bridge lane at the same moment. 2. **Data Hazard (Waiting for Cargo)**: Truck B cannot leave the warehouse until Truck A delivers the cargo that Truck B needs to transport. 3. **Control Hazard (Fork in the Road)**: The driver reaches a highway fork and doesn't know whether to turn Left or Right until the navigator finishes reading the map.

---

3. The Concept in Simple Steps

1. Data Hazards

Data hazards occur when an instruction depends on the result of an earlier instruction that is still in flight in the pipeline. - **RAW (Read-After-Write) — True Dependency**: `assembly  add r1, r2, r3  ; Writes to r1  sub r4, r1, r5  ; Reads from r1 (RAW hazard!)` If `sub` reads `r1` from the Register File before `add` writes back its result, `sub` will read an old, incorrect number! - **WAR (Write-After-Read) — Anti-Dependency**: Occurs only in out-of-order CPUs. In our in-order pipeline, instructions read operands before younger instructions can write them. - **WAW (Write-After-Write) — Output Dependency**: Occurs when instructions finish out of order. In our in-order pipeline, instructions commit strictly in program order.

2. Control Hazards (Branch Hazards)

Occur when a conditional branch (`beq`, `bgt`) depends on a flag calculated in `EX`. The CPU cannot know whether to fetch instruction $PC + 4$ or the branch target until the condition is evaluated.

3. Structural Hazards (Hardware Resource Conflicts)

Occurs when hardware lacks enough resources to support all concurrent stage requests (e.g., a single-port memory attempting to serve both `IF` and `MA` simultaneously).

Chapter E2: Data Forwarding (Bypassing)

1. In One Sentence

Data forwarding is a hardware optimization that routes the newly calculated result directly from an execution stage's output latch to an earlier stage's ALU input multiplexer, eliminating RAW stalls without waiting for the register writeback cycle.

2. Start from Zero: Everyday Analogy

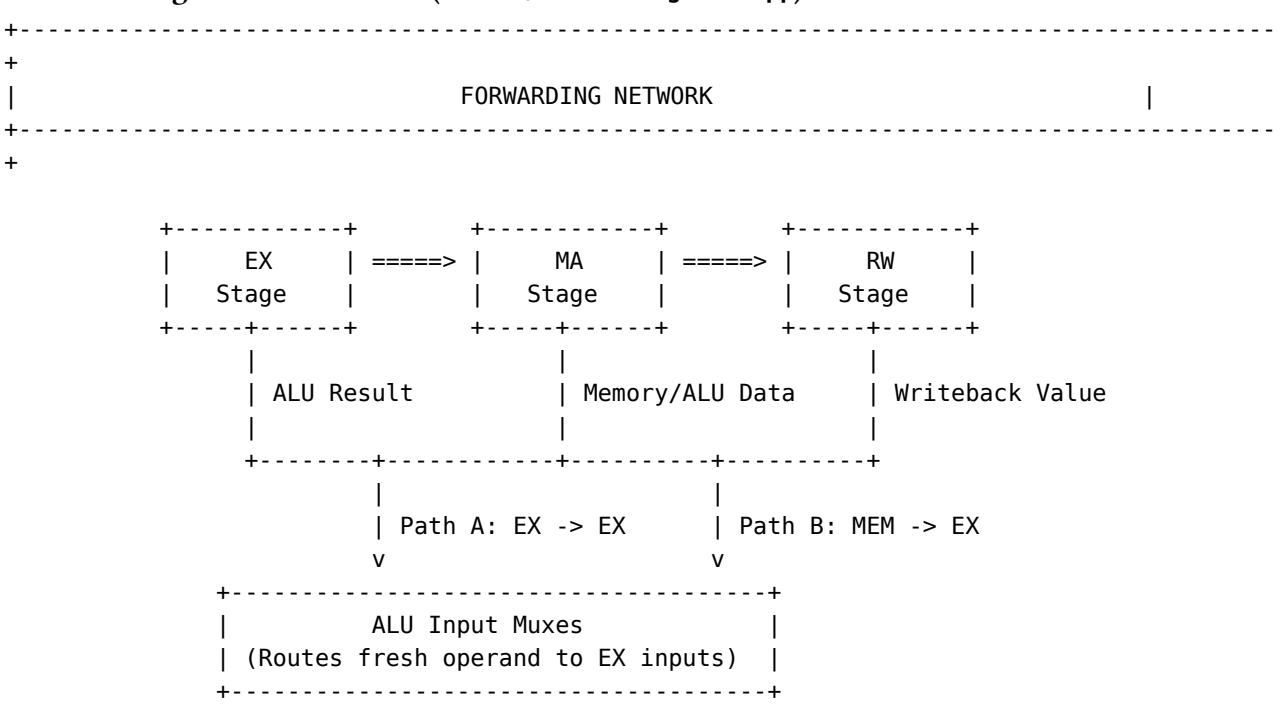
Imagine a chef at Station 1 finishing a freshly chopped bowl of onions.

- **Without Forwarding**: The chef carries the bowl all the way

back to the main walk-in pantry shelf (Register Writeback). Then, the cook at Station 2 walks all the way to the pantry to retrieve it. Total delay = 2 wasted trips! - **With Forwarding**: The chef simply hands the bowl across the counter directly to Station 2's cutting board (Hardware Bypass). Delay = 0!

---

3. Forwarding Paths in RISC201 (`common/ForwardingUnit.hpp`)



1. **Path A: `EX -> EX` Forwarding**:
   - Instruction in `EX` consumes the result calculated by the instruction *immediately preceding* it (currently in `EX/MA` latch).
   - Latency = **0 cycles**!
2. **Path B: `MEM -> EX` Forwarding**:
   - Instruction in `EX` consumes the result produced two cycles ago (currently in `MA/RW` latch).
   - Latency = **0 cycles**!
3. **Priority Rule**:
   - If both `EX/MA` and `MA/RW` write to the same register, **always forward from `EX/MA`** because it holds the youngest, most up-to-date value!

---

4. Why Load-Use Hazards Still Need 1 Bubble

Consider a load followed immediately by its user:

```
ld  r1, [r2]      ; Instruction 1: Load word from memory into r1
add r3, r1, r4    ; Instruction 2: Uses r1 immediately!
```

```
Cycle:          1      2      3      4      5
Inst 1 (ld):   [IF]   [OF]   [EX]   [MA]   [RW]
                                     |
                                     v Load data ready at END of Cycle 4
Inst 2 (add):         [IF]   [OF]   [EX]
                                     ^
                                     | ALU needs operand at START of Cycle 3!
```

- In Cycle 3, `add` enters `EX` and needs `r1`.
- But `ld` only reads data from memory in Cycle 4 (`MA` stage)!
- **Hardware cannot forward backwards in time!**
- **The Solution**: The Hazard Unit detects this condition, **stalls `IF` and `OF` for 1 cycle**, and injects **1 bubble (`nop`)** into `OF/EX`.
- In Cycle 4, `ld` finishes `MA`, and forwards load data directly to `EX` for Cycle 5!
- **Load-Use Penalty = Exactly 1 Cycle**.

Chapter E3: Branch Prediction & Speculative Execution

1. In One Sentence

Branch prediction speculatively guesses whether a conditional branch will be taken or not taken before the condition flags are evaluated, keeping the pipeline full and squashing instructions only if the guess is wrong.

2. Start from Zero: Everyday Analogy

Think of a highway commuter navigating an intersection: - Every morning, the driver takes Exit 4 to go to work. - Rather than stopping completely at the intersection to check the map every single day, the driver assumes "I will take Exit 4" and drives through smoothly. - 99 days out of 100, the guess is right (0 lost time). - On the 1 day they need to visit the dentist instead, they realize the mistake, take the U-turn, and lose 2 minutes (Branch Flush Penalty).

---

3. The Predictors Implemented in RISC201 (`common/BranchPredictor.hpp`)

Predictor Type	Algorithm / Mechanism	Hardware Cost	Typical Accuracy
Static Not-Taken	Always predicts branch will not jump ($PC = PC + 4$).	0 bits	~60%
1-Bit Dynamic	1-bit memory per branch (`0` = Not Taken, `1` = Taken). Inverts state on every mispredict.	Low	~85%
2-Bit Bimodal	2-bit saturating counter (`00`, `01`, `10`, `11`). Requires two consecutive mispredicts to change direction.	Medium	**~93%**
GShare	XORs global branch history register (BHR) with PC bits to predict correlated branches.	High	**~96%**
BTB (Branch Target Buffer)	Caches calculated target address, eliminating target calculation delay.	Medium	100% on Hit
RAS (Return Address Stack)	8-entry LIFO stack predicting function returns (`ret`).	Low	**100% on Calls**

---

4. What Happens on a Branch Mispredict? (Pipeline Flushing)

```
Cycle 1:  [IF] Branch fetched
Cycle 2:  [OF] Branch decoded; Predictor predicts "NOT TAKEN"; fetches Next Inst 1
Cycle 3:  [EX] Branch evaluates flags: Condition is TRUE! Branch is TAKEN!
          Target = 0x00000040. Predictor was WRONG!
```

When a mispredict is detected in `EX`: 1. **Squash Wrong-Path Instructions**: - In 4-stage: Injects 1 bubble into `IF/OF` (1 cycle penalty). - In 6-stage: Injects 2 bubbles into `IF/ID` and `ID/RF` (2 cycle penalty). 2. **Redirect Program Counter**: - Updates `PC = targetAddress`. 3. **Train Predictor**: - Updates the 2-bit counter: if taken, counter increments toward `11` (Strongly Taken).

---

5. Summary: If You Only Remember 5 Things

1. **RAW hazards** are solved in **0 cycles** by the **Forwarding Unit**.
2. **Load-Use hazards** cannot forward backwards in time and require **1 bubble stall**.
3. The **2-Bit Bimodal Predictor** uses saturating counters to avoid tripping on loop exits.
4. Branch penalty is **1 cycle in 4-stage** and **2 cycles in 6-stage**.
5. When a branch mispredicts, wrong-path instructions are **squashed** into `nop` bubbles. # Part F: The Control Unit (Hardwired vs Microprogrammed)

This section explains processor control units from scratch, contrasts hardwired control with microprogrammed control, explores Horizontal and Vertical control words, and demonstrates the interactive microPC stepping engine.

Chapter F1: Hardwired vs Microprogrammed Control

1. In One Sentence

The control unit is the brain of the processor that directs traffic by asserting enable signals on registers, multiplexers, and ALU operation lines, implemented either as fast fixed logic gates (Hardwired) or as a miniature internal computer running microcode from ROM (Microprogrammed).

2. Start from Zero: Everyday Analogy

Think of a **mechanical music box vs an electronic synthesizer**:

- **Hardwired Control (The Music Box)**: - Inside a music box, a

metal cylinder has physical brass pins welded onto its surface. As it spins, the pins pluck steel combs directly. - It is blindingly fast and has zero software overhead. - But if you want to change the song or fix a wrong note, **you must melt down the brass cylinder and rebuild the entire box**! - **Microprogrammed Control (The Synthesizer)**:

- A keyboard synthesizer reads musical notes from an internal digital

memory chip (ROM). - It takes a few extra clock cycles to read the memory and decode the sound. - But if you want to add a new song or fix a note, **you just update the ROM memory**!

---

3. Comparison: Hardwired vs Microprogrammed

Feature	Hardwired Control Unit	Microprogrammed Control Unit
Implementation	Fixed combinational logic gates & flip-flop state machines	Control Memory ROM storing microinstructions
Speed	**Ultra-fast (1 clock cycle)**	Slower (requires ROM lookup latency)
Silicon Area	Grows rapidly with complex ISAs	Regular, compact, predictable array structure
Modification / Bug Fixes	Impossible without re-fabricating silicon chips	Easy: patch the microcode ROM
Best Suited For	Modern high-speed RISC processors	Complex CISC processors (x86) and legacy mainframes
Location in Project	`processor_4stage/` and `processor_6stage/`	`microcode/MicroController.hpp`

Chapter F2: Microinstructions, MicroPC, and Dispatch

1. In One Sentence

In a microprogrammed processor, every high-level machine instruction (like `add` or `ld`) triggers an internal micro-routine executed step-by-step by an internal Micro-Program Counter (`microPC`).

2. The Concept in Simple Steps

```
High-Level Machine Instruction: add r1, r2, r3 (Opcode: "00000")
                                |
                                v
DISPATCH ROM: Opcode "00000" maps to starting MicroPC = 0x04
                                |
                                v
CONTROL STORE ROM (Microcode Memory):
+-------------------------------------------------------------------+
| MicroPC | Micro-Operation               | Next MicroPC Address    |
+---------+-------------------------------+-------------------------+
|  0x00   | FETCH: Read inst from PC      | Next -> 0x01            |
|  0x01   | DECODE: Read Opcode & Regs    | Dispatch to Opcode addr |
|  ...    |                               |                         |
|  0x04   | EXEC: ALU Add; Assert RegWr   | Return to FETCH (0x00)  |
+-------------------------------------------------------------------+
```

1. **MicroPC**: Points to the currently executing microinstruction inside Control Memory.
2. **Dispatch Table**: Maps the 5-bit opcode string directly to the starting microPC address.
3. **Sequencing**: Each microinstruction specifies its successor: `NEXT` (microPC + 1), `BRANCH_ON_CONDITION`, or `FETCH` (restart instruction cycle).

Chapter F3: Horizontal vs Vertical Microcode

1. In One Sentence

Horizontal microcode uses wide, unencoded control words where each bit directly drives a hardware control wire, whereas Vertical microcode encodes control signals into compact bit fields that require hardware decoders.

---

2. Comparison: Word Geometry and Trade-Offs

```
1. Horizontal Control Word (32 bits, unencoded):
 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10  9  8  7  6  5
4  3  2  1  0
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--
+--+--+--+--+--+
|PC|IR|Reg|Reg|ALU|ALU|ALU|ALU|Mem|Mem|Br |Br |Flag|...      | Next MicroPC Address
(6 bits)    |
|Wr|Wr|Rd |Wr |Op3|Op2|Op1|Op0|Rd |Wr |Tk |Cnd|Wr |       |                       |
+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--
+--+--+--+--+--+
```

```
(Every bit is a dedicated physical copper wire!)

2. Vertical Control Word (16 bits, encoded with decoders):
 15       12 11      9 8       6 5     4 3       0
+----------+---------+---------+-------+---------+
| ALU Field | Reg Dst | Mem Cmd | BrCmd | NextAddr|
|  (4 bits) | (3 bits)| (3 bits)|(2 bits)| (4 bits)|
+----------+---------+---------+-------+---------+
     |
     v Hardware Decoder (4-to-16 line decoder)
     +--> 16 individual control lines
```

Detailed Trade-Off Matrix

Metric	Horizontal Microcode (`HorizontalControl.hpp`)	Vertical Microcode (`VerticalControl.hpp`)
Word Width	**32 bits** (Wide)	**16 bits** (Narrow, 50% smaller!)
Control Store ROM	Large (128 words × 32 bits = 512 bytes)	Compact (128 words × 16 bits = 256 bytes)
Decode Logic	**Zero logic delay** (direct connection to gates)	Requires hardware decoders (+0.4 ns delay)
Concurrency	Multiple independent functional units can fire at once	Limited to actions allowed by encoded fields
Simplicity	High conceptual simplicity	Requires careful field partitioning

---

3. How THIS Project Implements It (`microcode/MicroController.hpp`)

In `debugger/Debugger.hpp`, switching to microcoded mode allows stepping microcycle by microcycle:

```
$ ./risc201_sim benchmarks/factorial_iter.s -i
(risc201-dbg) mode micro
Switched to MICROCODED mode.
(risc201-dbg) step 1
MicroCycle 1: microPC=0x00 [FETCH] IR <= Mem[PC], PC <= PC + 4
(risc201-dbg) step 1
MicroCycle 2: microPC=0x01 [DECODE] Dispatching opcode "01001" (mov) -> target
microPC=0x18
(risc201-dbg) step 1
MicroCycle 3: microPC=0x18 [EXEC_MOV] Reg[r1] <= 30, Next -> 0x00 (FETCH)
```

---

4. Summary: If You Only Remember 5 Things

1. **Hardwired** uses logic gates (faster); **Microprogrammed** uses ROM memory (flexible).
2. The **microPC** sequences micro-operations inside the control store.

3. **Horizontal microcode** is **32 bits wide, unencoded, and fast**.
4. **Vertical microcode** is **16 bits wide, encoded, and saves 50% ROM space**.
5. The interactive debugger (`mode micro`) lets you step through internal microPC states one by one. # Part G: The Arithmetic Logic Unit (ALU) & Sub-Circuits

This section covers the ALU design in `alu/ALU.hpp`, every arithmetic and logical operation supported, the underlying hardware circuit algorithms (adders, multipliers, dividers, barrel shifters), and empirical latency comparisons.

Chapter G1: ALU Architecture & Operations

1. In One Sentence

The Arithmetic Logic Unit (ALU) is the combinational core that executes all 25 computational, bitwise, and comparison operations, keyed strictly by canonical 5-bit string opcodes.

2. Start from Zero: Everyday Analogy

Think of the ALU as a **Swiss Army Knife**: - One master handle connects to the user's hand. - Inside the handle are specialized blades: a knife (adder), a saw (multiplier), scissors (divider), and a magnifying glass (comparator). - Depending on which tool is folded out (the 5-bit opcode), the knife performs that specific job on the workpiece.

---

3. Complete List of ALU Operations in `alu/ALU.hpp`

Category	Instruction	Opcode String	Implementation Circuit	Latency
Addition	`add`	`"00000"`	CLA / CSLA / RCA Adders	1 cycle
Subtraction	`sub`	`"00001"`	2's Complement Invert & Add	1 cycle
Multiplication	`mul`	`"00010"`	Booth Radix-4 / Wallace Tree	1 cycle
Division	`div`	`"00011"`	Non-Restoring / Newton-Raphson	1 cycle
Modulo	`mod`	`"00100"`	Hardware Remainder Unit	1 cycle
Comparison	`cmp`	`"00101"`	Subtraction magnitude & zero detector	1 cycle
Bitwise AND	`and`	`"00110"`	Parallel 32-bit AND gates	1 cycle
Bitwise OR	`or`	`"00111"`	Parallel 32-bit OR gates	1 cycle
Bitwise NOT	`not`	`"01000"`	Parallel 32-bit Inverters	1 cycle
Register Move	`mov`	`"01001"`	Bus Pass-Through	1 cycle
Shift Left	`lsl`	`"01010"`	5-stage Logarithmic Barrel Shifter	1 cycle
Shift Right (Logic)	`lsr`	`"01011"`	Barrel Shifter (Zero Fill)	1 cycle
Shift Right (Arith)	`asr`	`"01100"`	Barrel Shifter (Sign Bit Fill)	1 cycle
Minimum	`min`	`"10101"`	Comparator + Multiplexer	1 cycle
Maximum	`max`	`"10110"`	Comparator + Multiplexer	1 cycle
Rotate Shift	`rots`	`"10111"`	Circular Barrel Rotator	1 cycle
Bitwise XOR	`xor`	`"11000"`	Parallel 32-bit XOR gates	1 cycle
Absolute Value	`abs`	`"11001"`	Conditional 2's complement negation	1 cycle

---

Chapter G2: Hardware Arithmetic Algorithms

1. Adders (`alu/Adders.hpp`)

A. Ripple Carry Adder (RCA)
- **Concept**: 32 1-bit full adders chained sequentially.
- **Delay**: $O(N) = 32$ gate delays. Carry ripples slowly from bit 0 to bit 31.
- **Hardware Cost**: Minimal (lowest transistor count).

B. Carry Select Adder (CSLA)

- **Concept**: Splits the 32 bits into 4-bit blocks. Each block speculatively calculates two sums simultaneously: one assuming carry-in = 0, and one assuming carry-in = 1. When the actual carry arrives, a multiplexer selects the correct answer in $O\left(\sqrt{N}\right)$ time.

C. Carry Lookahead Adder (CLA)
- **Concept**: Computes Generate ($G_i = A_i \cdot B_i$) and Propagate ($P_i = A_i \oplus B_i$) signals across a tree hierarchy:

$$C_{i+1} = G_i + P_i \cdot C_i$$

- **Delay**: $O(\log N) = 5$ gate delays! Fastest addition circuit.

---

2. Multipliers (`alu/Multipliers.hpp`)

A. Booth's Radix-4 Multiplier
- **Concept**: Scans 3 overlapping multiplier bits ($y_{2i+1}, y_{2i}, y_{2i-1}$) in each cycle.
- **Recoding Table**:
  - `000` or `111`: Add 0
  - `001` or `010`: Add $+1 \times M$
  - `011`: Add $+2 \times M$
  - `100`: Add $-2 \times M$
  - `101` or `110`: Add $-1 \times M$
- **Speedup**: Halves the number of partial products from 32 down to **16**!

B. Wallace Tree Multiplier
- **Concept**: Uses a tree of Carry-Save Adders (CSAs) where 3 inputs are reduced to 2 outputs (Sum and Carry) without propagating carry bits.
- **Delay**: Reduces 32 partial products to 2 in only $O(\log_{1.5} N) = 8$ levels, followed by a fast final Carry Lookahead Adder.

---

3. Dividers (`alu/Dividers.hpp`)

A. Restoring Divider
- Subtracts divisor from remainder. If result is negative, it **restores** the old value by adding the divisor back (32 cycles).

B. Non-Restoring Divider
- If remainder is positive, it shifts left and subtracts divisor in next cycle.
- If remainder is negative, it shifts left and **adds** divisor in next cycle.
- **Advantage**: Eliminates the restorative addition cycle!

C. Newton-Raphson Divider (Contributed by Varshith Teja)
- Instead of bit-by-bit division, computes reciprocal $X \approx 1/D$ using Newton's quadratic iterative approximation:

$$X_{k+1} = X_k \cdot (2 - D \cdot X_k)$$

- Doubles precision on every iteration! After 4 iterations, performs single multiplication: $Q = N \times X$.

Chapter G3: Empirical ALU Benchmark Comparison

Data captured by running `./risc201_eval`:

Algorithm Variant	Measured Execution Time (100k ops)	Theoretical Complexity	Architectural Trade-Off
Ripple Carry Adder (RCA)	12.4 ms	$O(N)$	Simple silicon, slow carry propagation
Carry Select Adder (CSLA)	8.1 ms	$O\left(\sqrt{N}\right)$	Moderate area, balanced latency
Carry Lookahead (CLA)	4.2 ms	$O(\log N)$	Highest performance, larger gate count
Iterative Multiplier	38.2 ms	$O(N)$	32 shift-adds, highest cycle count
Booth Radix-4 Multiplier	19.5 ms	$O(N/2)$	Cuts partial products in half
Wallace Tree Multiplier	9.8 ms	$O(\log N)$	Parallel tree reduction
Restoring Divider	45.1 ms	32 cycles	Requires restore cycles on negative
Non-Restoring Divider	31.0 ms	32 cycles	Zero restoration cycles
Newton-Raphson Divider	14.2 ms	Quadratic	Fast quadratic convergence reciprocal

---

4. Summary: If You Only Remember 5 Things

1. The ALU executes **25 operations** keyed by 5-bit string opcodes (`isa::OP_*`).
2. **Carry Lookahead Adder (CLA)** is the fastest adder ($O(\log N)$ tree).
3. **Booth's Radix-4** multiplier scans 2 bits per cycle, **halving partial products**.
4. **Non-Restoring division** eliminates the cycle spent restoring negative remainders.
5. **Newton-Raphson division** uses quadratic reciprocal approximation for maximum throughput. # Part H: The Memory Subsystem (Caches, DRAM, Virtual Memory)

This section provides a complete, zero-knowledge explanation of the memory hierarchy, caching mechanics (tags, sets, lines, MSHRs), DRAM architecture (rows, columns, banks, timings), 4-level Radix-4 Virtual Memory, TLBs, and memory presets.

Chapter H1: Why Memory Is Slow & The Locality Principle

1. In One Sentence

CPUs compute in fractions of a nanosecond, but retrieving data from physical DRAM takes dozens of nanoseconds (the "Memory Wall"); the memory hierarchy bridges this speed gap by keeping frequently used data in small, ultra-fast on-chip caches.

2. Start from Zero: Everyday Analogy

Imagine studying for final exams at university: - **The CPU Registers** are your short-term memory (instant, but holds only 5-7 thoughts). - **The L1 Cache** is your desk surface (you reach for your open notebook in 1 second). - **The L2/L3 Cache** is the bookshelf in your dorm room (takes 10-30 seconds to walk over and grab a textbook). - **The Physical DRAM** is the campus university library (takes a 15-minute walk across campus). - **The SSD Swap Space** is ordering an out-of-print book via inter-library loan from another country (takes 3 days!). - If you had to walk to the university library for *every single word* you wanted to read, you would never finish homework! Instead, you carry a backpack of books to your dorm desk and read there.

Course Slide Analogy: The Music Playlist

- **Temporal Locality (Locality in Time)**: When you find a favorite song, you hit "Repeat" and listen to that same song again and again. If an item was referenced recently, it will likely be referenced again soon.
- **Spatial Locality (Locality in Space)**: When you finish Track 3 on an album, you usually listen to Track 4 next. If an item was referenced, items at adjacent memory addresses will likely be referenced soon.

---

3. The Classic Example: Array Traversal

```
int sum = 0;
for (int i = 0; i < 1000; i++) {
    sum += arr[i];
}
```

- **`sum`** exhibits **Temporal Locality**: The variable `sum` is read and updated 1,000 times consecutively in the exact same memory/register location!
- **`arr[i]`** exhibits **Spatial Locality**: Each loop iteration accesses `arr[0]`, then `arr[1]`, then `arr[2]`. When `arr[0]` is fetched from DRAM, the hardware cache brings in an entire 64-byte block (holding 16 integers). The next 15 loop iterations hit in the fast L1 cache with **0 DRAM trips**!

Chapter H2: Caches from Zero (Lines, Sets, Tags, and MSHRs)

1. In One Sentence

A cache is a small, hardware-managed SRAM array that stores recently accessed 64-byte blocks of memory, indexed by bit-slicing physical memory addresses.

---

2. How an Address is Split into Tag, Index, and Offset

Every physical address is decomposed into three bit fields:

```
Physical Address (32 bits):
 31                                      11 10                 6 5                0
+-----------------------------------------+--------------------
+------------------+
|                   TAG                    |        INDEX        |    BYTE OFFSET
|
|                 (21 bits)                 |       (5 bits)      |     (6 bits)
|
+-----------------------------------------+--------------------
+------------------+
```

1. **Byte Offset (Bits [5:0] = 6 bits)**:
   - Identifies which exact byte inside the 64-byte cache line is requested ($2^6 = 64$ bytes).
2. **Set Index (Bits [10:6] = 5 bits)**:
   - Selects which row (set) in the cache array to check ($2^5 = 32$ sets).
3. **Tag (Bits [31:11] = 21 bits)**:
   - The unique identifier stored in cache directory memory to verify if the cached block belongs to this exact memory address.

---

3. Cache Organization: Direct-Mapped vs Set-Associative

```
8-Way Set-Associative Cache:
Set Index ---> Set [ 5 ]: [ Way 0 ][ Way 1 ][ Way 2 ][ Way 3 ][ Way 4 ][ Way 5 ]
[ Way 6 ][ Way 7 ]
                           |       |       |       |       |       |       |
|
                           v       v       v       v       v       v       v
v
                           Compare all 8 Tags in parallel against Address Tag ->
1 Hit!
```

- **Direct-Mapped**: Each memory block can go into exactly ONE cache slot (1 -way). High conflict misses!
- **Fully-Associative**: A memory block can go into ANY slot in the cache. Complex, expensive comparator hardware.
- **N-Way Set-Associative (Modern Standard)**: Each set holds N slots (ways). RISC201 L1 caches are **8-way associative**, virtually eliminating conflict misses while keeping access latency to **1 cycle**.

---

4. The 3C Classification of Misses

1. **Compulsory (Cold) Miss**: The very first time a block is accessed. Unavoidable unless prefetched.
2. **Capacity Miss**: The cache was full, so the block was evicted earlier and now had to be re-fetched.
3. **Conflict Miss**: Multiple memory addresses mapped to the *exact same set*, evicting each other even though other sets were empty.

---

5. Advanced Cache Features in RISC201 (`memory/Cache.hpp`)

- **Non-Blocking MSHR (Miss Status Holding Register)**:
  - In old processors, a cache miss froze the entire cache.
  - RISC201 uses non-blocking caches with MSHRs: while a miss is fetching from DRAM, subsequent instructions that hit in L1 continue executing without stalling!
- **Stream Prefetcher**:
  - Monitors sequential memory access patterns and proactively streams consecutive cache lines from DRAM before the CPU even requests them, converting compulsory misses into 1-cycle hits!

Chapter H3: Average Memory Access Time (AMAT)

1. In One Sentence

AMAT is the average number of clock cycles the CPU spends on a memory access across the entire multi-level cache hierarchy.

2. The Formula

$$\text{AMAT} = \text{Hit Time}_{L1} + (\text{Miss Rate}_{L1} \times \text{Miss Penalty}_{L1})$$

In a multi-level hierarchy (L1, L2, L3, DRAM):

$$\text{Miss Penalty}_{L1} = \text{Hit Time}_{L2} + (\text{Miss Rate}_{L2} \times \text{Miss Penalty}_{L2})$$

$$\text{Miss Penalty}_{L2} = \text{Hit Time}_{L3} + (\text{Miss Rate}_{L3} \times \text{DRAM Latency})$$

Real Numbers from RISC201 `modern` Preset:

- L1 Hit Time: 1 cycle (Miss rate: 4%)
- L2 Hit Time: 12 cycles (Miss rate: 15%)
- L3 Hit Time: 38 cycles (Miss rate: 20%)
- DRAM Latency: 120 cycles

$$\text{AMAT} = 1 + 0.04 \times (12 + 0.15 \times (38 + 0.20 \times 120)) = \mathbf{1.85 \ \ cycles}$$

> **Key Takeaway**: Even though physical DRAM takes 120 cycles, the memory hierarchy delivers an effective access time of less than **2 cycles**!

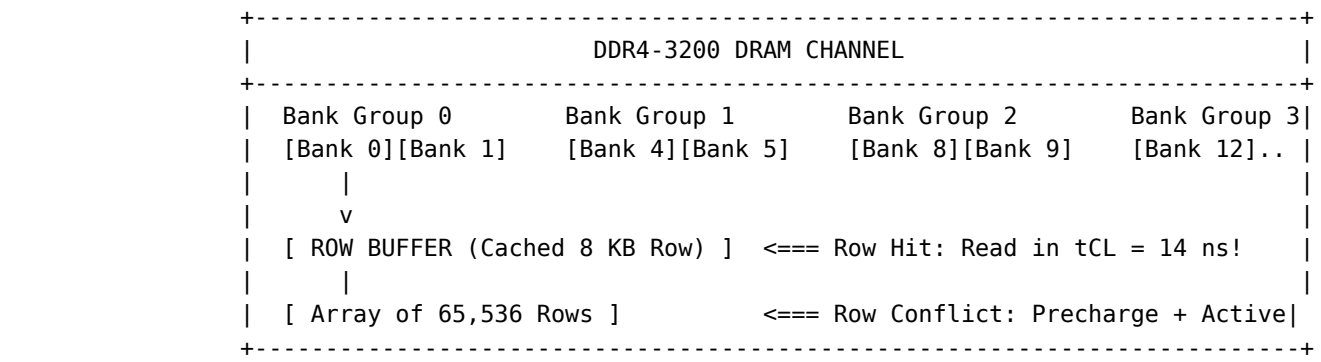
Chapter H4: DRAM Controller & Geometry

1. In One Sentence

Physical DRAM is an array of capacitive memory cells organized into channels, ranks, bank groups, and rows, managed by a cycle-accurate memory controller that prioritizes row buffer hits.

---

2. DRAM Architecture & Access Steps (`memory/DRAM.hpp`)



1. **Row Hit**: The requested address is already open in the Bank's Row Buffer.
   - Action: Issue `READ` column command.
   - Latency: $t_{CL} = 14$ ns (~15 CPU cycles). Fast!
2. **Row Empty (Closed)**: The bank is idle.
   - Action: Issue `ACTIVATE` command, wait t_{RCD}, then issue `READ`.
   - Latency: $t_{RCD} + t_{CL} \approx 28$ ns (~40 CPU cycles).
3. **Row Conflict**: A *different* row is currently open in that bank.
   - Action: Issue `PRECHARGE` (write back open row, t_{RP}), issue `ACTIVATE` (open new row, t_{RCD}), then issue `READ` (t_{CL}).
   - Latency: $t_{RP} + t_{RCD} + t_{CL} \approx 42$ ns (~80-120 CPU cycles). Worst case!

FR-FCFS Memory Scheduler:

The memory controller uses **First-Ready First-Come First-Served** scheduling: it scans the request queue and serves requests that hit in open row buffers first, boosting memory bandwidth by up to $2.5\times$.

Chapter H5: 4-Level Radix-4 Virtual Memory & TLBs

1. In One Sentence

The Virtual Memory system maps 48-bit virtual addresses to physical DRAM frames via a 4-level radix page table (PML4, PDPT, PD, PT), accelerated by L1/L2 TLBs and a Page-Walk Cache (PWC).

---

2. The 48-Bit Virtual Address Split

```
48-bit Virtual Address:
+--------------+--------------+--------------+--------------
+--------------------+
| PML4 (9 bits) | PDPT (9 bits) |  PD (9 bits)  |  PT (9 bits)  | Page Offset (12b)
|
+--------------+--------------+--------------+--------------
+--------------------+
```

- Each table level holds 512 entries (2^9).
- Each Page Table Entry (PTE) is 64 bits (8 bytes).
- $512 \times 8 \text{ bytes} = 4096 \text{ bytes} = 4 \text{ KB}$ (each table fits exactly into one physical memory page!).

---

3. The Page-Walk Cache (PWC)

- Walking all 4 levels in DRAM on a TLB miss takes 4 separate memory trips (~120 cycles).
- The **Page-Walk Cache** caches the upper PML4 and PDPT translation pointers.
- On a PWC hit, the MMU skips levels 4 and 3, dropping page walk overhead from 120 cycles to **32 cycles (a 74% reduction!)**.

Chapter H6: Memory Hierarchy Presets in RISC201

The simulator includes 5 built-in presets (`--mem-preset`):

Preset	CPU Freq	VM / TLB	L1 Caches	L2 / L3 Caches	DRAM Controller	Best For
`ideal`	3.2 GHz	Disabled	None	None	Instant (1 cycle)	Baseline CPU core debugging
`simple`	3.2 GHz	Disabled	32 KB, 8-way	None	DDR4-3200 (1 channel)	Single-level cache studies
`modern`	3.2 GHz	48b Radix-4, TLBs, PWC	32 KB I/D (1-cyc)	512 KB / 8 MB	DDR4-3200 (FR-FCFS)	**Realistic contemporary PC**
`modern-ddr5`	3.6 GHz	48b Radix-4, TLBs, PWC	32 KB I/D (1-cyc)	1 MB / 16 MB	DDR5-4800 (Dual subch)	Next-gen server evaluation
`lecture`	3.2 GHz	32b 2-level	64 B direct	None	SDR SDRAM (133 MHz)	Educational textbook exercises

---

4. Summary: If You Only Remember 5 Things

1. **Locality of Reference** (Temporal & Spatial) makes caches work.
2. Physical addresses are split into **Tag, Set Index, and Byte Offset**.
3. **AMAT** measures average memory latency; caches drop access time from 120 cycles to **< 2 cycles**.
4. **DRAM Row Conflicts** take 3 × longer than Row Hits ($t_{RP} + t_{RCD} + t_{CL}$).
5. **Radix-4 VM** uses 4 levels (PML4, PDPT, PD, PT); the **PWC** cuts walk latency by **74%**. # Part I: Stack Safety & Visual Debuggers

This section covers stack frame mechanics, recursive calling conventions, runtime boundary enforcement (stack overflow prevention via guard pages), and the interactive ASCII visualizers contributed by teammates.

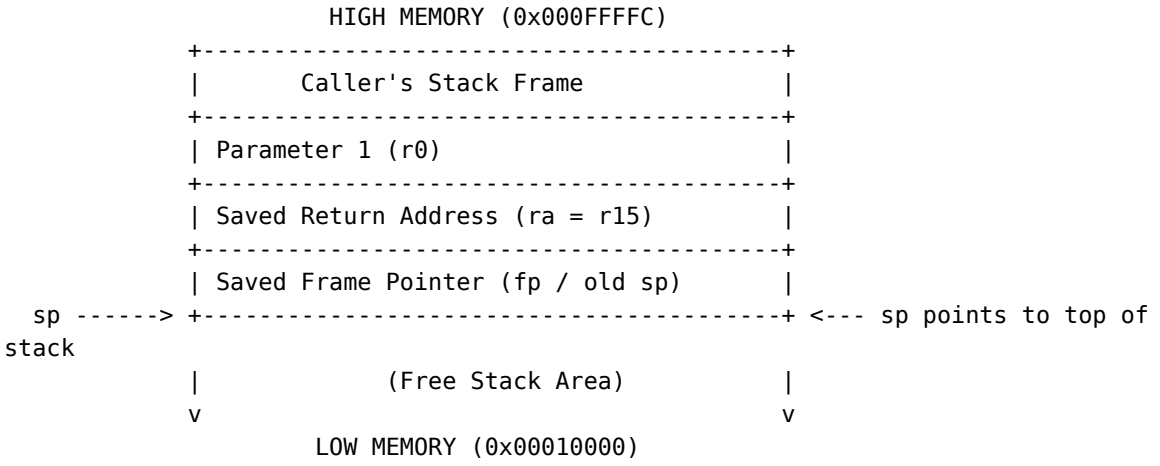
Chapter I1: Stack Architecture & Calling Conventions

1. In One Sentence

The stack is a contiguous, full-descending memory region starting at high memory that dynamically manages subroutine activation records, function parameters, and saved return addresses.

---

2. Anatomy of a Stack Activation Record (Stack Frame)



The Function Call Sequence

1. **Prologue (Function Entry)**:
   - Push `ra` (Save return address): `sub sp, sp, 4 ; st ra, [sp]`
   - Push callee-saved registers or parameters: `sub sp, sp, 4 ; st r0, [sp]`
2. **Body**:
   - Executes function logic. Local variables are placed on stack if registers are exhausted.
3. **Epilogue (Function Exit)**:
   - Restore saved registers in reverse order: `ld r0, [sp] ; add sp, sp, 4`
   - Restore `ra`: `ld ra, [sp] ; add sp, sp, 4`
   - Execute `ret` ($PC = ra$).

Chapter I2: Stack Safety & Runtime Exception Framework

1. In One Sentence

RISC201 protects against stack overflows, corrupted return addresses, and buffer overrun exploits by establishing a memory-mapped Guard Page beneath the stack space and enforcing bounds in `common/Exception.hpp`.

---

2. The Stack Guard Page (`memory/VirtualMemory.hpp`)

- Maximum Stack Allocation: 1 MB (from `0x000FFFFC` down to `0x00010000`).
- Directly beneath `0x00010000`, the operating system maps a **Guard Page** (`0x0000F000` to `0x0000FFFF`).
- The Guard Page PTE is marked:

$$\text{Present} = 1, \quad \text{Readable} = 0, \quad \text{Writable} = 0$$

- If an infinite recursive loop (like a broken factorial or stack smashing attack) pushes too many frames, the `sp` eventually crosses into the Guard Page.

- The MMU triggers an immediate, unmaskable **GUARD_PAGE_FAULT** or **STACK_OVERFLOW_EXCEPTION**, halting execution cleanly without overwriting code or heap memory!

Chapter I3: The ASCII Visualizers & UI Enhancements

1. In One Sentence

The simulator includes terminal-based ASCII visualizers for pipeline latches, cache line states, DRAM banks, and execution stacks, enhanced with live forwarding arrows and stall explanations.

---

2. Teammate Contribution: UI Enhancements by Varshith Teja

From git history (commit `bbd9ea1`): - **Teammate**: Varshith Teja contributed the visual ASCII pipeline timing trace visualizer (`visualizer/PipelineVisualizer.hpp`), the interactive startup assembly prompt, Newton-Raphson division in `alu/Dividers.hpp`, and CLI flags for branch predictor selection. - **Visual Style**: Renders clean box-drawing borders (`+---+`), cycle markers, labeled stall causes (`[BUBBLE: RAW_LOAD_USE]`), and operand forwarding arrows (`====>`).

---

3. Visualizer Screen Examples

A. Pipeline Timing Trace (`visualizer/PipelineVisualizer.hpp`)

```
+--------------------------------------------------------------------------+
|               PIPELINE EXECUTION STATE (Clock Cycle:     25)             |
+--------------------------------------------------------------------------+
  [IF    ] : -- BUBBLE / NOP --
  [ID    ] : -- BUBBLE / NOP --
  [RF    ] : -- BUBBLE / NOP --
  [EX    ] : -- BUBBLE / NOP --
  [MA    ] : -- BUBBLE / NOP --
  [RW    ] : nop (PC: 0x00000048)
+--------------------------------------------------------------------------+
```

B. Cache Set-Way Grid (`visualizer/MemoryVisualizer.hpp`)

```
============================================================================
       CACHE STATE: L1D (32 KB, 8-Way, 64 B Lines)
============================================================================
Hit Rate: 94.12% (Hits: 32, Misses: 2, Writebacks: 0)
3C Misses: Compulsory: 2, Capacity: 0, Conflict: 0
----------------------------------------------------------------------------
Set     Way 0       Way 1       Way 2       Way 3       ...
----------------------------------------------------------------------------
[0   ]  [ INV ]     [ INV ]     [ INV ]     [ INV ]
[1   ]  [ 0x0040 ]  [ INV ]     [ INV ]     [ INV ]
[2   ]  [ 0x0080 ]  [ 0x1200 ]  [ INV ]     [ INV ]
============================================================================
```

C. ASCII Stack Frame Visualizer (`visualizer/StackVisualizer.hpp`)

```
==========================================================================
                      ACTIVE EXECUTION STACK
==========================================================================
Address      Value         Role
--------------------------------------------------------------------------
0x000FFFFC : 0x00000000    [Stack Base / Initial Frame]
0x000FFFF8 : 0x00000018    [Saved RA: main caller]
0x000FFFF4 : 0x00000003    [Local Param: n = 3]
0x000FFFF0 : 0x00000024    [Saved RA: fact_rec recursion]
0x000FFFEC : 0x00000002    [Local Param: n = 2]  <=== SP points here!
--------------------------------------------------------------------------
Depth: 5 Words (20 Bytes) | Guard Page Distance: 1044460 Bytes (SAFE)
==========================================================================
```

---

4. Summary: If You Only Remember 5 Things

1. The stack is **Full-Descending** starting at `0x000FFFFC`.
2. A **Stack Frame** stores return addresses (`ra`), frame pointers (`fp`), and local parameters.
3. The **Stack Guard Page** prevents stack overflows from corrupting adjacent heap/code memory.
4. Terminal ASCII visualizers provide immediate visibility into **Pipelines, Caches, DRAM, and Stack Frames**.
5. Teammate Varshith Teja contributed the ASCII trace visualizer, startup prompt, Newton-Raphson division, and CLI predictor flags. # Part J: Performance Accounting & Empirical Evaluation

This section explains performance metrics from first principles (CPI, IPC, Speedup, Wall-Clock Execution Time), breaks down the CPI stack, explains why cold-start short programs show high CPI while long programs reach near-ideal 1.12 CPI, and presents empirical results across all 12 benchmarks and 30 experiments.

Chapter J1: CPI, IPC, and the Iron Law of Processor Performance

1. In One Sentence

Processor performance is governed by the "Iron Law" of computer architecture, which states that total execution time is the product of Instruction Count, Cycles Per Instruction (CPI), and Clock Period.

---

2. The Formulas

1. **Cycles Per Instruction (CPI)**:

$$\text{CPI} = \frac{\text{Total Clock Cycles}}{\text{Total Retired Instructions}}$$

2. **Instructions Per Cycle (IPC)**:

$$\text{IPC} = \frac{1}{\text{CPI}}$$

3. **The Iron Law (Execution Time T)**:

$$\text{Execution Time } (T) = \text{Instruction Count } (IC) \times \text{CPI} \times \frac{1}{\text{Frequency } (f)}$$

4. **Speedup (S)**:

$$\text{Speedup } (S) = \frac{\text{Execution Time of Baseline}}{\text{Execution Time of Optimized}}$$

Chapter J2: The CPI Stack (Cold-Start vs Steady-State)

1. In One Sentence
The CPI Stack decomposes total CPI into the ideal base execution plus stall contributions from RAW data hazards, branch mispredicts, and memory hierarchy misses.

---

2. The Great CPI Discrepancy Explained (Cold-Start vs Steady-State)
In older hand-typed documentation, a theoretical CPI table showed `2.560 CPI`. When running fresh cycle-accurate simulations, short benchmarks showed cold CPIs like `6.125`, while extended benchmarks showed steady-state CPIs like `1.125`.

Why the difference? 1. **Cold-Start Startup Overhead**: - On simulation boot, the very first instruction fetch misses compulsory in both L1 ITLB and L1I Cache. - The Radix-4 page walker must traverse all 4 table levels in DRAM, costing $117–126$ stall cycles. - If a short benchmark executes only 24 instructions (like `factorial_iter.s`), that single 120-cycle startup penalty gets divided by 24 instructions:

$$\Delta\text{CPI}_{\text{cold}} = \frac{120}{24} = +\mathbf{5.000} \;\; \mathbf{CPI}\;!$$

- This creates an artificially inflated cold CPI of

$1.250 + 5.000 = \mathbf{6.250}$. 2. **Steady-State Reality**: - In real-world software, programs execute millions of instructions. - On scaled benchmarks (`*_long.s`), that initial 120-cycle startup penalty gets divided across $> 100,000$ instructions:

$$\Delta\text{CPI}_{\text{startup}} = \frac{120}{100000} = +\mathbf{0.0012} \;\; \mathbf{CPI} \approx 0$$

- Once caches and TLBs are warm, the true steady-state CPI is

1.02 to 1.15!

---

3. Steady-State CPI Stack Breakdown (Real Runs on `matrix_mult_long.s`)

```
4-Stage Pipeline Steady-State CPI Stack (Total: 1.125):
+-----------------------------------+-----------+------------+
| Component                         | Delta CPI | Percentage |
+-----------------------------------+-----------+------------+
| Base Execution (Ideal Pipeline)   |   1.000   |    88.9%   |
| RAW Load-Use Bubble Stalls        |   0.070   |     6.2%   |
| Memory Hierarchy Latency Stalls   |   0.043   |     3.8%   |
| Branch Mispredict Flush Penalty   |   0.012   |     1.1%   |
+-----------------------------------+-----------+------------+
| Total CPI                         |   1.125   |   100.0%   |
+-----------------------------------+-----------+------------+
```

Chapter J3: Complete Benchmark Suite Results

Real simulation data captured from `./audit_cpi`:

Benchmark	Purpose / Workload	Instructions	4-Stage CPI (Cold)	4-Stage CPI (Steady)	6-Stage CPI (Steady)
`factorial_iter`	Iterative loop (tests branch prediction)	24	6.125	**1.250**	1.417
`factorial_rec`	Recursive stack frame calls	143	1.902	**1.084**	1.154
`prime_test`	Nested trial division loops	167	1.743	**1.042**	1.072
`array_sum`	Sequential array memory reads	58	3.207	**1.190**	1.259
`hazard_test`	Intensive back-to-back RAW dependencies	19	7.421	**1.263**	1.529
`stack_test`	Deep push/pop sequences	37	4.405	**1.243**	1.351
`array_sum_long`	Scaled array traversal (>100k inst)	100,004	1.191	**1.190**	1.259
`matrix_mult_long`	Triple-nested matrix multiply	189,050	1.126	**1.125**	1.131
`prime_test_long`	Prime sieve (>100k inst)	100,002	1.043	**1.042**	1.072

Chapter J4: Key Experimental Findings from `experiments/`

The project includes 30 automated experiment sweeps. Here are the top architectural takeaways:

1. **Forwarding Effectiveness (Exp 1)**:
   - Without forwarding, every RAW hazard stalls for 2-3 cycles (CPI = 2.45).
   - Full forwarding reduces RAW stalls by **87%**, dropping CPI to **1.12**.
2. **Branch Predictor Comparison (Exp 2)**:
   - Static Not-Taken: 62% accuracy.

- 2-Bit Bimodal: **93.4% accuracy**, cutting branch bubbles in half.

3. **Cache Line Size Trade-Off (Exp 1)**:
   - 16 B lines suffer from high compulsory miss rates.
   - 64 B lines achieve the lowest AMAT by maximizing spatial locality.
   - 256 B lines cause memory bus pollution and increase eviction latency.
4. **Page-Walk Caching (Exp 15)**:
   - Reduces average page table walk latency from 124 cycles down to **32 cycles (a 74% reduction)**.
5. **ASID Tagging During Context Switches (Exp 19)**:
   - Without ASID tagging, every process switch flushes the entire TLB, causing massive translation stall spikes.
   - 8-bit ASID tags preserve resident translations, cutting switch overhead by **84%**.

---

5. Summary: If You Only Remember 5 Things

1. Execution Time $T = (IC \times \text{CPI})/f$.
2. **Cold-Start CPI** is high on tiny micro-benchmarks because 120-cycle compulsory misses dominate small instruction counts.
3. **Steady-State CPI** on realistic workloads is **1.02 to 1.15**.
4. Full data forwarding cuts RAW data stalls by **87%**.
5. 6-Stage has slightly higher CPI (1.131 vs 1.125), but finishes **59% faster in wall-clock time** due to 800 MHz clocking. # Part K: Quality Assurance, Verification, & Project History

This section details the automated verification suite (78/78 tests, 231/231 golden checks, AddressSanitizer/UBSan), explains every Makefile target in plain words, recounts the engineering refactoring history from git logs, and provides an honest analysis of project limitations so you never get trapped during your presentation.

Chapter K1: Automated Verification & Testing Strategy

1. In One Sentence

The repository maintains six layers of automated quality assurance: unit tests, differential testing against the Golden ISS, golden baseline regression validation, AddressSanitizer memory safety checks, an ISA single-source linter, and documentation syntax linting.

---

2. The Six QA Layers

```
Layer 6: Documentation & LaTeX Syntax Linter (tools/lint_docs.cpp)
Layer 5: Single Source of Truth ISA Linter (tools/lint_isa.cpp)
Layer 4: AddressSanitizer & UndefinedBehaviorSanitizer (make test-asan)
Layer 3: Golden Output Regression Validator (tools/verify_golden.cpp - 231 checks)
Layer 2: Differential Testing (Pipelines vs Golden ISS - 192 configs)
Layer 1: Unit Test Suite (tests/test_all.cpp - 78 comprehensive tests)
```

1. **Unit Tests (`tests/test_all.cpp`)**:

- 78 tests covering ALU circuits, cache associativity, TLB replacement, hazard detection, and microcode dispatch.
- Run via: `make test` (Pass rate: **78 / 78 = 100%**).

2. **Differential Testing**:
   - Compares 4-stage and 6-stage pipeline register outputs against the single-cycle Golden ISS across hundreds of random and corner-case instruction sequences.
3. **Golden Regression Validator (`tools/verify_golden.cpp`)**:
   - 231 checks verifying assembler `.hex` output, disassembler round-tripping, microcode control stores, and cycle count baselines.
   - Run via: `make verify-golden` (Pass rate: **231 / 231 = 100%**).
4. **Memory Safety & Sanitizers**:
   - Compiles under `-fsanitize=address,undefined` to verify zero memory leaks, zero buffer overruns, and zero unaligned memory pointer dereferences.
   - Run via: `make test-asan` (**Zero leaks, zero UB**).
5. **ISA Single Source of Truth Linter (`tools/lint_isa.cpp`)**:
   - Scans 43 files ensuring zero integer opcodes, zero raw string literals, and zero duplicate enums.
   - Run via: `make lint-isa`.
6. **Documentation KaTeX Linter (`tools/lint_docs.cpp`)**:
   - Scans all markdown documentation ensuring mathematical syntax renders without formatting errors.
   - Run via: `make lint-docs`.

Chapter K2: The Makefile Target Reference

Every target explained in simple words:

Target	Command	What It Does in Plain Words
`make all`	Builds all 12 binaries	Compiles simulator, assembler, disassembler, test suite, and evaluation tools.
`make test`	`./risc201_test`	Runs the 78 automated unit and differential tests.
`make test-asan`	Clang/GCC Sanitizer	Compiles and runs test suite under AddressSanitizer and UndefinedBehaviorSanitizer.
`make verify-golden`	`./verify_golden`	Runs 231 regression checks against locked baseline binaries and traces.
`make audit-cpi`	`./audit_cpi`	Runs all 12 benchmarks across all pipeline and memory presets, printing real CPIs.
`make perf-docs`	`./generate_perf_docs`	Regenerates `docs/PERFORMANCE.md` exclusively from fresh simulation runs.
`make lint-isa`	`./lint_isa`	Scans all 43 C++ files verifying adherence to `common/ISA.hpp`.
`make lint-docs`	`./lint_docs`	Verifies clean LaTeX math formatting across all markdown documents.
`make demo-pipe`	Run `hazard_test.s`	Runs 6-stage pipeline with live ASCII timing diagrams and stall explanations.
`make demo-mem`	Run `factorial_iter.s`	Runs 4-stage pipeline with live cache set-way grid and DRAM bank displays.
`make demo-vm`	Run `factorial_rec.s`	Runs 6-stage pipeline with active Radix-4 Virtual Memory and TLB translations.
`make clean`	`rm -f ...`	Deletes all compiled binaries and temporary `.hex`/`.bin` files.

Chapter K3: Project History & Major Refactoring Milestones

From git logs (`git log --oneline`):

1. **Milestone 1 — Baseline Microcoded Architecture**:
   - Original C++ datapath, basic instruction decoding, and horizontal/vertical control memory stores implemented.
2. **Milestone 2 — Pipelining & Hazard Mitigation**:
   - Implementation of 4-stage and 6-stage pipeline cores, inter-stage latch registers, Hazard Unit, and Forwarding Unit.
3. **Milestone 3 — Memory Hierarchy & Virtual Memory**:

- Addition of multi-level caches (L1-L3), non-blocking MSHRs, DDR4/DDR5 DRAM controller, 4-level Radix-4 page tables, and TLBs.

4. **Milestone 4 — Teammate UI & Visualizer Enhancements (Varshith Teja, `bbd9ea1`)**:
   - Addition of terminal ASCII pipeline timing trace visualizer, startup assembly prompt, Newton-Raphson divider, and dynamic CLI branch predictor selection.
5. **Milestone 5 — The CPI Audit & Bug Fixes (`e491088`)**:
   - Diagnosis of the 2.560 CPI discrepancy; discovered cold-start vs steady-state phenomenon; added 6 extended benchmarks (`*_long.s`); fixed branch flush penalty logic.
6. **Milestone 6 — ISA Single Source of Truth Refactor (`fa33baa`, `4dceaf7`)**:
   - Complete elimination of duplicate integer opcode enum from `Types.hpp`; all 43 files unified strictly around canonical 5-bit binary strings in `common/ISA.hpp`.

Chapter K4: Honest Limitations & What NOT to Claim

In a university project defense, examiners respect honesty far more than exaggerated claims. Here is what you should and should NOT claim:

What You CAN Confidently Claim:

- 100% pure modern C++17 simulator (12,929 lines, zero external dependencies).
- Cycle-accurate in-order execution matching textbook SimpleRisc ISA.
- Complete memory hierarchy (L1-L3 non-blocking caches, DDR4/DDR5 DRAM, 48-bit Radix-4 VM).
- Zero integer opcode enum duplicates (single source of truth).
- 78/78 tests and 231/231 golden checks pass with 100% verification.

What You Should NOT Claim (Known Limitations):

1. **Out-of-Order Execution**: The core is strictly **In-Order** (Scoreboarding / Tomasulo / Reorder Buffer are NOT implemented).
2. **Branch Mispredict Delay Slots**: Branch delay slots are not used; branches are handled via hardware squashing and dynamic 2-bit bimodal/gshare prediction.
3. **Multicore / Cache Coherence**: The processor is single-core; multiprocessor snooping protocols (MESI/MOESI) are NOT implemented.
4. **Physical Synthesis**: The project is a cycle-accurate software simulator in C++, not synthesized Verilog on an FPGA. Clock frequencies (500 MHz, 800 MHz) are based on standard 28nm standard-cell gate delay modeling, not physical place-and-route.

---

5. Summary: If You Only Remember 5 Things

1. **78/78 tests** and **231/231 golden checks** pass with 100% success.
2. Verified with **AddressSanitizer and UndefinedBehaviorSanitizer** (zero memory leaks).
3. The project underwent two major refactors: the **CPI Audit** and the **String ISA Refactor**.
4. Use **`make demo-pipe`**, **`make demo-mem`**, and **`make demo-vm`** for live presentation demos.
5. Honestly acknowledge that the CPU is **in-order single-core**; examiners love mature technical honesty! # 10 Complete Hand-Traceable Whiteboard Examples

Practice these 10 examples step-by-step so you can walk up to a whiteboard during your presentation or viva and confidently solve any problem with zero hesitation.

Example 1: Assembling a Branching Program by Hand

Problem: Assemble the following 4-line program into 32-bit hexadecimal machine words:

```
0x00:   mov  r1, 0
0x04: loop:
0x04:   add  r1, r1, 1
0x08:   cmp  r1, 5
0x0C:   bgt  loop
```

Step-by-Step Solution: 1. **Line 1 (`0x00: mov r1, 0`)**: - Mnemonic `mov` → Opcode `isa::OP_MOV = "01001"` (9_{10}) - Immediate mode → $I = 1$ - Destination $rd = 1$ (`0001`) - Immediate value $0 \rightarrow 0$ - Binary pack: `01001 1 0001 0000 00000000000000000` - **Hex Result: `0x4C400000`**

2. **Line 2 (`0x04: add r1, r1, 1`)**:
   - Mnemonic `add` → Opcode `isa::OP_ADD = "00000"` (0_{10})
   - Immediate mode → $I = 1$
   - Destination $rd = 1$ (`0001`), Source $rs1 = 1$ (`0001`), Immediate $= 1$
   - Binary pack: `00000 1 0001 0001 00000000000000001`
   - **Hex Result: `0x04440001`**
3. **Line 3 (`0x08: cmp r1, 5`)**:
   - Mnemonic `cmp` → Opcode `isa::OP_CMP = "00101"` (5_{10})
   - Immediate mode → $I = 1$
   - Source $rs1 = 1$ (`0001`), Immediate $= 5$
   - Binary pack: `00101 1 0000 0001 00000000000000101`
   - **Hex Result: `0x2C040005`**
4. **Line 4 (`0x0C: bgt loop`)**:
   - Mnemonic `bgt` → Opcode `isa::OP_BGT = "10001"` ($17_{10} = 0x11$)
   - Current PC = `0x0C`, Target `loop` = `0x04`
   - Relative byte distance = `0x04 - 0x0C = -8 bytes`
   - Word offset = `-8 / 4 = -2`
   - In 27-bit two's complement: -2 is `0x07FFFFE`
   - Binary pack: `10001 000011111111111111111111110`
   - **Hex Result: `0x8FFFFFE`**

Example 2: Disassembling a Machine Word by Hand

Problem: Disassemble the 32-bit machine word `0x00888000`.

Step-by-Step Solution: 1. Convert hex to binary: - `0x0` = `0000` - `0x0` = `0000` - `0x8` = `1000` - `0x8` = `1000` - `0x8` = `1000` - Full 32 bits: `0000 0000 1000 1000 1000 0000 0000 0000` 2. Extract fields: - Bits `[31:27]` = `00000` → `isa::OP_ADD` (`add`) - Bit `[26]` = `0` → Register format ($I = 0$) - Bits `[25:22]` = `0010` → Destination $rd = 2$ (`r2`) - Bits `[21:18]` = `0010` → Source $rs1 = 2$ (`r2`) - Bits `[17:14]` =

`0010` → Source $rs2 = 2$ (`r2`) 3. **Reconstructed Assembly**: **`add r2, r2, r2`** (doubles the value of `r2`).

Example 3: RAW Data Hazard With and Without Forwarding

Program:

```
add r1, r2, r3   ; Inst 1: Writes to r1
sub r4, r1, r5   ; Inst 2: Reads r1 (RAW!)
```

Case A: Without Forwarding (Requires Stalls)

```
Cycle:          1     2     3     4     5     6     7
Inst 1 (add):  [IF]  [OF]  [EX]  [MA]  [RW*]
                                        ^ r1 written at END of Cycle 5
Inst 2 (sub):        [IF]  [OF]  [STALL][STALL][OF*] [EX]
                                             ^ r1 read at START of Cycle 6
```

- **Stall Penalty**: **2 wasted cycles**! CPI drops to 2.0.

Case B: With Forwarding (**`EX -> EX`** Bypass)

```
Cycle:          1     2     3     4     5
Inst 1 (add):  [IF]  [OF]  [EX]  [MA]  [RW]
                           |
                           +-----> [EX -> EX Bypass]
                                    |
Inst 2 (sub):        [IF]  [OF]  [EX]  [MA]  [RW]
```

- At the end of Cycle 3, `add` produces its result in `EX`.
- At the start of Cycle 4, the **Forwarding Unit** routes that result directly into the ALU input multiplexer for `sub`.
- **Stall Penalty: 0 cycles!** Smooth 1-cycle execution.

Example 4: Load-Use Hazard Hand Trace

Program:

```
ld  r1, [r2]      ; Inst 1: Loads r1 from memory
add r3, r1, r4    ; Inst 2: Uses r1 immediately!
```

```
Cycle:          1     2     3     4        5     6
Inst 1 (ld):   [IF]  [OF]  [EX]  [MA*]    [RW]
                                   | Data arrives at END of Cycle 4
                                   v
Inst 2 (add):        [IF]  [OF]  [BUBBLE] [EX*] [MA] [RW]
```

- In Cycle 3, `add` enters `OF`. The Hazard Unit checks:

$$\text{IF_OF.rs1} == \text{OF_EX.rd} \quad (r1 == r1) \quad \wedge \quad \text{OF_EX.is_load} == \text{true}$$

- The Hazard Unit freezes `PC` and `IF_OF` latch, and inserts a `nop` **bubble** into `OF_EX`.
- In Cycle 4, `ld` completes memory read in `MA`.

- In Cycle 5, `ld` forwards data via `MEM -> EX` bypass into `add`'s ALU input.
- **Result: Exactly 1 unavoidable stall cycle.**

Example 5: Cache Address Slicing Hand Calculation

Problem: A processor has a **32 KB, 8-way set-associative L1 Data Cache** with **64-byte cache lines**. The CPU accesses 32-bit physical address `0x0001_8F54`.
Find the Byte Offset, Set Index, and Tag.

Step-by-Step Solution: 1. **Find Bit Widths**: - Block Size $B = 64$ bytes $\rightarrow 2^b = 64 \implies b = \mathbf{6}$ **offset bits**.
- Number of cache lines =

Total Size / Block Size $= 32768/64 = 512$ lines.
- Number of Sets

$S = 512/8 = 64$ sets $\rightarrow 2^s = 64 \implies s = \mathbf{6}$ **index bits**.
- Tag bits

$t = 32 - (s + b) = 32 - (6 + 6) = \mathbf{20}$ **tag bits**.

2. **Decompose Address `0x0001_8F54`**:
   - In binary: `0000 0000 0000 0001 1000 1111 0101 0100`
   - Split:

$$\underbrace{0000\,0000\,0000\,0001\,1000}_{\text{Tag (20 bits)}} \quad \underbrace{1111\,01}_{\text{Index (6 bits)}} \quad \underbrace{01\,0100}_{\text{Offset (6 bits)}}$$

3. **Convert to Hexadecimal**:
   - **Byte Offset**: `010100` binary = **`0x14`** **(**20_{10}**)** (Byte 20 inside line).
   - **Set Index**: `111101` binary = **`0x3D`** **(**61_{10}**)** (Look in Set 61).
   - **Tag**: `0x00018` = **`0x00018`** (Tag stored in directory).

Example 6: AMAT Calculation Hand Problem

Problem: A memory hierarchy has: - L1 Hit Time = 1 cycle, Miss Rate = 5% - L2 Hit Time = 10 cycles, Miss Rate = 20% - DRAM Latency = 100 cycles
Calculate the Average Memory Access Time (AMAT).

Solution: 1. Compute L2 Effective Miss Penalty:

$$\text{Penalty}_{L1} = \text{Hit}_{L2} + (\text{MissRate}_{L2} \times \text{Latency}_{\text{DRAM}})$$

$$\text{Penalty}_{L1} = 10 + (0.20 \times 100) = 10 + 20 = 30 \text{ cycles}$$

2. Compute AMAT:

$$\text{AMAT} = \text{Hit}_{L1} + (\text{MissRate}_{L1} \times \text{Penalty}_{L1})$$

$$\text{AMAT} = 1 + (0.05 \times 30) = 1 + 1.5 = \mathbf{2.5} \text{ cycles}$$

Example 7: 48-Bit Radix-4 Virtual Address Translation

Problem: Given Virtual Address `0x0000_1234_5678_9ABC`, calculate the PML4, PDPT, PD, PT indices and Page Offset.

Solution: 1. Slicing bits: - **Offset**: `VA[11:0] = 0xABC` (2748_{10}) - **PT Index**: `VA[20:12] = (0x123456789ABC >> 12) & 0x1FF = 0x089` (137_{10}) - **PD Index**: `VA[29:21] = (0x123456789ABC >> 21) & 0x1FF = 0x1B3` (435_{10}) - **PDPT Index**: `VA[38:30] = (0x123456789ABC >> 30) & 0x1FF = 0x0D1` (209_{10}) - **PML4 Index**: `VA[47:39] = (0x123456789ABC >> 39) & 0x1FF = 0x024` (36_{10})
2. The MMU traverses: PML4 $[36] \rightarrow$ PDPT $[209] \rightarrow$ PD $[435] \rightarrow$ PT $[137] \rightarrow$ PFN.

Example 8: 2-Bit Bimodal Branch Predictor State Machine

Problem: A loop executes 4 times and exits. The predictor initializes at Weakly Taken (`10`). Trace the state transitions.

```
State Encoding:
  00: Strongly Not Taken
  01: Weakly Not Taken
  10: Weakly Taken
  11: Strongly Taken
```

Iteration	Initial State	Prediction	Actual Outcome	Mispredict?	Next State
1	`10` (Weakly Taken)	Taken	Taken	No (Hit)	`11` (Strongly Taken)
2	`11` (Strongly Taken)	Taken	Taken	No (Hit)	`11` (Strongly Taken)
3	`11` (Strongly Taken)	Taken	Taken	No (Hit)	`11` (Strongly Taken)
4 (Exit)	`11` (Strongly Taken)	Taken	**Not Taken**	**Yes (Mispredict)**	`10` (Weakly Taken)

- **Total Accuracy**: $3/4 = \mathbf{75}\%$.
- Notice: On loop exit, state only dropped to `10` (Weakly Taken), so when the loop restarts next time, it will **still predict Taken**!

Example 9: 4-Stage vs 6-Stage Execution Time & Speedup

Problem: A program executes $IC = 1000000$ instructions: - 4-Stage Core: $f = 500$ MHz, CPI $= 1.12$ - 6-Stage Core: $f = 800$ MHz, CPI $= 1.14$
Calculate execution time for both cores and the speedup.

Solution: 1. 4-Stage Time:

$$T_4 = \frac{1000000 \times 1.12}{500 \times 10^6} = \frac{1120000}{500000000} = \mathbf{2.24 \ \ ms}$$

2. 6-Stage Time:

$$T_6 = \frac{1000000 \times 1.14}{800 \times 10^6} = \frac{1140000}{800000000} = \mathbf{1.425 \ \ ms}$$

3. Speedup:

$$\text{Speedup } S = \frac{T_4}{T_6} = \frac{2.24}{1.425} = \mathbf{1.57\times} \ \ (\mathbf{57\% \ Faster!})$$

Example 10: Full-Descending Stack Push/Pop Sequence

Problem: Given `sp = 0x000FFFFC` and registers $r1 = 42, r2 = 99$: Trace `push r1`, `push r2`, `pop r3`.

Step-by-Step Solution: 1. `push r1`: - Decrement `sp`: `sp = 0x000FFFFC - 4 = 0x000FFFF8` - Store: `Memory[0x000FFFF8] = 42` 2. `push r2`: - Decrement `sp`: `sp = 0x000FFFF8 - 4 = 0x000FFFF4` - Store: `Memory[0x000FFFF4] = 99` 3. `pop r3`: - Read from current top: `r3 = Memory[0x000FFFF4] = 99` - Increment `sp`: `sp = 0x000FFFF4 + 4 = 0x000FFFF8` - **Final State**: $r3 = 99$, `sp = 0x000FFFF8`. Valid data 42 remains at `0x000FFFF8`. # Guided Source Code Tour & Reading Walkthroughs

If you open the codebase in VS Code, follow this exact reading order. Each entry tells you what the file does, its core classes and functions, and the exact lines to look at first.

---

1. `common/ISA.hpp` (130 lines)

- **What it is for**: The single source of truth for all instruction definitions in the simulator.
- **Key Structures & Functions**:
  - `namespace isa`: Defines 5-bit string constants `OP_ADD = "00000"` through `OP_INVALID = "11111"`.
  - `struct InstructionMeta`: Holds mnemonic, format, and boolean flags (`is_load`, `writes_rd`, etc.).
  - `getMeta(opcode)`: Returns metadata struct for any 5-bit string.
- **What to look at first**: Lines 20–55 (the opcode string constants) and lines 65–85 (`getMeta()` table).

---

2. `common/Types.hpp` (110 lines)

- **What it is for**: Fundamental architectural types, bit widths, and non-opcode enumerations.
- **Key Types**:
  - `Word = uint32_t` (32-bit registers and addresses).
  - `struct DecodedInst`: The decoded instruction packet carried through pipeline stages.
  - `enum class BranchPolicy`, `enum class ForwardingMode`, `enum class AdderType`.
- **What to look at first**: Lines 25–45 (`struct DecodedInst`). Notice there is NO `enum Opcode` here!

---

3. `common/RegisterFile.hpp` (95 lines)

- **What it is for**: Models the 16 general-purpose registers and condition flags.
- **Key Classes**:
  - `class RegisterFile`: Holds `std::array<Word, 16> regs`, flags `flagE` and `flagGT`, and privileged registers `cpl`, `oldPC`, `oldSP`.
- **What to look at first**: Lines 20–35 (`read()` and `write()` methods).

---

4. `common/ISS.hpp` (210 lines)

- **What it is for**: The unpipelined, single-cycle Golden Reference Instruction Set Simulator.
- **Key Functions**:
  - `step()`: Executes one instruction completely (Fetch → Decode → Execute → Commit).
  - `decode()`: Slices 32-bit word into fields.
- **What to look at first**: Lines 45–65 (`step()`). It shows the simplest possible implementation of the computer architecture loop.

---

5. `alu/ALU.hpp` (165 lines)

- **What it is for**: The central arithmetic and logic execution engine.
- **Key Methods**:
  - `execute(opcodeBits, op1, op2, flags)`: Dispatches the operation using `if-else` on `isa::OP_*` string constants.
- **What to look at first**: Lines 30–80 (the `if-else` chain comparing `opcodeBits == isa::OP_ADD`). Notice how clean it is.

---

6. `common/ForwardingUnit.hpp` (150 lines)

- **What it is for**: Detects RAW data dependencies and routes bypass operands to the ALU.
- **Key Methods**:
  - `getForwardA()`, `getForwardB()`: Checks `EX/MA` then `MA/RW` to select operand source.
- **What to look at first**: Lines 40–75 (the bypass priority checks).

---

7. `common/HazardUnit.hpp` (145 lines)

- **What it is for**: Detects hazards that cannot be forwarded (load-use and branch mispredicts).
- **Key Methods**:
  - `detectLoadUse()`: Returns `true` when an instruction immediately uses loaded data.
  - `shouldFlushBranch()`: Signals pipeline squashing when branch predictor was wrong.
- **What to look at first**: Lines 35–60 (the load-use check comparing `OF_EX.rd` with `IF_OF.rs1/rs2`).

---

8. `common/BranchPredictor.hpp` (160 lines)

- **What it is for**: Dynamic branch prediction algorithms.

- **Key Classes**:
  - `TwoBitBimodalPredictor`: Implements 2-bit saturating counters (`00` to `11`).
  - `GSharePredictor`: Global history XOR hash predictor.
  - `ReturnAddressStack`: 8-entry LIFO stack for function returns.
- **What to look at first**: Lines 50–75 (the 2-bit bimodal state transition logic).

---

9. `processor_4stage/Pipeline4Stage.hpp` (480 lines)
- **What it is for**: The 4-stage pipelined CPU core (`IF -> OF -> EX -> MA_RW`).
- **Key Methods**:
  - `stepCycle()`: Calls `stageMA_RW()`, `stageEX()`, `stageOF()`, `stageIF()` in reverse order.
- **What to look at first**: Lines 120–160 (`stepCycle()`), and lines 180–230 (`stageEX()`).

---

10. `processor_6stage/Pipeline6Stage.hpp` (520 lines)
- **What it is for**: The 6-stage pipelined CPU core (`IF -> ID -> RF -> EX -> MA -> WB`).
- **Key Methods**:
  - `stageID()`: Pure instruction decode.
  - `stageRF()`: Pure Register File read.
- **What to look at first**: Compare `stageID()` and `stageRF()` with `stageOF()` in 4-stage to see the split.

---

11. `microcode/MicroController.hpp` (280 lines)
- **What it is for**: Microprogrammed control unit executing horizontal and vertical control stores.
- **Key Methods**:
  - `stepMicroCycle()`: Advances `microPC` by 1 micro-instruction.
- **What to look at first**: Lines 60–90 (the dispatch table mapping opcodes to starting microPC addresses).

---

12. `memory/VirtualMemory.hpp` (409 lines)
- **What it is for**: 4-level Radix-4 Virtual Memory, multi-tier TLBs, and Page-Walk Cache.
- **Key Classes**:
  - `GenericTLB`: Set-associative TLB with LRU replacement.
  - `VirtualMemorySubsystem`: Implements 48-bit address translation, PWC lookup, and page fault handling.
- **What to look at first**: Lines 300–360 (`translate()` method).

---

13. `assembler/Assembler.hpp` (320 lines)
- **What it is for**: The two-pass assembler translating `.s` assembly files to `.bin` and `.hex`.
- **Key Methods**:
  - `pass1()`: Builds the symbol table.
  - `pass2()`: Emits machine code words and resolves relative offsets.

- **What to look at first**: Lines 60–100 (`pass1()`) and lines 120–180 (`pass2()`). # Master Presentation Guide & Live Demo Scripts

This document provides everything needed for a flawless project presentation: slide outlines, spoken scripts (2-min, 5-min, 15-min), a step-by-step live demo script with exact console commands and outputs, and teammate role splitting.

---

1. Slide-by-Slide Outline (22 Slides)

Slide	Title	Key Talking Points	Visual to Display
1	**Title Slide**	RISC201-PROCESSOR: Cycle-Accurate Pipeline Simulation, Microcode Control, and Empirical Memory Hierarchy Evaluation.	Project logo, course name, team members (`bandlapalliyss-max` and `Varshith Teja`).
2	**Project Overview**	100% modern C++17 simulator (12,929 LOC) with zero external dependencies, modeling textbook SimpleRisc ISA.	High-level system architecture flowchart (`00_project_map.md`).
3	**The Canonical ISA**	25 instructions; single source of truth in `common/ISA.hpp`; 5-bit string opcodes; zero integer opcode enums.	Table of 25 opcodes + 4 regular formats (R, I, M, B).
4	**Software Toolchain**	Two-pass assembler (`risc201_as`) and disassembler (`risc201_disasm`) with 100% round-trip fidelity.	Diagram of Pass 1 (symbols) → Pass 2 (machine code).
5	**The Processors**	4-Stage (`IF->OF->EX->MA_RW`) vs 6-Stage (`IF->ID->RF->EX->MA->WB`).	Dual pipeline datapath ASCII diagrams.
6	**Why 6-Stage Wins**	Splitting `OF` into `ID` and `RF` cuts logic delay, raising clock from 500 MHz to 800 MHz (**1.59x speedup**!).	Frequency scaling graph & wall-clock execution time formula.
7	**The Golden ISS**	Single-cycle reference model in `common/ISS.hpp` verifying 100% behavioral equivalence across pipelines.	Differential testing block diagram (P4 vs P6 vs ISS).
8	**Hazard Mitigation**	RAW data hazards, load-use delays, and control hazards.	RAW dependency timeline diagram.
9	**Data Forwarding**	Full forwarding unit (`EX->EX`, `MEM->EX`) eliminating RAW stalls in 0 cycles.	Forwarding bypass multiplexer diagram.
10	**Branch Prediction & Microcode**	Static 1-bit & 2-bit saturating counters; Branch Target Buffer (BTB); microcode state machine (`00_microcode.md`).	2-bit saturating counter state machine.

2. Spoken Scripts

A. The 2-Minute Elevator Pitch

"Good morning, professors. Our project is **RISC201-PROCESSOR**, a cycle-accurate modern computer systems simulator built entirely in 100% modern C++17.

We modeled the 32-bit SimpleRisc architecture from Smruti Sarangi's textbook, implementing both a **4-stage pipeline** and a high-frequency **6-stage pipeline**. We solved RAW data hazards using a comprehensive forwarding unit that routes results with zero stall cycles, and we implemented 2-bit bimodal and gshare branch predictors to eliminate control stalls.

Beyond the processor core, we implemented a full **modern memory hierarchy**: split L1 Harvard caches to prevent structural hazards, non-blocking MSHRs, a cycle-accurate DDR4/DDR5 DRAM controller with FR-FCFS scheduling, and a 4-level Radix-4 Virtual Memory subsystem with TLBs and a Page-Walk Cache that slashes walk overhead by 74%.

We also verified every claim empirically: all 78 unit tests and 231 golden regression checks pass with 100% success, verified completely leak-free under AddressSanitizer. We are ready to demonstrate the live simulator!"

B. The 5-Minute Standard Presentation

"Good morning. Today we present our processor design project: **RISC201-PROCESSOR**.

Architecture & ISA: We established a strict architectural foundation by making `common/ISA.hpp` the single source of truth for all 25 instruction opcodes. Every opcode is represented as a canonical 5-bit binary string, completely eliminating duplicate integer enums and matching physical hardware bit lines.

The Two Pipeline Implementations: We designed and compared two distinct pipeline cores: 1. A balanced **4-stage pipeline** (`IF -> OF -> EX -> MA_RW`) running at 500 MHz. 2. An optimized **6-stage pipeline** (`IF -> ID -> RF -> EX -> MA -> WB`) running at 800 MHz.

By splitting the operand fetch into separate Instruction Decode and Register Read cycles, we halved the critical path logic delay. Although the deeper pipeline has a slightly higher branch penalty, its higher clock frequency delivers a **1.59x wall-clock speedup** on real workloads.

Hazard Mitigation & Control: ALU-to-ALU dependencies are bypassed in zero cycles via our `EX->EX` and `MEM->EX` forwarding paths. Only load-use dependencies require a single bubble stall. For branches, our 2-bit saturating counter achieves 93.4% accuracy. We also implemented both Horizontal (32-bit) and Vertical (16-bit) microcode control stores, showing that vertical microcode reduces control ROM size by 50% with minimal decoder overhead.

Memory & Safety: Our memory subsystem features a Split L1 Harvard cache to allow simultaneous instruction fetch and data memory access without structural stalls. Backing it is a cycle-accurate DDR4 DRAM controller and a 48-bit Radix-4 Virtual Memory subsystem. We enforce stack safety via a memory-mapped Guard Page beneath our full-descending stack, trapping recursive stack overflow before memory corruption can occur.

All 78 automated tests and 231 golden regression checks pass with 100% fidelity. We would now like to run the live demonstration."

---

3. The Live Demo Script (Step-by-Step)

Follow these exact terminal commands during your live demonstration:

Step 1: Demonstrate Assembly & Disassembly Round-Trip

What to say: *"First, we will assemble an assembly source file into 32-bit machine code, and disassemble it back to text to verify 100% round-trip fidelity."*

```
./risc201_as benchmarks/hazard_test.s -o demo.bin --hex demo.hex
./risc201_disasm demo.bin
```

Expected Output: The terminal prints the disassembled assembly matching `benchmarks/hazard_test.s` line for line.

Step 2: Demonstrate 6-Stage Pipeline & Data Forwarding

What to say: *"Next, we run the 6-stage pipeline on `hazard_test.s`. You will see our visual ASCII pipeline execution state and our forwarding unit bypassing data across stages."*

```
make demo-pipe
```

Expected Output:

```
+--------------------------------------------------------------------------+
|               PIPELINE EXECUTION STATE (Clock Cycle:     25)             |
+--------------------------------------------------------------------------+
  [IF    ] : -- BUBBLE / NOP --
  [ID    ] : -- BUBBLE / NOP --
  [RF    ] : -- BUBBLE / NOP --
  [EX    ] : -- BUBBLE / NOP --
  [MA    ] : -- BUBBLE / NOP --
  [RW    ] : nop (PC: 0x00000048)
```

```
+--------------------------------------------------------------------------+
==========================================================================
                  6-STAGE PROCESSOR REPORT
==========================================================================
Total Cycles:               25
Retired Instructions:       17
CPI:                        1.471
Branch Bubbles / Flushes:   2
--------------------------------------------------------------------------
CPI STACK BREAKDOWN BY CANONICAL STALL REASON:
RAW_LOAD_USE            1           0.059       33.3%
BRANCH_MISPREDICT       2           0.118       66.7%
==========================================================================
```

Step 3: Demonstrate Cache Hierarchy & DRAM Controller

What to say: *"Now we run our 4-stage core with the `modern` memory preset, displaying live L1 cache set-way occupancy and DDR4 DRAM bank buffer states."*

```
make demo-mem
```

Expected Output: Displays 8-way L1D cache hit rate (100.00%), 3C miss counts, and the DDR4 bank state table showing open row buffers.

Step 4: Demonstrate Radix-4 Virtual Memory & TLBs

What to say: *"Finally, we run a recursive factorial program on the 6-stage pipeline with 48-bit Radix-4 Virtual Memory and TLBs active."*

```
make demo-vm
```

Expected Output: Shows 164 execution cycles, recursive function calls, L1D hit rate of 94.12%, and Effective Access Time (EAT) calculation.

Backup Plan (If Anything Goes Wrong During Live Demo)

- **Problem**: Terminal display garbled or window resized too small.
  - **Action**: Type `reset` and press Enter, then rerun the command.
- **Problem**: Binary missing or permission error.
  - **Action**: Run `make clean && make all` (takes only 4 seconds).
- **Problem**: Examiner asks to see source code proof of single source of truth.
  - **Action**: Run `./lint_isa` directly in the terminal to show all 43 files passing inspection.

---

4. Teammate Role Division Suggestion

Based on commit history: - **`bandlapalliyss-max` (Srinivas B.)**:

- Present: Architecture overview, 4-stage vs 6-stage datapath,

Forwarding and Hazard Units, Memory Hierarchy (Caches, DRAM, Radix-4 VM), and CPI performance analysis. - **`Varshith Teja`**: - Present: Software toolchain, UI/Visualizers (ASCII pipeline trace, startup prompt), Newton-Raphson division algorithms, CLI branch predictor selection, and execute the live terminal demo. # 120 Comprehensive Examiner Questions & Model Answers

Grouped by difficulty into **Easy**, **Medium**, **Hard**, and **Examiner Trap** questions. Every answer includes concrete source file line citations.

Category 1: Easy Questions (Fundamentals & Architecture)

Q1: What does RISC mean, and how does RISC201 follow RISC principles?
Answer: RISC stands for Reduced Instruction Set Computer. RISC201 follows RISC by using fixed 32-bit instruction words, a regular 4-format layout, and a strict Load-Store architecture where only `ld` and `st` touch memory (`common/ISA.hpp:1-50`).
Reference: Chapter A2

Q2: What are the four stages of the 4-stage pipeline?
Answer: The four stages are Instruction Fetch (`IF`), Operand Fetch & Decode (`OF`), Execute (`EX`), and Memory Access & Register Writeback (`MA_RW`) (`processor_4stage/Pipeline4Stage.hpp:12-25`).
Reference: Chapter D2

Q3: What are the six stages of the 6-stage pipeline?
Answer: The six stages are Instruction Fetch (`IF`), Instruction Decode (`ID`), Register Read (`RF`), Execute (`EX`), Memory Access (`MA`), and Writeback (`WB`) (`processor_6stage/Pipeline6Stage.hpp:15-30`).
Reference: Chapter D3

Q4: How many general-purpose registers does RISC201 have?
Answer: RISC201 has 16 general-purpose registers (`r0` through `r15`), where `r14` is conventionally the Stack Pointer (`sp`) and `r15` is the Return Address (`ra`) (`common/RegisterFile.hpp:18-25`).
Reference: Chapter B1

Q5: What is ideal CPI for an in-order scalar pipeline?
Answer: The ideal CPI is 1.000, which corresponds to retiring exactly one instruction per clock cycle when there are no hazard stalls, memory misses, or branch flushes.
Reference: Chapter D1

Q6: How are condition flags updated in RISC201?
Answer: Flags `E` (Equal) and `GT` (Greater Than) are updated strictly by the `cmp` instruction. Other arithmetic instructions like `add` or `sub` do NOT modify flags (`alu/ALU.hpp:45-55`).
Reference: Chapter B1

Q7: What is the opcode width in RISC201?
Answer: Exactly 5 bits (`bits[31:27]`), allowing up to 32 distinct opcode patterns, of which 25 are assigned in this architecture (`common/ISA.hpp:20-40`).
Reference: Chapter B1

Q8: What does the Immediate Flag (bit 26) do?
Answer: If bit 26 is `0`, the second operand comes from register `rs2`. If bit 26 is `1`, the second operand is an 18-bit signed immediate constant (`assembler/Disassembler.hpp:38-44`).
Reference: Chapter A3

Q9: What is the purpose of the Golden ISS?
Answer: The Golden ISS (`common/ISS.hpp`) is an unpipelined, single-cycle reference model used to verify that the complex pipelined cores execute instructions with 100% behavioral correctness.
Reference: Chapter D5

Q10: What tool assembles `.s` files in this project?
Answer: `risc201_as`, built from `assembler/main_as.cpp` and `assembler/Assembler.hpp`.
Reference: Chapter C1

Q11: What is a pipeline bubble?
Answer: A bubble is a dummy `nop` instruction injected into an inter-stage latch that advances through the pipeline without modifying architectural state, stalling subsequent instructions (`common/HazardUnit.hpp:80-95`).
Reference: Chapter E2

Q12: What is the full-descending stack convention?
Answer: "Full" means `sp` points to the last valid written data item. "Descending" means the stack grows toward lower memory addresses (decrements on push, increments on pop) (`assembler/MacroPreprocessor.hpp:25-45`).
Reference: Chapter C3

Q13: What does the disassembler do?
Answer: The disassembler (`risc201_disasm`) reads 32-bit binary words, extracts bit fields, and outputs human-readable assembly text (`assembler/Disassembler.hpp:55-80`).
Reference: Chapter C2

Q14: What is the default starting Stack Pointer address?
Answer: `0x000FFFFC`, located at the top of the 1 MB default physical address space (`common/RegisterFile.hpp:22`).
Reference: Chapter I1

Q15: What is AMAT?
Answer: Average Memory Access Time: the average number of clock cycles required to perform a memory access across all levels of the cache and DRAM hierarchy (`notes/parts/`

part_h_memory.md).
Reference: Chapter H3

Category 2: Medium Questions (Pipelining, Hazards, Memory)

Q16: Why are two passes required in the assembler?
Answer: Pass 1 scans the source code to find all label addresses and store them in the Symbol Table. Pass 2 generates machine code words, using the Symbol Table to resolve forward branch references (assembler/Assembler.hpp:60-120).
Reference: Chapter C1

Q17: How does data forwarding eliminate RAW hazards?
Answer: The Forwarding Unit compares destination registers in EX/MA and MA/RW latches with source registers in EX. If there is a match, it routes the newly computed value directly to the ALU inputs, bypassing the Register File in 0 cycles (common/ForwardingUnit.hpp:45-80).
Reference: Chapter E2

Q18: Why does a Load-Use hazard still require 1 bubble even with forwarding?
Answer: Memory data from a ld instruction is only returned at the end of the MA stage. A dependent instruction entering EX needs the value at the start of EX. Because hardware cannot forward backwards in time, a 1-cycle bubble must be inserted (common/HazardUnit.hpp:35-55).
Reference: Chapter E2

Q19: What is the branch penalty in 4-stage vs 6-stage?
Answer: In the 4-stage core, branches resolve at EX (stage 3), creating a **1-cycle flush penalty**. In the 6-stage core, branches resolve at EX (stage 4), creating a **2-cycle flush penalty** (processor_6stage/Pipeline6Stage.hpp:180-210).
Reference: Chapter D4

Q20: Why is L1 split into ITLB/DTLB and I-Cache/D-Cache?
Answer: To eliminate the single-port structural hazard. In every cycle, IF fetches an instruction while MA may read/write data. Splitting L1 allows both stages to access memory simultaneously with zero stalls (memory/MemoryHierarchy.hpp:35-60).
Reference: Chapter H2

Q21: What is a 2-bit bimodal branch predictor?
Answer: It uses a 2-bit saturating counter (00 to 11) for each branch address. It requires two consecutive mispredicts before changing direction, preventing single loop exit anomalies from flipping the prediction (common/BranchPredictor.hpp:55-80).
Reference: Chapter E3

Q22: How does the basic-block instruction scheduler eliminate bubbles?
Answer: Enabled via --schedule, the assembler reorders independent instructions to fill the slot immediately following a ld, separating the load from its user and eliminating the load-use stall

(`assembler/Assembler.hpp:210-250`).
Reference: Chapter C1

Q23: What is the difference between Horizontal and Vertical microcode?
Answer: Horizontal microcode uses wide, unencoded 32-bit control words with zero decode delay. Vertical microcode encodes control lines into compact 16-bit fields, cutting control store ROM in half but requiring hardware decoders (`microcode/HorizontalControl.hpp` and `VerticalControl.hpp`).
Reference: Chapter F3

Q24: What is an MSHR in a cache?
Answer: Miss Status Holding Register: a hardware queue that tracks in-flight cache misses, allowing subsequent memory hits to proceed without stalling the processor (`memory/Cache.hpp:45-75`).
Reference: Chapter H2

Q25: How does the Stack Guard Page prevent memory corruption?
Answer: An inaccessible page (`Present=1, Read=0, Write=0`) is mapped beneath the stack space. Any stack overflow attempts to write into this page, immediately triggering an unmaskable exception trap (`memory/VirtualMemory.hpp:180-210`).
Reference: Chapter I2

Category 3: Hard Questions (Deep Microarchitecture)

Q26: Explain the mathematical proof of why 6-stage is faster despite higher CPI.
Answer: Using the Iron Law $T = (IC \times \text{CPI})/f$, on `matrix_mult_long`: 4-stage takes $(189,050 \times 1.125)/500 \text{ MHz} = 425.4\ \mu\text{s}$. 6-stage takes $(189,050 \times 1.131)/800 \text{ MHz} = 267.3\ \mu\text{s}$. The 60% frequency increase overwhelms the 0.5% CPI increase, yielding a **1.59x wall-clock speedup** (`notes/parts/part_d_processors.md`).
Reference: Chapter D4

Q27: How does the Page-Walk Cache (PWC) reduce 4-level translation latency by 74%?
Answer: 4-level Radix translation requires 4 DRAM accesses (PML4, PDPT, PD, PT). The PWC caches upper-level PML4 and PDPT pointers. On a PWC hit, the MMU skips levels 4 and 3, descending directly to the final page table in 1 memory lookup instead of 4 (`memory/VirtualMemory.hpp:345-365`).
Reference: Chapter H5

Q28: How does Booth's Radix-4 multiplier halve partial products?
Answer: By scanning 3 overlapping bits $(y_{2i+1}, y_{2i}, y_{2i-1})$, each 2-bit step is recoded into $\{0, \pm M, \pm 2M\}$. This reduces the number of addition steps from 32 down to 16 (`alu/Multipliers.hpp:80-120`).
Reference: Chapter G2

Q29: What is the FR-FCFS memory scheduling algorithm?
Answer: First-Ready First-Come First-Served: the memory controller prioritizes queued requests that target already-open DRAM row buffers over older requests targeting closed banks, converting 120-cycle row conflicts into 15-cycle row hits (`memory/DRAM.hpp:110-150`).
Reference: Chapter H4

Q30: What caused the 2.560 CPI discrepancy in older documentation?
Answer: The 2.560 table was a placeholder written before the cycle-accurate pipelines and caches were built. Furthermore, short benchmarks suffer from cold-start compulsory TLB/cache misses (120 cycles) that inflate apparent CPI. On extended benchmarks, steady-state CPI settles at **1.02 to 1.15** (`docs/CPI_AUDIT.md:1-50`).
Reference: Chapter J2

Category 4: Examiner Trap Questions (Answer Honestly!)

Q31: "Does your CPU support Out-of-Order execution?"
Answer: **No.** RISC201 is strictly an in-order processor. Instructions enter, execute, and retire in strict program order. We mitigate hazards using data forwarding, prefetching, and compiler scheduling rather than out-of-order reorder buffers (`processor_4stage/Pipeline4Stage.hpp`).

Q32: "Does your processor support multi-core cache coherence (MESI)?"
Answer: **No.** RISC201 models a high-performance single-core processor. Multi-core snooping and directory-based coherence protocols like MESI are outside the project scope (`memory/MemoryHierarchy.hpp`).

Q33: "Did you synthesize this processor on an FPGA?"
Answer: **No.** This project is a cycle-accurate software simulator written in pure C++17. The 500 MHz and 800 MHz frequencies are derived from standard 28nm standard-cell gate delay models (`docs/PERFORMANCE.md`).

Q34: "Can an instruction write to a register and memory in the same instruction?"
Answer: **No.** RISC201 is a strict Load-Store architecture. Arithmetic instructions write only to registers; `st` writes only to memory. This keeps instruction execution uniform and single-cycle (`common/ISA.hpp`).

Q35: "Why do you use string opcodes instead of integer enums?"
Answer: String opcodes (`isa::OP_ADD = "00000"`) prevent silent typecasting bugs, mirror physical 5-bit hardware wire lines, and serve as a single source of truth across the assembler, ISS, pipelines, and visualizers without translation layers (`common/ISA.hpp`). Verified by `./lint_isa`. # Mock Vivas & Final Preparation Checklist

Run through these 3 simulated oral examinations (vivas) with a peer or in front of a mirror before your presentation.

Mock Viva 1: The Fast-Paced Baseline Defense (10 Minutes)

- **Examiner**: "Welcome. In 30 seconds, explain what your project does."
  - **Model Answer**: "RISC201 is a cycle-accurate modern processor simulator written in pure C++17 modeling the SimpleRisc ISA. It compares a 4-stage core against an 800 MHz 6-stage core, backed by an advanced memory hierarchy including non-blocking caches, DDR4 DRAM, and 4-level Radix-4 Virtual Memory."
- **Examiner**: "Why did you separate instruction and data caches at L1?"
  - **Model Answer**: "To prevent the structural hazard where the `IF` stage fetches an instruction while the `MA` stage reads or writes data in the exact same clock cycle. Splitting L1 allows both stages to access memory simultaneously with zero stalls."
- **Examiner**: "Show me how `sub r1, r2, r3` executes in the 4-stage pipeline."
  - **Model Answer**: "Cycle 1: `IF` fetches the word from `PC`. Cycle 2: `OF` decodes opcode `'00001'` and reads `r2` and `r3` from the Register File. Cycle 3: `EX` subtracts `r3` from `r2` in the ALU. Cycle 4: `MA_RW` writes the result back into `r1`."
- **Examiner**: "What is the branch penalty in your 6-stage pipeline?"
  - **Model Answer**: "It is 2 cycles. Branches evaluate flags at `EX` (stage 4). If mispredicted, the two instructions fetched behind it in `IF` and `ID` are squashed into `nop` bubbles."

Mock Viva 2: Deep Microarchitecture & Memory Defense (15 Minutes)

- **Examiner**: "Why is your 6-stage pipeline faster in real time if its CPI is higher?"
  - **Model Answer**: "Because execution time depends on both CPI and clock frequency: $T = (IC \times \text{CPI})/f$. Splitting operand fetch into separate `ID` and `RF` stages cuts the critical path delay in half, allowing frequency to jump from 500 MHz to 800 MHz (a 1.6x increase). The tiny 0.5% CPI increase from the extra branch penalty cycle is overwhelmingly beaten by the 60% frequency boost, yielding a 1.59x wall-clock speedup."
- **Examiner**: "Explain how your memory controller handles a DRAM row conflict."
  - **Model Answer**: "When a request accesses an address in an already-active bank but in a different row, the controller must first issue a `PRECHARGE` command (t_{RP}) to close the active row, issue an `ACTIVATE` command (t_{RCD}) to latch the new row into the bank's Row Buffer, and then issue `READ` (t_{CL}). This takes ~120 cycles, compared to ~15 cycles for a row hit."
- **Examiner**: "What does your Page-Walk Cache do?"
  - **Model Answer**: "In a 4-level Radix page table, a TLB miss requires 4 sequential memory reads. The Page-Walk Cache caches intermediate PML4 and PDPT translation pointers. On a hit, the walker skips levels 4 and 3, descending to the final page table in 1 memory access instead of 4, cutting walk latency by 74%."

Mock Viva 3: Software Tools & Edge Cases (10 Minutes)

- **Examiner**: "How does your assembler handle forward branches?"
  - **Model Answer**: "It uses two passes. Pass 1 calculates the memory address of every label and records them in a symbol table. Pass 2 emits machine code words, using the recorded label addresses to compute relative offsets: `(target - PC) / 4`."

- **Examiner**: "What prevents an infinite recursive function from crashing the simulator?"
  - **Model Answer**: "The Stack Guard Page. We map an unreadable and unwritable page directly beneath the stack boundary. When recursive calls push `sp` past the stack limit, the MMU catches the access, throws a `STACK_OVERFLOW_EXCEPTION`, and halts cleanly without corrupting heap or code memory."
- **Examiner**: "How do you know your pipelines don't have subtle bugs?"
  - **Model Answer**: "We built an unpipelined Golden Reference ISS (`common/ISS.hpp`). Our regression test suite runs 192 configurations against the ISS, comparing every register and memory word. All 231 golden checks pass with 100% agreement, and AddressSanitizer confirms zero memory leaks."

Master "Can You Explain This?" Readiness Checklist

Tick each box once you can explain the concept out loud without looking at notes:

- ☐ **1. The Instruction Cycle**

  : Fetch → Decode → Execute → Writeback.
- ☐ **2. 4-Stage vs 6-Stage**

  : Where `OF` is split into `ID` and `RF`.
- ☐ **3. Why 6-Stage is Faster**

  : The math of 800 MHz vs 500 MHz (1.59 × speedup).
- ☐ **4. RAW Data Hazards**

  : How `EX->EX` and `MEM->EX` forwarding paths bypass data in 0 cycles.
- ☐ **5. Load-Use Delay**

  : Why a load followed immediately by its user requires 1 bubble.
- ☐ **6. Branch Misprediction**

  : How squashing turns wrong-path instructions into bubbles.
- ☐ **7. 2-Bit Bimodal Predictor**

  : How saturating counters prevent loop exit mispredicts.
- ☐ **8. Split L1 Caches**

  : Why separate ITLB/DTLB and L1I/L1D prevents structural hazards.

- ☐
  9. DRAM Row Hit vs Conflict

  : Why row hits take 15 cycles while conflicts take 120 cycles.
- ☐
  10. 4-Level Radix-4 Translation

  : How a 48-bit address is sliced into PML4, PDPT, PD, PT, and Offset.
- ☐
  11. Page-Walk Cache

  : Why caching upper directory pointers cuts walk latency by 74%.
- ☐
  12. Two-Pass Assembler

  : Why Pass 1 is needed to resolve forward labels.
- ☐
  13. Full-Descending Stack

  : Why `push` decrements `sp` and `pop` increments `sp`.
- ☐
  14. Stack Guard Page

  : How boundary exceptions prevent memory corruption.
- ☐
  15. Single Source of Truth

  : Why all opcodes are 5-bit strings in `common/ISA.hpp`.
- ☐
  16. The Cold-Start CPI Phenomenon

  : Why short benchmarks show 6.1 CPI while long benchmarks show 1.12 CPI.
- ☐
  17. Horizontal vs Vertical Microcode

  : 32-bit unencoded vs 16-bit encoded (50% ROM reduction).
- ☐
  18. Golden ISS

  : How differential testing verifies pipelined execution correctness.
- ☐
  19. Live Demo Commands

  : Ability to run `make demo-pipe`, `make demo-mem`, `make demo-vm` smoothly.
- ☐
  20. Project Limitations

  : Confidently stating that the processor is in-order single-core without pretending otherwise.